
TRACKING AND EVASION USING CO-TRAINING WITH CONTEXT KNOWLEDGE

OGUZ ALTAN

CHAIR OF INFORMATION THEORY AND DATA ANALYTICS
RWTH AACHEN UNIVERSITY

TRACKING AND EVASION USING CO-TRAINING WITH CONTEXT KNOWLEDGE

OGUZ ALTAN

A thesis submitted to the
Faculty of Electrical Engineering and Information Technology,
RWTH Aachen University,

reviewed by Univ.-Prof. Dr. Ing. Anke Schmeink and supervised by Paul Zheng, André
Brandenburger and Dr. Alexander Charlish

CHAIR OF INFORMATION THEORY AND DATA ANALYTICS
RWTH AACHEN UNIVERSITY

Eidesstattliche Versicherung

Name, Vorname

Matrikelnummer

Ich versichere hiermit an Eides Statt, dass ich die vorliegende Masterarbeit mit dem Titel

Tracking and Evasion using Co-Training with Context Knowledge

selbständig und ohne unzulässige fremde Hilfe erbracht habe. Ich habe keine anderen als die angegebenen Quellen und Hilfsmittel benutzt. Für den Fall, dass die Arbeit zusätzlich auf einem Datenträger eingereicht wird, erkläre ich, dass die schriftliche und die elektronische Form vollständig übereinstimmen. Die Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Aachen, den 13th September 2023

Ort, Datum

Unterschrift

Belehrung:

§ 156 StGB: Falsche Versicherung an Eides Statt

Wer vor einer zur Abnahme einer Versicherung an Eides Statt zuständigen Behörde eine solche Versicherung falsch abgibt oder unter Berufung auf eine solche Versicherung falsch aussagt, wird mit Freiheitsstrafe bis zu drei Jahren oder mit Geldstrafe bestraft.

§ 161 StGB: Fahrlässiger Falscheid; fahrlässige falsche Versicherung an Eides Statt

(1) Wenn eine der in den §§ 154 bis 156 bezeichneten Handlungen aus Fahrlässigkeit begangen worden ist, so tritt Freiheitsstrafe bis zu einem Jahr oder Geldstrafe ein.

(2) Strafflosigkeit tritt ein, wenn der Täter die falsche Angabe rechtzeitig berichtigt. Die Vorschriften des § 158 Abs. 2 und 3 gelten entsprechend

Die vorstehende Belehrung habe ich zur Kenntnis genommen:

Aachen, den 13th September 2023

Ort, Datum

Unterschrift

Abstract

In the evolving technological landscape, autonomous systems driven by Artificial Intelligence (AI) have gained importance. Among these, Unmanned Aerial Vehicles (UAVs) exemplify the synergy of AI and robotics, finding applications in diverse domains, from surveillance to disaster response. This thesis addresses a specific challenge: optimizing UAV trajectories for precise target tracking within urban environments. It employs Reinforcement Learning (RL) with a game-theoretic co-training approach, simultaneously training an evading target and a tracking UAV. This approach promotes diverse target strategies and mitigates the risk of observer policy overfitting. While previous research has focused on empty environments, this thesis introduces urban environments into the RL framework, enhancing realism and practicality. Urban environments, containing buildings and roads, offer a more authentic and realistic context compared to empty environments. To ensure effective learning and diversity, a procedural map generation technique is employed to generate maps, resembling typical urban layouts. The procedural map generation introduces a novel dimension to this research and complements the utilization of context knowledge, referring to the knowledge introduced by the representation of the urban environments with maps. This context knowledge, extracted from the maps using Convolutional Neural Networks (CNNs) which are integrated into RL models, enriches agent observations regarding urban environments. The thesis evaluates the performance of the agents under various scenarios with different maps and observation spaces. The results are assessed by analyzing the utilization of context knowledge, both alone and in combination with state observations. These assessments include evaluations of target evasion and observer tracking performance, as well as the presentation of exemplary agent trajectories.

Acknowledgements

I extend my heartfelt appreciation to André Brandenburger for his unwavering dedication and guidance throughout my thesis journey. His consistent support and close collaboration have played a pivotal role in shaping the direction of this work. His insights and expertise have been an invaluable asset, and I am indebted for his continuous involvement.

I would like to express my sincere gratitude to Paul Zheng for his insightful comments, feedback, and valuable contributions to my thesis. His thoughtful perspectives and inputs significantly enriched the quality of this work. His generosity in sharing his thoughts and knowledge has been essential in refining my ideas.

I also extend my gratitude to Univ.-Prof. Dr. Ing. Anke Schmeink and Dr. Alexander Charlish. I am truly thankful for the opportunity they provided me to delve into this subject and for their unwavering belief in my potential. I am honored to have had the privilege of working under their guidance.

To my beloved family, your support and encouragement have been my anchor throughout this journey. Your belief in me has fueled my determination to overcome challenges and succeed. *Canım ailem, iyi ki varsınız!*

Contents

Acronyms	xvii
Notation glossary	xix
1. Introduction	1
1.1. Motivation	1
1.2. Aims	2
1.3. Thesis structure	2
2. Background	5
2.1. Mathematical foundations	5
2.1.1. Centroid of a polygon in 2D	5
2.1.2. Multivariate normal distribution	5
2.2. Deep learning	6
2.2.1. Overview	6
2.2.2. Convolutional neural networks	9
2.2.3. Optimization of CNN and hyperparameter tuning	11
2.3. Reinforcement learning	12
2.3.1. Single-agent reinforcement learning	12
2.3.2. Multi-agent reinforcement learning	14
2.3.3. Deep reinforcement learning	16
2.3.4. RL algorithms	16
2.3.5. Proximal policy optimization	17
3. Related work	21
3.1. Path planning in MARL	21
3.1.1. Traditional path planning	21
3.1.2. CNN-based path planning	22
3.1.3. RL-based path planning	22
3.1.4. CNN-enhanced RL for path planning	24
3.2. Procedural map generation	27
4. Co-training an observer and an evading target	31
4.1. Introduction	31
4.2. Tracking framework	34

4.3. Agent models	35
4.3.1. Observation spaces	35
4.3.2. Action spaces	37
4.3.3. Reward functions	37
4.3.4. Termination criteria	38
4.4. Policy explanations	38
4.5. Novel contributions and advancements to original paper	39
5. Procedural map generation	41
5.1. Basic maps	41
5.2. Voronoi diagram-based maps	44
5.2.1. Generating points and partition	44
5.2.2. Road network	45
5.2.3. Distance-based building sampling with simplex noise	47
6. Exploiting context knowledge and awareness in MARL	53
6.1. Fundamentals of context knowledge	53
6.1.1. Coordinate system transformation	53
6.1.2. Environment boundary constraint	54
6.1.3. Building collision	55
6.1.4. Adaptive target speed	57
6.1.5. Initial positions	57
6.1.6. Occlusion	57
6.2. Expanding observation spaces	60
6.2.1. State-related observations	60
6.2.2. Context-related observations	62
6.2.3. Forming context-related observations	63
6.3. Utilizing CNN in actor-critic RL model	65
6.3.1. Processing context-related observations with a CNN	65
6.3.2. CNN-integrated RL model	65
7. Results and evaluations	67
7.1. Procedural map generation	67
7.2. Descriptions of tasks	68
7.2.1. Task A: Escape	68
7.2.2. Task B: Tracking and evasion	70
7.3. Results	72
7.3.1. Task A: Escape	72
7.3.2. Task B: Tracking and evasion	84
8. Conclusion and future work	95
A. Appendix	99
Bibliography	105

List of Tables

7.1. Escape scenarios and tracking and evasion scenarios	72
7.2. Collision rate and distance results obtained after 20 million time steps of training for A.2.1, A.2.2, A.2.3 and A.2.4	83
7.3. Observer mean rewards obtained at the end of first 50 million time steps of training of B.1.1, B.1.2, B.2.1 and B.2.2	93
A.1. RL model parameters	99
A.2. PPO parameters	99
A.3. Map parameters	99
A.4. Agent parameters	99

List of Figures

2.1. Artificial neuron model	6
2.2. Multilayer perceptron	8
2.3. Various ANN activation functions	8
2.4. Exemplary CNN architecture	9
2.5. Exemplary 2D convolution operation on an image	10
2.6. Illustrations of pooling operations	11
2.7. Interaction of an agent and an environment in an MDP	13
2.8. Stochastic game in MARL	15
2.9. Taxonomy of RL algorithms	18
3.1. Street creation system applied to Manhattan	28
3.2. Wave function collapse method	29
3.3. Voronoi diagram	29
3.4. Comparison of the white noise, Perlin noise, and simplex noise	30
4.1. An overview of a classical UAV control task using RL	33
4.2. Visualization of the observation space of the original work	36
4.3. Actor critic network model of the original work	36
4.4. Decision tree generated using artificial observations	39
5.1. Binary maps with three different configurations	42
5.2. Binary map, height mask and basic map	43
5.3. Three generating point sampling configurations	46
5.4. Effect of upscaling factor λ on line segment approximation	47
5.5. Voronoi edges and resulting road network	48
5.6. Scalar and 2D views of the polygonwise noise threshold function	50
5.7. Complete Voronoi map with sampled buildings and road network	51
5.8. Voronoi diagram-based map generation pipeline	52
6.1. Environment and map	54
6.2. Building collision	56
6.3. 2D and 3D illustrations of the occlusion	58
6.4. Illustrations of the occlusion thresholds	60
6.5. Illustration of a position map	63
6.6. Rotation map	64
6.7. Architecture of the CNN	65

6.8. CNN-integrated actor-critic RL model	66
7.1. Basic map and Voronoi diagram-based map	68
7.2. Trajectory plots of A.1: Basic map with context-related observations	73
7.3. Collision rate and distance plots of A.1	74
7.4. Trajectory plots of A.2.1: Voronoi diagram-based map with context-related	75
7.5. Collision rate and distance plots of A.2.1	75
7.6. Trajectory plots of A.2.2: Voronoi diagram-based map with context-related and state-related observations	76
7.7. Collision rate and distance plots of A.2.2	76
7.8. Collision rate and distance plots of A.2.1 and A.2.2	77
7.9. Trajectory plots of A.2.3: Voronoi diagram-based map and the road map, with context-related observations	78
7.10. Collision rate and distance plots of A.2.3	79
7.11. Trajectory plots of A.2.4: Voronoi diagram-based map and road map with context-related and state-related observations	80
7.12. Collision rate and distance plots of A.2.4	80
7.13. Collision rate and distance plots of A.2.3 and A.2.4	81
7.14. Collision rate and distance plots of A.2.1, A.2.2, A.2.3 and A.2.4	82
7.15. Trajectory plots of B.1.1: Basic map with context-related observations . . .	85
7.16. Mean observer reward and occlusion rate plots of B.1.1	86
7.17. Trajectory plots of B.1.2: Basic map with context and state-related obs . .	87
7.18. Mean observer reward and occlusion rate plots of B.1.2	87
7.19. Mean observer reward plots of B.1.1 and B.1.2	88
7.20. Trajectory plots of B.2.1: Voronoi diagram-based map with context-related observations	89
7.21. Mean observer reward and occlusion rate plots of B.2.1	89
7.22. Trajectory plots of B.2.2: Voronoi diagram-based map with context-related and state-related observations	90
7.23. Mean observer reward and occlusion rate plots of B.2.2	91
7.24. Mean observer reward plots of B.2.1 and B.2.2	91
7.25. Collision rate and distance plots of the target in B.1.1, B.1.2, B.2.1 and B.2.2	91
7.26. Mean observer reward plots of B.1.1, B.1.2, B.2.1 and B.2.2	92
A.1. More trajectory plots of the scenario A.1	100
A.2. More trajectory plots of the scenario A.2.1	100
A.3. More trajectory plots of the scenario A.2.2	101
A.4. More trajectory plots of the scenario A.2.3	101
A.5. More trajectory plots of the scenario A.2.4	102
A.6. More trajectory plots of the scenario B.1.1	102
A.7. More trajectory plots of the scenario B.1.2	103
A.8. More trajectory plots of the scenario B.2.1	103
A.9. More trajectory plots of the scenario B.2.2	104

Acronyms

AI Artificial Intelligence vii, 1

ANN Artificial Neural Network xv, 6, 8, 9

CI Computational Intelligence 22

CNN Convolutional Neural Network xv, xvi, 9, 11, 16, 21, 22, 24–27, 40, 60, 65, 66, 95, 96

DL Deep Learning 6, 8, 16, 25

DRL Deep Reinforcement Learning 16, 23–26

EKF Extended Kalman Filter 33, 34, 55, 58, 85

FOV Field of View 31, 34, 58

GAE Generalized Advantage Estimation 19

MARL Multi-Agent Reinforcement Learning xv, 14, 15, 25, 32, 95

MDP Markov Decision Process xv, 12–15

ML Machine Learning 1, 6, 8, 12, 16

MLP Multilayer Perceptron 7, 11

PPO Proximal Policy Optimization 17–20, 23, 35, 65, 72

RL Reinforcement Learning xv, xvi, 1, 2, 12, 14, 16–18, 21, 23–27, 33, 35, 37, 38, 40, 47, 53, 55, 65–67, 72

SARL Single-Agent Reinforcement Learning 12, 15

SL Supervised Learning 12, 22, 24

SLAM Simultaneous Localization and Mapping 25

UAV Unmanned Aerial Vehicle xv, 23, 26, 31, 33–36

UL Unsupervised Learning 12

Notation glossary

$n \in \mathbb{N}$	Natural number
$a \in \mathbb{R}$	Real number, scalar
$\mathbf{a} \in \mathbb{R}^n$	Real vector of size n
$\mathbf{a} \in \mathbb{R}^{n \times m}$	Real matrix of size $n \times m$
$:=$	Is defined as
$a_i, [\mathbf{a}]_i$	i th element in vector $\mathbf{a}$
$a_{ij}, [\mathbf{a}]_{ij}$	Element at i th row and j th column in matrix $\mathbf{a}$
$\mathbf{a}_{i:}, [\mathbf{a}]_{i:}$	i th row in matrix $\mathbf{a}$
$\mathbf{a}_{:j}, [\mathbf{a}]_{:j}$	j th column in matrix $\mathbf{a}$
$ \mathcal{I} $	Cardinality of set $\mathcal{I}$
$\mathbf{I}_n$	Identity matrix of size $n \times n$
$\mathbf{0}_n, \mathbf{0}_{n \times m}$	Zero-vector of size n or zero-matrix of size $n \times m$
$\mathbf{1}_n, \mathbf{1}_{n \times m}$	One-vector of size n or one-matrix of size $n \times m$
$\mathbf{A}^\top$	Transposed matrix
$\mathbf{A}^{-1}$	Matrix inverse
$\mathbf{A} \odot \mathbf{B}$	Hadamard product
$\text{diag}(\mathbf{a})$	Diagonal matrix with elements of $\mathbf{a}$
$ \cdot $	Absolute value
$\ \cdot\ _2$	Euclidean norm
$a \wedge b$	Logical AND
$a \vee b$	Logical OR

1. Introduction

In today’s rapidly evolving technological landscape, the autonomous systems empowered by AI gained importance. Among these, Unmanned Aerial Vehicles (UAVs) stand as an example of the merging of AI and robotics. UAVs have been used in various fields, from surveillance to disaster response. A specific challenge addressed in this thesis revolves around the optimization of UAV flight paths to enable precise target tracking within urban environments.

1.1. Motivation

Reinforcement Learning (RL) is a tool within the domain of Machine Learning (ML) and it is employed to enable policies to be trained by autonomous agents for potentially solving complex tasks. It has found widespread application in the field of robotics. Consequently, the perspective of RL is applied to address the objective of target tracking. The UAV is tasked with tracking a mobile ground-based target using a range-bearing sensor. The goal of the UAV is not to physically reach the target but rather to achieve the most accurate target localization.

In this work, a game-theoretic approach is adopted, where the target’s intent is assumed to be the maximization of tracking performance degradation. Instead of modeling specific target trajectories, the focus is on considering the worst-case scenario of an evading target. Training an observer policy under this worst-case assumption is expected to lead to improved tracking performance for various target behaviors.

Thus, a parallel training approach, termed *co-training*, is adopted. This approach entails simultaneous training of the evading target’s policy alongside the policy of the tracking UAV. The target is considered as another agent with the objective of maximizing the degradation of the observer’s tracking performance. Through training in this adversarial manner, overfitting of the observer policy to the target’s current behavior is avoided, thus incentivizing the exploration of various strategies by the target.

In the work [1] by Brandenburger et al., the authors focus on the approaches mentioned above for the task of tracking and evasion, and they provide valuable insights into the behavior of agents in an empty environment.

Enhancing the realism and practicality of the presented RL task by substituting the empty environment with an urban environment, containing urban map components such as buildings and roads, introduces new challenges and distinct characteristics stemming from the contextual information associated with urban environments. This novelty not only reflects a more realistic scenario but also introduces tangible elements into the environment.

1.2. Aims

The main aim of this thesis is to extend the existing co-training approach to urban environments. Therefore, maps that include buildings and roads are generated in order to represent urban environments. The inclusion of buildings and roads necessitates the implementation of precise simulations accounting for the physical constraints that influence agent behavior within these environments.

This contextual information informed by the urban maps gives rise to a distinct type of knowledge, referred to as *context knowledge*. It constitutes an additional set of observations for the agents, conveying information about the map and its components. Without context knowledge, the agents can not be aware of the complete content of the environment.

Given that maps are essentially images, image processing techniques, such as CNN, are employed to extract the context knowledge from the maps. This knowledge is then included as an additional set of observations for the agents, presenting information about the urban environments that they perform within. The tracking and evasion behaviors of the agents are then evaluated under various scenarios where they are provided with context knowledge.

To ensure effective learning and good generalization abilities, agents are provided new maps in each RL episode. This necessitates the generation of a substantial number of maps. Instead of manually creating maps, a technique known as *procedural generation* is employed. Procedural generation enables the random generation of maps while preserving the similarity with typical urban maps and ensuring diversity among these maps. Consequently, procedural generation becomes a key element and research focus of this project, and it introduces a novel aspect, similar to context knowledge, which complements and enhances the work presented in the original paper.

1.3. Thesis structure

The thesis is structured into several chapters. Chapter 2 is dedicated to laying the mathematical foundations and exploring the technical background of subjects relevant to this work.

Chapter 3 provides a comprehensive review of related works, offering a broader context for the research. Chapter 4 provides an overview of the paper *Co-Training an Observer and an Evading Target* [1] which constitutes the foundational basis of this thesis. Chapter 5 delves into the intricacies of procedural map generation and presents the characteristics of the maps generated through this technique. Chapter 6 discusses the integration of the context knowledge, introduced by the urban maps, into the tracking and evasion problem. In Chapter 7, the tasks undertaken and the results related to the tracking and evasion problem, with the integration of the context knowledge, are presented and critically evaluated. Chapter 8 serves as the concluding chapter, summarizing the work accomplished in this thesis while also suggesting promising directions for future research.

2. Background

2.1. Mathematical foundations

2.1.1. Centroid of a polygon in 2D

Let $\{(x_i, y_i)\}_{i=0}^{N-1} \subset \mathbb{R}^2$ be a closed polygon in the plane, and let the vertices be ordered counterclockwise. Then it is known that the polygon encloses the area

$$A = \frac{1}{2} \sum_{i=0}^{N-1} (x_i y_{i+1} - x_{i+1} y_i) , \quad (2.1.1)$$

and its centroid [2] is given by

$$\frac{1}{6A} \left(\sum_{i=0}^{N-1} (x_i + x_{i+1}) (x_i y_{i+1} - x_{i+1} y_i), \sum_{i=0}^{N-1} (y_i + y_{i+1}) (x_i y_{i+1} - x_{i+1} y_i) \right)^T \in \mathbb{R}^2 . \quad (2.1.2)$$

2.1.2. Multivariate normal distribution

The multivariate normal distribution is mathematically defined such that

$$f(\mathbf{x}|\mu, \Sigma) = \frac{1}{\sqrt{(2\pi)^k |\Sigma|}} \exp \left(-\frac{1}{2} (\mathbf{x} - \mu)^T \Sigma^{-1} (\mathbf{x} - \mu) \right) , \quad (2.1.3)$$

where

- $\mathbf{x}$ is the vector of sampled points.
- μ is the mean vector, indicating the center of the distribution.

- Σ is the covariance matrix, controlling the shape and orientation of the distribution.
- $|\Sigma|$ represents the determinant of the covariance matrix.
- n denotes the dimensionality of the multivariate normal distribution.

2.2. Deep Learning (DL)

2.2.1. Overview

Deep Learning (DL) constitutes a subset of ML techniques centered on learning data representations through the utilization of the Artificial Neural Network (ANN) [3].

ANNs originated as a graph of boolean functions, subsequently evolving into a versatile pattern-matching model known as the *perceptron* [4]. Subsequent progress led to the development of a more intricate network model incorporating multiple layers. However, the widespread adoption of large neural networks was primarily facilitated by the introduction of the backpropagation algorithm and the remarkable surge in computing power over the years, beginning from the early stages of computing [5].

Artificial neural model and multilayer perceptron

A standard ANN comprises numerous connected processing units referred to as *neurons*, with each neuron generating a sequence of real-valued activations [6]. Input neurons receive activation through sensors that perceive the surrounding environment, while other neurons receive activation through weighted connections from previously active neurons. Certain neurons might impact the environment by initiating actions. The process of learning involves identifying weights that enable the ANN to demonstrate the desired behavior.

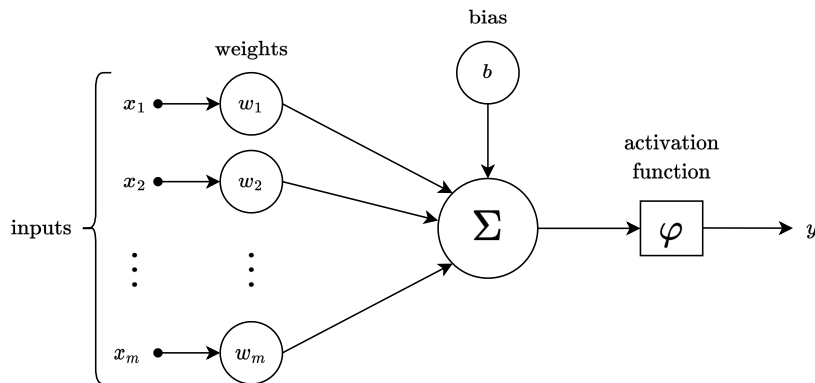


Figure 2.1.: Artificial neuron model.

The output y of this artificial neuron model, consisting of input $\mathbf{x} \in \mathbb{R}^m$ and weights $\mathbf{w} \in \mathbb{R}^m$ can be calculated as

$$y = \varphi(x_1w_1 + x_2w_2 + \cdots + x_mw_m + b) = \varphi\left(\sum_{i=1}^m x_iw_i + b\right), \quad (2.2.1)$$

where φ denotes the activation function that is used to add nonlinearity to the model and b is the offset parameter called *bias*. When an input variable $x_0 = 1$ is introduced, the bias can be integrated as an element of the input, resulting in a simplified form

$$y = \varphi\left(\sum_{i=0}^m x_iw_i\right). \quad (2.2.2)$$

An architecture comprising a single layer of input neurons transmitting information to an output layer of neurons, characterized by full connectivity, is termed a *perceptron*. Furthermore, a Multilayer Perceptron (MLP) denotes a neural network equipped with one or more hidden layers. Within each layer, every neuron in a layer is connected to all neurons in the previous layer.

The output $o_{l,k}$ of the k^{th} neuron at layer l , and its corresponding activation value can be calculated using the weight $w_{k',k}^l$ associated with the connection from neuron k' at layer $l-1$ to that neuron such that

$$\begin{aligned} po_{l,k} &= \sum_{k'=0}^{K_{l-1}} a_{l-1,k'} w_{k',k}^l \\ a_{l,k} &= \varphi(o_{l,k}), \end{aligned} \quad (2.2.3)$$

where $\forall l \in \{0, \dots, L\}, k \in \{0, \dots, K_l\}$. The number of the neurons at layer l is denoted as $K_l + 1$, and φ is the activation function. Using vectorized notation, the output of the layer l can be computed as $\mathbf{o}_l = \mathbf{a}_{l-1} \mathbf{w}^l$. The input of the MLP is $\mathbf{x} = \mathbf{o}_0$ and the output $\mathbf{y}$ is calculated as $\mathbf{y} = \mathbf{a}_L = \varphi(\mathbf{o}_L)$.

MLP exhibits an intricate network of interconnections, rendering its structure more intricate compared to perceptrons. This complexity enables the MLP to address more intricate tasks, distinguishing it from the capabilities of a perceptron [7].

2. Background

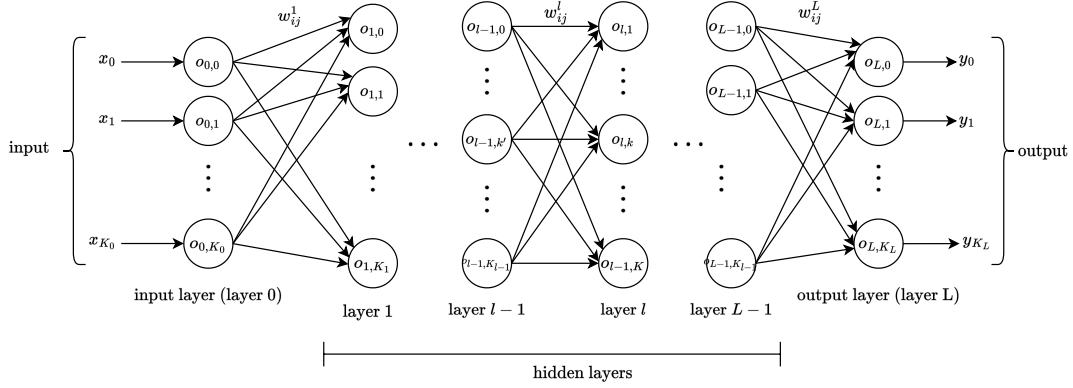


Figure 2.2.: Illustration of the multilayer perceptron.

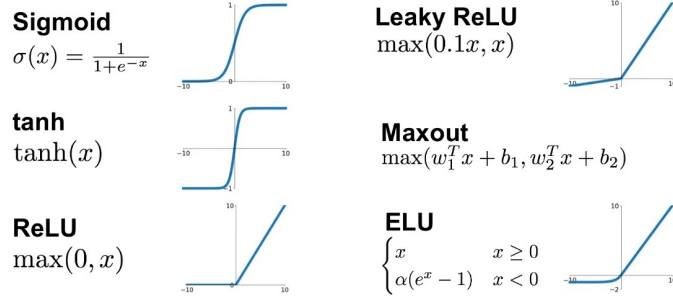


Figure 2.3.: Various ANN activation functions [10].

Deep vs. shallow networks

Traditional ML often employs shallow networks, consisting of an input and an output layer, with at most one hidden layer situated between them. In contrast, DL is characterized by networks with more than three layers, encompassing both input and output layers. Hence, as the number of hidden layers increases, the network's depth intensifies, defining its qualification as a deep neural network [3].

DL achieves intricate nonlinearity by employing numerous nonlinear activation functions. The motivation for this multilayer architecture stems from the observation that real-world datasets, like images, encompass distinct tiers of features, where lower-level features lay the foundation for higher-level constructs [8]. Depth in networks efficiently extracts representative features from datasets, contributing to better performance. In contrast, augmenting the width (e.g., the number of basis functions) in a shallow model offers limited enhancements. This implies that deep learning models are adept at capturing complex datasets by inhabiting a function space distinct from that of shallow models [9].

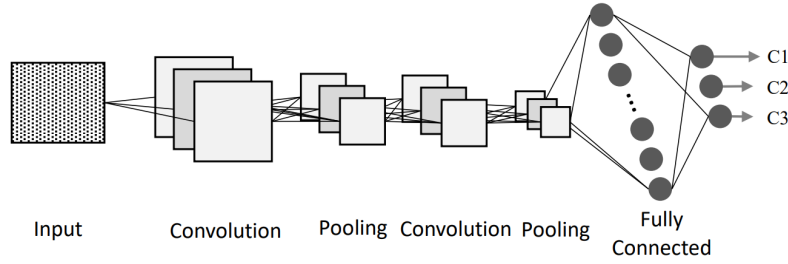


Figure 2.4.: Exemplary CNN architecture consisting of convolutional, pooling and fully connected layers [12].

2.2.2. Convolutional Neural Network (CNN)

A Convolutional Neural Network (CNN) is a specialized type of ANN designed for analyzing data with distinct spatial patterns, particularly images [9]. Convolutional Neural Network (CNN)s are predominantly employed in tasks related to pattern recognition within images. This design choice enables CNNs to encode image-specific features directly into their architecture, rendering them well-suited for image-related tasks. This architectural advantage also contributes to a reduction in the overall number of parameters required for model setup [11]. An intriguing observation is that CNNs trained exclusively on raw inputs, such as pixel data extracted from images, can yield informative data representations. This suggests that post-training, the individual units within CNNs are capable of encoding fundamental features like edges, corners, wheels, and eyes [9].

CNNs typically consist of three types of layers, which are convolutional layers, pooling layers, and fully-connected layers [11].

Convolutional layers

A convolutional layer comprises multiple filters, referred to as *kernels*, which are typically spatially smaller than the input images but have greater depth. This type of layer executes a mathematical operation known as *convolution* between two matrices, which are the kernel and the confined section of the image's receptive field. Kernels represent the set of learnable parameters within a convolutional layer. Convolution involves sliding or convolving the kernel across the input image, conducting element-wise multiplications, and consolidating the outcomes to produce a feature map. These feature maps contain extracted features from the input images, as implied by their name.

The *depth* of a kernel indicates the number of channels it operates on within the input. In a convolutional layer, numerous kernels are employed on the input, generating distinct channels in the resulting feature map. This design allows the network to capture diverse features and patterns across various channels [11].

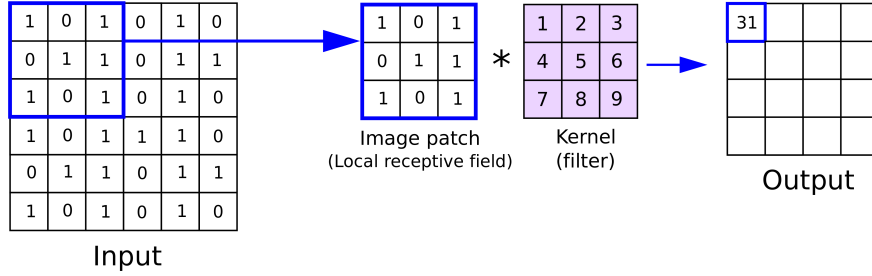


Figure 2.5.: Exemplary 2D convolution operation on an image [13].

The *stride* signifies the increment by which the kernel shifts or slides across the input image during convolution. A greater stride value results in a diminished spatial dimension in the output feature map, while a smaller stride value maintains more spatial detail. For instance, with a stride of 1, the kernel advances pixel by pixel, whereas a stride of 2 corresponds to a two-pixel advancement. The choice of stride influences the pace of output size reduction as convolutions are performed [9].

Zero-padding is a method that involves appending additional rows and columns of zeros to the periphery of the input. This technique proves useful in handling spatial dimensions and regulating the resultant size of feature maps [11].

By adjusting parameters such as depth, stride, zero-padding, and receptive field, the dimensions of the derived feature maps and the encapsulated information can be further controlled and modified.

Pooling layers

Pooling layers serve the purpose of gradually diminishing the dimensionality of the representation by aggregating data from nearby pixels, leading to downsampling. This reduction in dimensionality contributes to a decrease in parameters and computational complexity within the model. It is important to note that the depth dimension of the volume remains unaltered [11].

Max-pooling, a common pooling operation, involves partitioning the feature map into non-overlapping regions and selecting the maximum value within each region. This process results in a downsampled feature map composed of these selected maximum values. Max pooling is effective in retaining critical features while concurrently decreasing computational complexity [14].

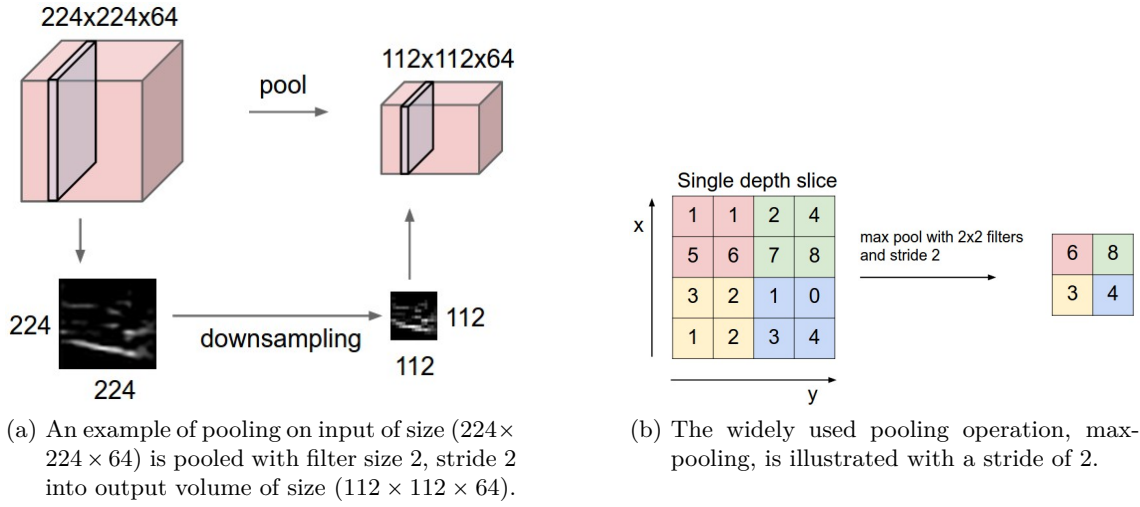


Figure 2.6.: Illustrations of pooling operations [15]. These pooling layers perform spatial downsampling on the input volume, independently for each depth slice.

Fully connected layers

Fully connected layers consist of neurons directly connected to neurons in the adjacent layers, similar to the arrangement in MLPs (see Section 2.2). These layers process and transform features from earlier layers, eventually leading to predictions or classifications [16].

2.2.3. Optimization of CNN and hyperparameter tuning

Optimization and hyperparameter tuning of CNNs are utilized for improving the performance of the CNN, as well as decreasing the training time, by simplifying the whole network architecture and tuning the parameter to fit better with the current goal. Various strategies employed in this work include [17]:

- **Using smaller kernel sizes:** Smaller kernel sizes (such as 3×3) require fewer computations and therefore can lead to faster training times compared to larger kernel sizes (such as 5×5 or 7×7).
- **Reducing the number of layers:** A CNN with many layers can be computationally expensive and take a long time to train. The architecture of the CNN can be simplified by reducing the number of layers or removing unnecessary layers that do not contribute significantly to the performance.
- **Using pooling layers:** Pooling layers can reduce the size of feature maps and help to reduce the number of parameters in the model. Max pooling layers are commonly used to downsample feature maps by taking the maximum value of each pool.

2.3. Reinforcement Learning (RL)

The Reinforcement Learning (RL) constitutes a subset of ML focused on enabling an *agent* to acquire decision-making capabilities through iterative interactions with an *environment* [18]. In RL, the fundamental goal is for the agent to acquire a *policy*, which is a function mapping *states* to *actions*, with the intention of maximizing an accumulative *reward* over the long term [19]. This paradigm is particularly applicable to scenarios involving sequential decision-making, therefore the notion of time and dynamics is essential. The objective is to ensure that the agent attains an optimal policy by responding to the reward signal. This entails reinforcing favorable behaviors while discouraging undesirable actions, thus earning the nomenclature **reinforcement learning** [20].

ML paradigms are categorized into three main types: Supervised Learning (SL), Unsupervised Learning (UL), and RL. In SL, a model is trained to map inputs to outputs using labeled examples provided by an expert. UL, on the other hand, focuses on identifying patterns and relationships in unlabeled data without explicit expert guidance. Techniques such as clustering and association are commonly employed in this context.

In contrast to SL and UL, RL doesn't rely on pre-labeled input-output pairs or the discovery of latent structures within data. Instead, it acquires knowledge through interactions with the environment and the rewards obtained therein. The RL agent learns to make decisions that maximize cumulative rewards, without prior knowledge of the optimal actions.

2.3.1. Single-Agent Reinforcement Learning (SARL)

The Single-Agent Reinforcement Learning (SARL) constitutes a framework involving an agent interacting with its environment. Markov Decision Process (MDP) is a way to mathematically model decision-making within a stochastic environment [21]. In SARL, the problem can be represented as an Markov Decision Process (MDP). An MDP is comprised of various components, including a set of states and actions, a state transition probability function that determines the probability of moving from one state to another when a certain action is taken, and a reward function that specifies the reward gained for taking a particular action while in a given state.

The MDP offers a mathematical framework for formalizing decision-making in stochastic environments [21]. In the context of SARL, the problem can be effectively represented using an MDP. An MDP comprises distinct elements, including a collection of states and actions, a state transition probability function that quantifies the probability of transitioning from one state to another when a specific action is executed, and a reward function that delineates the reward obtained for undertaking a particular action in a given state.

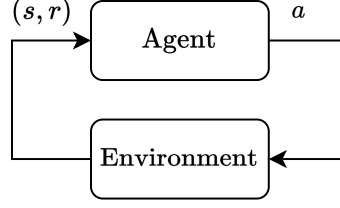


Figure 2.7.: Interaction of an agent and an environment in an MDP.

An MDP is defined by a tuple $(\mathcal{S}, \mathcal{A}, R, P)$, consisting of the set of states $\mathcal{S}$, the set of actions $\mathcal{A}$, the state transition probability function $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ and the reward function $R : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$. A state $s_t \in \mathcal{S}$ describes the state of the environment, an action $a_t \in \mathcal{A}$ describes the action taken by the agent, and the reward function $R(s_t, a_t, s_{t+1}) = r_t$ defines a reward.

An MDP satisfies the Markov property, i.e. at time step t , the future state s_{t+1} is only related to the state s_t and the action a_t of the present [22].

The interaction between the agent and the environment unfolds in a sequence of discrete time steps. At each time step t , the agent observes the state $s_t \in \mathcal{S}$. Based on the state, the agent chooses an action $a_t \in \mathcal{A}$. The environment reacts by transitioning to a new state $s_{t+1} \in \mathcal{S}$. The probability of transitioning from the current state s_t to the next state s_{t+1} under the selected action a_t is denoted as $P(s_{t+1} | s_t, a_t)$.

Subsequently, the agent receives a reward $r_t = R(s_t, a_t, s_{t+1})$. In the context of a finite MDP with a horizon of length T , the agent and the environment collaborate to establish a trajectory which is a sequence presented as

$$\tau = (s_0, a_0, s_1, r_0), (s_1, a_1, s_2, r_1), \dots, (s_{T-1}, a_{T-1}, s_T, r_{T-1}) . \quad (2.3.1)$$

A policy outlines the approach of an agent to select an action based on a given state. In the context of stochastic behavior, the agent learns a policy, which is denoted by $\pi : \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$. The probability for the agent to perform an action a_t in state s_t is denoted as $\pi(a_t | s_t)$. This policy is optimized during training to maximize the cumulative reward, also called return $G(\tau)$, obtained over a finite horizon of length T . The return is the sum of rewards at each time step, indicated as

$$G(\tau) = \sum_{t=0}^{T-1} r_t . \quad (2.3.2)$$

2. Background

In RL, the return formulation is adapted to the discounted return, introducing a discount factor $\gamma \in [0, 1]$ such that

$$G(\tau) = \sum_{t=0}^{T-1} \gamma^t r_t. \quad (2.3.3)$$

The discount factor γ determines the relative importance attributed to rewards in the present compared to those in the future. A value of γ near 0 prioritizes immediate rewards, while a value close to 1 places greater weight on future rewards when striving to maximize the overall return.

The expected long-term cumulative discounted reward starting in state s_t while following policy π is defined by the *value function* $V^\pi(s) : S \rightarrow \mathbb{R}$ such that

$$V^\pi(s) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid a_t = \pi(s_t) \right]. \quad (2.3.4)$$

The goal of RL is to choose a policy that yields the maximized expected return when the agent follows it, which implies finding the policy π^* that maximizes $V^\pi(s_0)$ for starting state s_0 [20]. Therefore, value functions are inherently coupled to the goal of RL and the value function evaluates how well the policy π performs w.r.t. the goal of RL. The optimal state-value function is presented as

$$V_*(s) = \max_{\pi} V^\pi(s) \text{ for all } s \in S. \quad (2.3.5)$$

2.3.2. Multi-Agent Reinforcement Learning (MARL)

In the domain of Multi-Agent Reinforcement Learning (MARL), several agents coexist within a shared environment, learning through an iterative process of trial and error. Each agent operates based on its distinct states and rewards, undertaking actions to advance its own objectives. These multiple agents not only engage with the environment but also interact with each other through their chosen actions. The overarching goal of MARL is to acquire individual or collaborative policies that collectively maximize the cumulative reward for all participating agents. The generalization of the MDPs to the multi-agent case is the stochastic game [23].

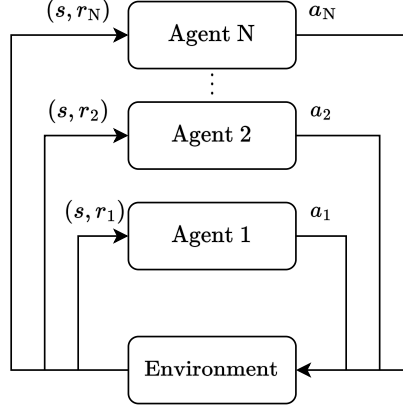


Figure 2.8.: Stochastic game in MARL.

A stochastic game is defined by a tuple $(\mathcal{N}, \mathcal{S}, \mathcal{A}_{i \in \mathcal{N}}^i, P, R_{i \in \mathcal{N}}^i, \gamma)$, where:

- $\mathcal{N} = 1, \dots, N$ denotes the set of $N > 1$ agents,
- $\mathcal{S}$ denotes the state space observed by all agents,
- $\mathcal{A}^i$ denotes the action space of agent i where $\mathcal{A} := \mathcal{A}^1 \times \dots \times \mathcal{A}^N$,
- $P := \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ denotes the state transition probability function,
- $R^i : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$ is the reward function that determines the immediate reward received by agent i for a transition from (s_t, a_t) to s_{t+1} ,
- $\gamma \in [0, 1)$ is the discount factor.

For each time step, each agent i observes the state $s_t \in \mathcal{S}$ of the environment and takes an action $a_t \in \mathcal{A}^i$ based on the observation, then the environment transitions into state s_{t+1} with probability $P(s_t, a_t, s_{t+1})$ and each agent receives the reward $r_i = R_i(s_t, a_t)$ [23].

In the given definition of a stochastic game, the states in the shared environment are assumed to be fully observable by all agents. $N = 1$ simplifies the problem to a SARL MDP, whereas $N > 2$ is considered as multi-agent case. The local policies $\pi_i : \mathcal{S} \times \mathcal{A}^i \rightarrow [0, 1]$ of the individual agents combine to form a joint policy π .

In scenarios where multiple agents operate within a common environment, their interests can either be cooperative or conflicting. MARL facilitates the exploration of diverse alignments of interests, shedding light on how these alignments impact the behavior of the agents. The framework of stochastic games provides a versatile structure that encompasses a range of settings within MARL, as outlined below [24]:

- **Cooperative setting** refers to a scenario where multiple agents work together to achieve a common goal or task. In such a setting, the agents' actions are coordinated in a way that optimizes a common long-term return such that $R = R_1 = \dots = R_N$,

rather than focusing solely on individual rewards.

- **Competitive setting** involves multiple agents with conflicting objectives, where each agent aims to maximize its own reward, often at the expense of other agents' success. This setting is typically modeled as a zero-sum stochastic game where $\sum_{i \in \mathcal{N}} R^i(s_t, a_t, s_{t+1}) = 0$ for any (s_t, a_t, s_{t+1}) . For the setting with $n = 2$ agents, the reward of one agent is exactly the loss of the other, such that $R_1 = -R_2$.
- **Mixed-sum setting** characterizes a scenario where agents are not bound by specific constraints on their goals or relationships. Each agent is driven by self-interest and possesses distinct reward functions $R_1, \dots, R_N$.

2.3.3. Deep Reinforcement Learning (DRL)

The Deep Reinforcement Learning (DRL) stands as a subset of ML that merges RL with DL techniques. Traditional control methods often fail to resolve decision-making challenges within numerous real-world applications characterized by high-dimensional states and actions [25]. DRL surmounts this issue through the integration of deep neural networks, enabling more effective problem-solving in such complex contexts. DRL leverages deep neural networks to approximate the agent's policy or value function. While deep neural networks hold the potential to construct intricate and comprehensive models, as highlighted in [26–28], instances may arise where simpler, more data-efficient approaches are more favorable [29].

DRL has achieved a significant portion of its successes by extending previous RL methodologies to address the complexities of high-dimensional scenarios. This success is attributed to neural networks' capacity for learning low-dimensional feature representations and their robust function approximation abilities. By means of representation learning, DRL effectively addresses the difficulties linked to high-dimensional contexts, setting it apart from tabular and conventional nonparametric methods [30]. A pertinent example lies in the utilization of CNNs as integral components of RL agents, enabling them to directly learn from raw, high-dimensional visual inputs. In general, DRL is based on training deep neural networks to approximate the optimal policy π^* and/or the optimal value functions V^* , Q^* , and A^* [31].

2.3.4. RL algorithms

Model-free and model-based RL

In the realm of RL, a pivotal distinction is whether an RL algorithm is classified as

model-free or *model-based*. This classification hinges on the algorithm’s ability to engage a dynamics model during control computation or policy updates, as delineated by [31]. Given a model of the environment, one can employ dynamic programming over all feasible actions, sample trajectories for heuristic search, or undertake exhaustive exploration. RL primarily concentrates on learning without direct access to the underlying environment model. However, interactions with the environment can contribute to the acquisition of value functions, policies, and even a model. Model-free RL techniques directly glean insights from interactions with the environment, whereas model-based RL methods can employ the acquired model to simulate transitions, thereby enhancing sample efficiency. This is especially critical in scenarios where each interaction with the environment incurs substantial costs. Nonetheless, learning a model introduces additional complexities and the potential risk of being impacted by model inaccuracies, which subsequently influence the learned policy.

Value function-based and policy search methods

Solving RL problems typically involves two primary approaches, these are the value function-based methods and policy search methods. Additionally, a hybrid approach known as *actor-critic* combines both value functions and policy search techniques [31].

Value-based methods focus on estimating value functions $V^\pi(s)$ and subsequently deducing the policy. These techniques are well-suited for handling high-dimensional state spaces, although they might be less effective in continuous action spaces [32]. On the other hand, policy search methods center on the policy itself. The probability of action a being taken at time t , given state s , and policy, which is parametrized by θ , is denoted as $\pi(a \mid s, \theta)$. The objective is to optimize the policy parameters θ through search methods. Unlike value-based methods, policy search approaches circumvent the need for value function estimation and can demonstrate higher sample efficiency, particularly in continuous action spaces [32]. Actor-critic methods blend both value-based and policy-based approaches. In these methods, the critic estimates the value function, while the actor updates the policy based on the estimated value function. This combination allows actor-critic methods to strike a balance between the benefits of value-based and policy-based techniques, offering a more nuanced approach to RL problem-solving [31].

2.3.5. Proximal Policy Optimization (PPO)

Policy gradient methods form a subset of model-free policy search algorithms, aiming to enhance policies by approximating gradients of expected rewards with respect to policy adjustments [34]. A notable challenge in policy gradient methods involves determining an optimal step size for policy updates. An erroneous step size can result in notable reductions in policy effectiveness, largely stemming from the dependence of input data on the prevailing policy. The idea of Proximal Policy Optimization (PPO) is to improve

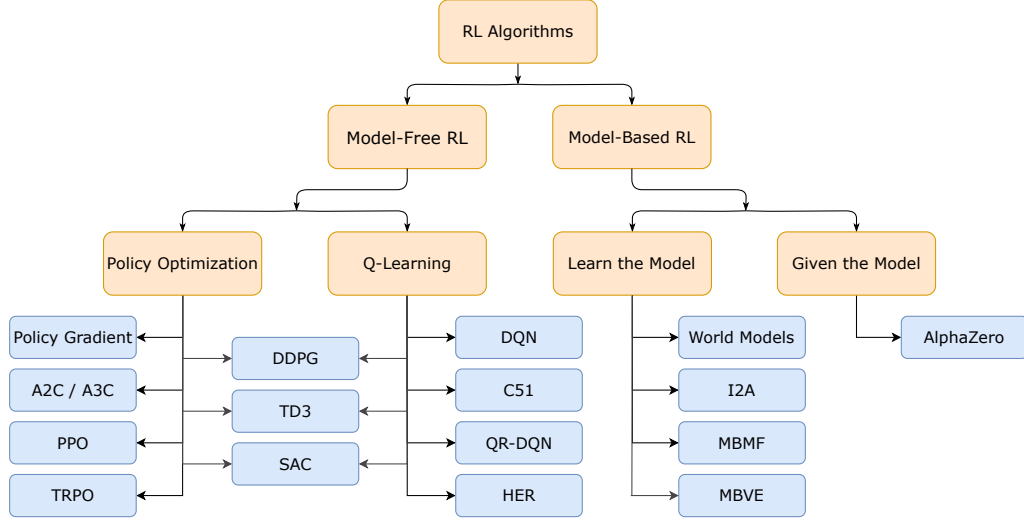


Figure 2.9.: Taxonomy of RL algorithms [33].

the training stability of the policy by limiting the change to make to the policy at each training epoch [35]. Too large policy updates should be avoided. Empirical observations have shown that smaller policy updates during the training process tend to yield a higher likelihood of converging to an optimal solution. Oversized policy updates can lead to undesirable outcomes, such as descending into suboptimal policies and facing significant challenges or even impossibility in recovering from them.

In order to enhance training stability within the PPO framework, a conservative approach is adopted for policy updates. This involves quantifying the extent of change between the current policy and its predecessor through a ratio calculation. Subsequently, this ratio is constrained within a range of values, specifically within the interval of $[1 - \epsilon, 1 + \epsilon]$, where ϵ represents a small margin. By employing this clipping mechanism, the motivation for the current policy to deviate excessively from the previous one is eliminated, leading to the incorporation of the proximal policy term [36].

PPO is well-suited for environments featuring either discrete or continuous action spaces [35]. The algorithm is purposefully crafted to strike a balance between factors such as implementation simplicity, sample complexity, and tuning requirements, all while minimizing deviations from the prior policy. PPO’s demonstrated effectiveness in addressing diverse and demanding tasks, coupled with its straightforward implementation and tuning process [34], makes it a compelling choice for tackling the RL problem dealt with in this thesis.

Algorithm

As explained in (2.3.1), the agent interacts with the environment to collect trajectories.

These trajectories are used to estimate advantages and update the policy $\pi_\theta(a_t|s_t)$. The advantages estimate how good an action is compared to the average action for a specific state. Generalized Advantage Estimation (GAE) is commonly used to compute advantage estimations $\hat{A}_t$ [37], such that

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + \dots + (\gamma\lambda)^{T-t+1}\delta_{T-1}, \quad (2.3.6)$$

where $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ is the temporal difference error, γ is the discount factor, λ is the GAE parameter and T is the length of the trajectory.

The probability ratio between the new policy and the old policy can be denoted as

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}. \quad (2.3.7)$$

The clipped surrogate objective encourages the policy to move in the direction that improves the expected rewards. It is designed to be maximized, subject to a constraint that ensures the policy update is not too large. If the probability ratio between the new policy and the old policy falls outside the range $1 - \epsilon$ and $1 + \epsilon$, the advantage function is clipped.

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (2.3.8)$$

To encourage exploration, PPO also includes an entropy term in the objective function, that penalizes overly certain policies, encouraging the agent to explore different actions.

$$L^{ENT}(\theta) = -\hat{\mathbb{E}}_t [\pi_\theta(a_t|s_t) \log \pi_\theta(a_t|s_t)] \quad (2.3.9)$$

When a neural network architecture is utilized, which shares parameters between the policy and value function, it is necessary to employ a loss function that combines the policy surrogate with a value function error term. This objective can be further enhanced by the addition of an entropy term to ensure sufficient exploration. The complete objective function can be written as

$$L(\theta) = \hat{\mathbb{E}}_t [L^{CLIP}(\theta) - c_1 L^{VF}(\theta) + c_2 L^{ENT}(\theta)], \quad (2.3.10)$$

2. Background

where c_1 and c_2 are coefficients, and $L^{VF}(\theta)$ is a squared-error loss $(V_\theta(s_t) - \hat{V}_t)^2$. Therefore, PPO employs an objective function that balances the clipped surrogate objective, entropy regularization, and value function loss. This function is maximized each iteration [35].

PPO with the clipped objective is described in Algorithm 1. It employs fixed-length trajectory segments. In each iteration, a total of N parallel actors gather data for T timesteps. Subsequently, the surrogate loss is formulated using this collected data spanning NT timesteps, and it is optimized using minibatch SGD over K epochs [35].

Algorithm 1 PPO, Actor-Critic Style

```
for iteration=1, 2, ... do
  for actor=1, 2, ...,  $N$  do
    Run policy  $\pi_{\theta_{\text{old}}}$  in environment for  $T$  timesteps
    Compute advantage estimates  $\hat{A}_1, \dots, \hat{A}_T$ 
  end for
  Optimize surrogate  $L$  wrt  $\theta$ , with  $K$  epochs and minibatch size  $M \leq NT$ 
   $\theta_{\text{old}} \leftarrow \theta$ 
end for
```

For a given state, the actor network produces a probability distribution over all possible actions. To decide which action to take, the PPO algorithm samples an action from the probability distribution generated by the policy network. The choice of action is influenced by a trade-off between exploitation, meaning choosing actions that are known to yield high rewards and exploration. The actor network’s output distribution embodies this trade-off: actions with higher probabilities are more likely to be selected (exploitation), while the distribution’s entropy encourages some degree of uncertainty and exploration [38].

3. Related work

3.1. Path planning in MARL

In this chapter, traditional path-planning methods, CNN-based approaches, RL-based approaches, and CNN-integrated RL methods are presented and discussed. Since one of the goals of this thesis is to optimize the observer’s trajectory and maximize its tracking performance, conducting literature research on trajectory and path planning becomes essential. Therefore, this section presents an overview of various research studies focused on these topics.

3.1.1. Traditional path planning

One of the major challenges for autonomous agents is to develop a safe and robust collision avoidance policy to navigate from a starting position to a desired goal without running into obstacles in unknown potentially cluttered environments [39], [40]. Hence, various path-planning methods have been developed for collision avoidance.

Global path planning algorithms, such as Dijkstra, A* (A-star), Probabilistic Roadmap Method (PRM), Spherical Expansion (SE), and Rapidly Exploring Random Tree (RRT) algorithms are commonly used to obtain feasible paths from a global map [41], [42], [43], [44], [45].

Traditional global map path planning methods are primarily categorized as geometry-based and sampling-based methods. Dijkstra is a classic algorithm known for finding the shortest path between two points, making it optimal for global planning. On the other hand, the A* algorithm represents a classical geometry-based search and employs heuristics to guide its search, which extends the capabilities of Dijkstra and enhances performance [46]. Sampling-based methods, like the SE and the PRM, determine the shortest path by connecting the points selected from sampling points. The time complexity and solution quality of these methods are controlled by adjusting the number of sampling points.

However, most global path planning algorithms are designed for static scenes and rely

on prior map information, which hinders their efficiency as the environment becomes more complex and uncertain [47]. The efficiency of these algorithms diminishes in such cases. In addition, traditional algorithms normally involve a number of parameters that need to be tuned manually [48] rather than being able to learn from past experience automatically [49]. It is difficult for these approaches to generalize well to unanticipated scenarios.

Traditional path planning suffers from issues such as map dependence and low real-time performance [47]. Moreover, obtaining completely accurate prior information about environments and models is nearly impossible. Many Computational Intelligence (CI) methods have been to improve the learning ability of algorithms, such as neural networks [50], fuzzy logic [51], and others. Most CI methods are SL, which need enough prior knowledge or supervised instances to meet the training requirements.

3.1.2. CNN-based path planning

In the work [52] of Sartori et al., particular emphasis is placed on the motion planning phase. This involves the determination of a series of waypoint coordinates to establish a path connecting the initial point to the final goal point, given the features of a known environment. To expedite this path-planning computation, they introduce a novel approach utilizing a CNN to substitute the traditional grid search algorithm. The key of this approach lies in the CNN architecture, which is trained using a dataset comprising paths precomputed through the utilization of the Theta* search algorithm [53]. The CNN receives an image of the top-down view of the environment as an input, with color codes indicating free spaces, obstacles, start point, and goal point, and it outputs the estimated location of the path waypoints. This approach for offline global motion planning is SL.

Although CNN-based path planning methods that are mentioned above utilize CNN for processing map images in order to develop policies, as it is also aimed in this thesis, the mentioned approaches still rely on using CNNs trained through SL. For this reason, CNNs require pre-training based on an already prepared dataset. Nonetheless, the works mentioned in this subsection still offer an inspiring foundation for employing CNNs to process the context knowledge for path-planning purposes.

3.1.3. RL-based path planning

Meanwhile, RL has emerged as a notable technique, attracting an increasing interest due to its capacity to learn without the requirement of pre-existing supervised instances [54]. However, a significant drawback hindering its widespread adoption, particularly in

scenarios with limited computational resources, is the excessively lengthy training duration associated with RL.

The initial application of DRL in discrete control for mobile robots aimed at achieving obstacle avoidance [40]. In the work [55], Tai and Liu introduced an end-to-end obstacle avoidance strategy for mobile robots based on deep Q learning. Furthermore, they extended their efforts in [56], where they developed a map-free indoor navigation system utilizing sparse laser radar signals and DRL. Similarly, in the work [57], Brandenburger et al. proposed to use a DRL-based collision-free path planner for the map-free indoor navigation system, so that the planner learns a latent world model, where the environment is perceived by RGB-only images. These works provide valuable methods for collision avoidance in ground vehicle navigation by utilizing environmental sensing data, which offer essential insights for the collision simulation in this thesis. Nevertheless, it should be emphasized that one of the aims of this thesis is to provide agents with environment maps. In contrast, the works mentioned above primarily focus on map-free approaches and do not explore the utilization of comprehensive map representations of the environments.

In [40], the authors propose an approach that enhances the efficiency and convergence of DRL-based path planning for mobile robots. State-related observations at each time step are obtained from the robot's LIDAR, including angle, distance to the target point, linear velocity, and angular velocity from the previous step. These inputs guide the path-planning process within the DRL framework.

In the work [58], Faust et al. achieved long-range robotic navigation by combining PRM and RL methodologies. Similarly, in [59], PRM was employed as the sampling-based planner, while AutoRL served as the RL technique for indoor navigation. Furthermore, in [60], the authors explored large-scale indoor navigation by considering PRM and RRT as global path-planning methods. These methods were employed to identify intermediate waypoints for the DRL algorithm, further improving the efficiency of the navigation process. Hence, to enhance the effectiveness of path planning, traditional path planning algorithms are combined with the DRL algorithm.

In the work by Xinming et al. [61], a two-level method is presented, called Spherical Expansion-Proximal Policy Optimization (SE-PPO), for Unmanned Aerial Vehicle (UAV) path planning. SE-PPO combines SE and PPO algorithms, considering Unmanned Aerial Vehicle (UAV)s' kinematic constraints and dense obstacles, to establish feasible trajectories for UAVs. PPO is preferred for its effectiveness in continuous and large state spaces [62]. Initially, SE generates an initial path and subgoals. Then, a training environment with dense obstacles and an RL-based training process yields the path optimizer. The optimizer employs sub-goals and an optimal policy to optimize sub-paths between adjacent sub-goals. SE-PPO exhibits good adaptability to new environments, efficiently handling dense obstacles and UAV kinematic constraints without retraining. The work relies on state-related observations, such as the position of the UAV and its distance to the closest obstacle, and employs SE for generating initial collision-free paths using quasi-random

sampling, rather than map image processing. Similar to [58] and [59], traditional path planning methods are utilized together with state-of-the-art DRL algorithms.

The primary limitation of the works mentioned in this subsection, considering what is aimed in this thesis, is their reliance on map-free approaches that use environmental sensor data instead of complete maps for path planning. In addition, they maintain fixed start and end points throughout the entire simulation run. Moreover, they adopt path planning algorithms based on traditional algorithms and DRL to generate paths on the environment without processing them as map images. Consequently, these methods only rely on state-related observations and their interaction with the environment. Still, they provide quite impactful approaches in the utilization of the DRL in path-planning, while removing the necessity of large datasets and pre-trainings, as was the case for the works in Section 3.1.2, and proposing on-the-fly learning.

3.1.4. CNN-enhanced RL for path planning

In Section 3.1.2, it was discussed that although CNNs have been used for path-planning tasks, they require pre-training based on an already prepared dataset. Meanwhile, the RL-based approaches that are presented in Section 3.1.3 remove the need for pre-training, but they mostly consider map-free navigation problems and also consider fixed goal points. This subsection presents the previous research involving the utilization of the CNN for map image processing within the RL framework, along with DRL models. These works also delve into the methods of delivering the extracted knowledge from these images to the agents, aiding them in tasks related to path planning and environmental navigation. Given the congruence between the objectives of this thesis and these approaches, they offer valuable ideas and insights.

It was mentioned in Section 3.1.2 that SL methods require extensive manually labeled datasets for training. While it can be partially mitigated by self-supervised learning, it still relies on generating accurate training labels. Similarly, agent-level methods like the one presented in [63] use DRL for collision avoidance but demand perfect sensing, making the system computationally expensive and less robust in the face of perception uncertainty. Recent research indicates a preference for implementing RL within an SL-based framework [64], with SL methods like experience replay [65] and kernel representation [60] adapted for RL.

In the drone patrolling problem discussed in [66], the map images are directly fed to the agent, without any processing. In this work, a local relevance map of the patrolling area is used, showing the agent’s immediate surroundings and cropped to a fixed size. This map serves as input to an agent, without considering information about the physical environment or navigation constraints like obstacle avoidance or flying time. Furthermore, in [67], data is collected simultaneously from ground and aerial vehicles on a small map

featuring obstacles. Due to the small map size, DRL agents can use full global map information as input for their operations, without the need for any image processing with the CNN.

DL has achieved remarkable performance in various perception problems, such as image classification [68] and speech recognition [69]. In RL, DL, particularly CNNs, is utilized to learn feature representations from large-scale state inputs like sequential images and approximate Q-learning is applied to find optimal policies. In the work [70], Tai et al. demonstrate the effectiveness of a hierarchical structure combining a CNN with a decision process to produce control commands from depth images. In recent advances, CNNs have been successfully integrated into approximate Q-learners, enabling significant accomplishments in domains like playing Atari video games [71] and Go games [72]. Therefore, the CNNs are integrated within RL frameworks in order to extract the information from the used images.

In the work [73], Chu et al. introduce a model-free MARL approach that employs CNNs to extract valuable lower-dimensional features from large-scale multi-agent system (MAS) images. They utilize a stack of 2D images to represent a MAS. Each pixel contains specific information related to its location in the MAS, hence the spatial correlations among the pixels are preserved. They employ CNNs on these images, with each image carrying a single channel of information, to learn hierarchical feature maps for RL purposes. The CNN-integrated RL agent's performance is tested in a traffic grid signal control simulation, outperforming traditional RL agents. Furthermore, this study confirms that CNN can be used in an RL method, with feature extraction integrated into the learning model. A key contribution of the paper lies in the development of an image-based representation method, maintaining spatial correlations, and effectively mapping the MAS state-action information to the stack of 2D images. The authors emphasize that the CNN model can ensure stable and efficient feature extraction for approximating the Q-function. The numerical experiments conducted on a traffic signal grid highlight the advantages of using the CNN-enhanced RL model.

Chen et al. propose [39] an end-to-end model-free DRL algorithm to improve the performance of autonomous decision-making in complex environments. This algorithm directly maps local probabilistic cost maps to an agent's steering commands in terms of target position and robot velocity. The DRL-based local planner takes the velocity information generated by the Simultaneous Localization and Mapping (SLAM) module, the local goal generated by the global path planner, and the cost maps from the generator which can fuse multi-sensor information. Then, it outputs the linear velocity and angular velocity of the robot. Cost maps are used in order to represent environmental information perceived by multi-sensors. The input of the network has three local maps and a four-dimensional vector with local goal coordinates and the velocity of the robot. The output of the network is the Q-values for actions. The input cost maps are fed into the CNN. The four-dimensional vector with local goal coordinates and the velocity of the robot is processed by one fully connected layer. Then, it is pointwise added to each point in the response map of the

3. Related work

processed cost map by tiling the output over the special dimensions.

The works in [73] and [39] provide helpful insights about how to make use of the map images in RL tasks. In this thesis, the context knowledge is aimed to be provided to the agents so that they become aware of the context of the urban environment. Therefore, extracting this knowledge from the map images using CNNs, as it was done in these works, is an impactful method.

Going on with the works that involve CNN in RL tasks, in the work [74], Theile et al. introduce a path-planning method for UAVs. This method addresses two distinct scenarios: area coverage path planning (CPP) and data harvesting (DH) [75]. CPP aims to maximize area coverage while avoiding obstacles or no-fly zones. On the other hand, in the DH scenario, the UAV’s objective is to collect data from various devices in an urban environment, which entails navigating challenging radio channel conditions due to alternating line-of-sight and non-line-of-sight links between the UAV and the devices, often obstructed by buildings. The authors state that both scenarios can be treated as RL problems, sharing nearly identical path-planning constraints with differing goal functions.

They leverage a DRL approach by directly feeding spatial map information to the DRL agent through the CNN. The primary focus of this work is to propose advancements in existing DRL methodologies for generalized, large-scale UAV path planning tasks. To address the Partially Observable Markov Decision Process inherent in these scenarios, they employ Double Deep Q-Networks (DDQNs) [76]. The environment contains regulatory no-fly zones (NFZs), and obstacles. The movement of the UAV is constrained through collision avoidance with obstacles and not entering NFZs. In CPP, the mission is to cover a designated target area by flying above or near it, such that it is in the field of view of a camera-like sensor mounted underneath the UAV. Additionally, buildings can block line-of-sight. To aid an agent in interpreting the large state space, two map processing steps are used. The first is centering the map around the agent’s position to improve the agent’s performance. The downside of this approach is that it increases the representation size of the state space even further. Using maps as a direct input becomes problematic for large map sizes, as the network’s size and training time increase equivalently. They introduce a global-local map scheme to address the scalability problems of the standard map-based input. With the global map, a compressed version of the complete environment map centered on the agent’s position, general information about all objects on the map is provided to the agent. In contrast, the local map, uncompressed but cropped to show only the UAV agent’s immediate surroundings, provides detailed local information.

The main contribution of this work is to present the centered map as two inputs: a full-detail local map showing the agent’s immediate surroundings and a compressed global map showing the entire environment with less detail. The environment map and target map are stacked and centered around the UAV position and then converted into global and local observation components. The thesis shares similar constraints and concepts with this work,

particularly in addressing challenges related to occlusion and collision avoidance within maps that include buildings, as well as managing a large state space. Additionally, this work introduces valuable strategies, such as leveraging spatial map information through CNNs to assist DRL agents, which can be applicable to the thesis.

The concepts outlined in this subsection hold significant value, particularly in the context of extracting and delivering context knowledge to the agents. These works demonstrate the feasibility of integrating CNNs into RL models to process images in conjunction with scalar state observations. Furthermore, they illustrate how this context knowledge can be effectively employed to make decisions and formulate policies for executing the tracking and evasion task.

3.2. Procedural map generation

Providing agents with lots of diverse data helps them build more robust, general, and effective policies. In addition, it prevents overfitting to specific situations, leading to better performance and adaptability in a variety of environments. Therefore, each RL episode uses unique maps, such that at the start of each RL episode, required maps are generated and then employed throughout the episode.

In addition, CNNs need a substantial volume of data to train [77]. Seeing lots of data during training enables CNNs to learn complex patterns [78] that are essential for making informed decisions in different scenarios.

While this data can be generated by humans, it can be laborious and time-consuming especially when a large volume of data is required. This situation highlights the importance of efficient data generation methods. One of the automation techniques that provide time-efficient data generation is called *procedural generation*. By definition, procedural generation is the automatic creation of digital assets based on predefined algorithms and patterns that require minimal input [79].

Urban maps mainly consist of non-uniform buildings and road networks [80]. Several works are described below, which propose methods in order to procedurally generate urban maps with a natural appearance.

In the work [81], Deussen et al. introduce the L-System method, which is a generative technique that utilizes formal grammar rules to create complex and organic patterns through recursive growth. It begins with a starting axiom and a set of production rules, which are then iteratively applied to rewrite the initial chain. Originally employed in the modeling of plants, L-Systems have found applications in city map generation.

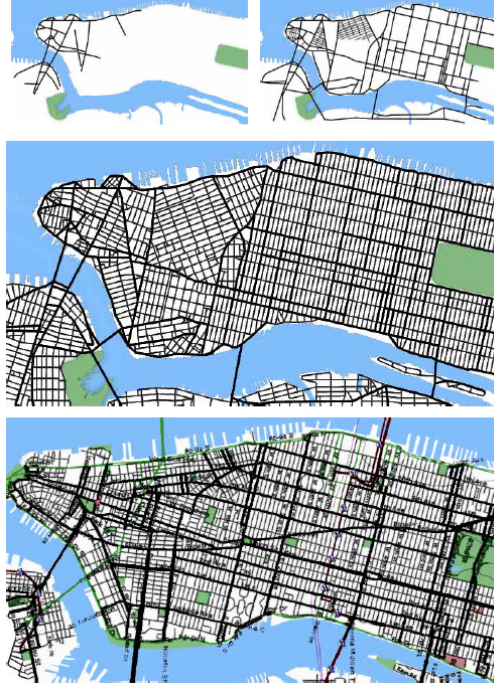


Figure 3.1.: Street creation system applied to Manhattan. Top row: The network after 28 and 142 steps. Middle: The final roadmap. Lower: A real map of Manhattan streets for comparison [82].

In Figure 3.1, an example of how the L-system operates, involving starting with a simple axiom and applying the production rules recursively to generate a chain with increasing complexity, is shown. L-Systems offers a flexible approach to urban map generation, enabling the generation of diverse city layouts by adjusting the production rules accordingly.

In the work [83], Gumin proposes the Wave Function Collapse (WFC) method, which is inspired by ideas from quantum mechanics. This method offers a technique for generating bitmaps that exhibit local similarities to the input bitmap. In this method, the generated output is constrained to only contain existing patterns of pixels from the input. The distribution of patterns in the output is designed to resemble the distribution of patterns observed over a sufficiently large number of outputs, ensuring that the probability of encountering a particular pattern in the output aligns closely with the density of such patterns in the input. This approach allows for the generation of city maps based on smaller urban map chunks [84]. Numerous examples demonstrating the capabilities of the WFC method are depicted in Figure 3.2.

Voronoi diagrams are geometric representations that divide a space into regions based on their Euclidean distance to specific points within the plane [85]. Each region corresponds

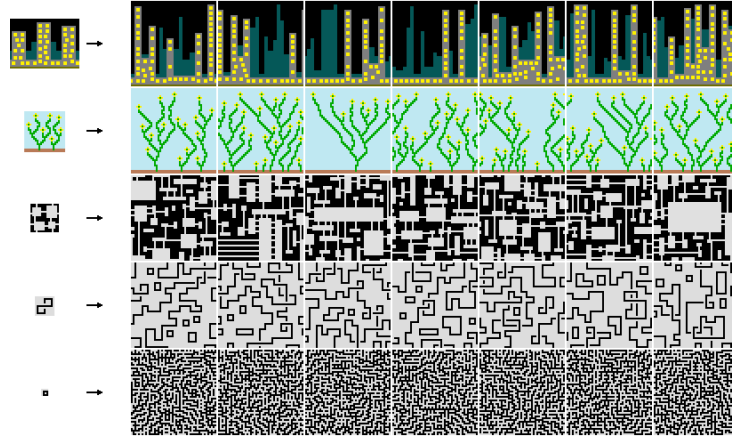


Figure 3.2.: Various examples of output images based on smaller input bitmaps, generated using the WFC method [83].

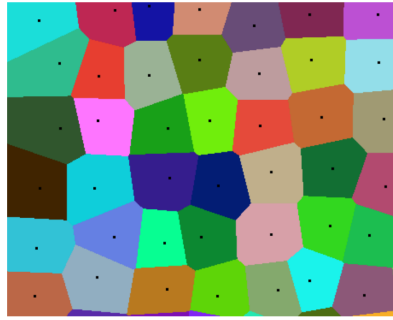


Figure 3.3.: In Voronoi diagrams, the plane is partitioned into polygons, based on sampled points [86].

to a set of points that are closer to a particular generating point than to any other.

The Perlin noise algorithm, developed by Ken Perlin in 1985 [87], has become widely used due to its ability to generate random patterns that exhibit smooth transitions between values. The Perlin noise is a gradient noise, meaning it establishes random gradients at regularly spaced points in space and then interpolates a smooth function between those points [88]. This kind of noise provides precise control of the distribution of frequencies present in an image. In contrast to classical white noise, Perlin noise produces smooth or structured randomness while also involving locally correlated sampling and coherent patterns across multiple dimensions, making it convenient for procedural generation [89].

The simplex noise is an improvement over Perlin. It is shown to be more efficient than its predecessor Perlin and has no directional artifacts [88] (see Figure 3.4c).

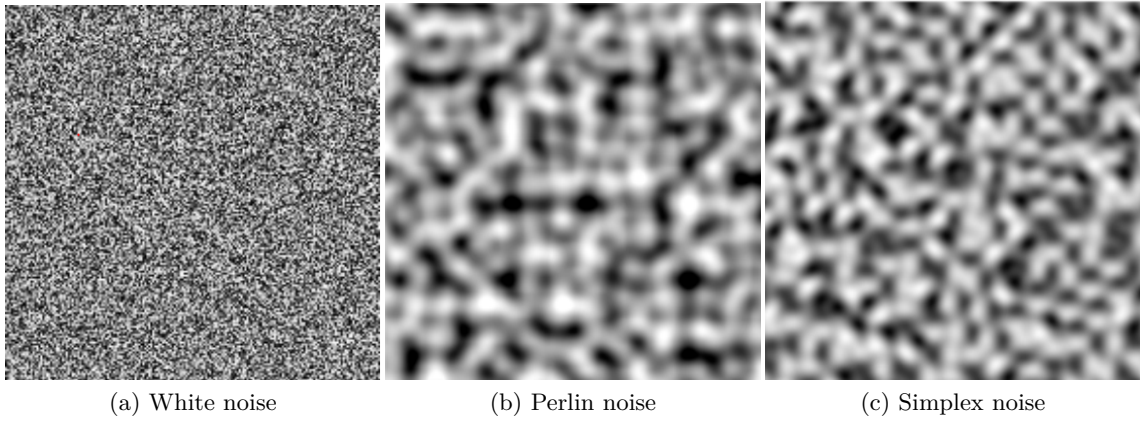


Figure 3.4.: Comparison of the white noise, Perlin noise, and simplex noise. Perlin noise and simplex noise have a coherent and locally correlated distribution, whereas white noise is based on an uncorrelated random distribution. In addition, the horizontal and vertical lines in the Perlin noise are considered dimensional artifacts, which are absent in simplex noise [90], [91].

4. Co-training an observer and an evading target

This chapter is based on the paper *Co-Training an Observer and an Evading Target*, authored by André Brandenburger, Folker Hoffmann, and Alexander Charlish [1]. This thesis can be regarded as an evolution of this paper, contributing to the existing body of knowledge in the field by incorporating new perspectives and insights. The presented advancements and extensions represent the author’s unique contributions to the subject matter, further enriching and advancing the research beyond the scope of the foundational work. This chapter provides a summary of the original work, establishing the foundational principles upon which this thesis is built. By presenting the fundamentals from the mentioned work, this thesis lays the groundwork for further exploration and extension of the research.

4.1. Introduction

The problem that is considered in this work is the sensor path planning problem, where the trajectory of an UAV is optimized. The UAV is tasked with tracking a mobile ground-based target using a range-bearing sensor that has limitations in its Field of View (FOV), akin to radar measurements. The optimization of sensor performance through path planning is a classical problem in sensor management, and numerous algorithms have been proposed, primarily focusing on online trajectory optimization.

This work adopts a game-theoretic approach, where the target’s intent is assumed to be maximizing the degradation of tracking performance. Instead of modeling specific target trajectories, the worst-case scenario of an evading target is considered. Training an observer policy under this worst-case assumption is expected to lead to improved tracking performance for various target behaviors. The concept of an evading target, which poses challenges in path planning based on online optimization, has been extensively explored in the literature.

Thus, a parallel training approach is adopted which implies simultaneously training the evading target’s policy alongside the tracking UAV’s policy. The target is treated as another agent with the objective of maximizing the degradation of the observer’s tracking

performance. By training in this adversarial manner, the observer policy avoids overfitting to the target’s current behavior, incentivizing the target to explore various strategies. In doing so, the target learns that modifying its behavior can lead to less accurate tracking by the observer, prompting it to adapt and adopt different behaviors accordingly. This game-theoretic training method ensures a more robust and adaptive observer policy that can handle a range of potential target behaviors. As the target is henceforth considered an agent, this work extends to the field of MARL.

In this setting, each agent’s learned policy is influenced not only by the environment but also by the policies of the other agents. Since these agents may have diverse goals, each agent’s policy must consider the policies of the others. As the other agents improve in this competitive setting, it becomes increasingly challenging for any agent to achieve its own objectives. This feedback mechanism resembles a form of curriculum learning [92].

Co-training agents and their antagonists has led to significant breakthroughs, notably through self-play, where the policy plays against a potentially different version of itself. This approach has been successfully applied in various domains, such as learning to play Go [72] or Starcraft [93]. In this paper, a setting related to pursuit-evasion problems, where a single or a group of pursuers aim to catch one or multiple evading targets by reaching their positions is introduced.

Several solutions to this problem have been proposed, based on explicitly modeling agent behavior [94–96]. More recently, learning-based approaches have also been explored [97, 98]. One approach involves training an additional policy for evading targets, which can lead to a complex co-evolution of strategies [99]. However, while addressing a similar application, the problem studied in this paper differs from the traditional pursuit-evasion category mentioned in previous work. In the conventional pursuit-evasion setting, pursuers aim to reach the target’s position. In contrast, this paper addresses a sensor management problem, where the pursuer seeks to optimize its measurement geometry to localize the evading target optimally. The objective is not to physically reach the target but rather to obtain the most accurate target localization. To address this challenge, MARL is employed to track the evading target in this paper.

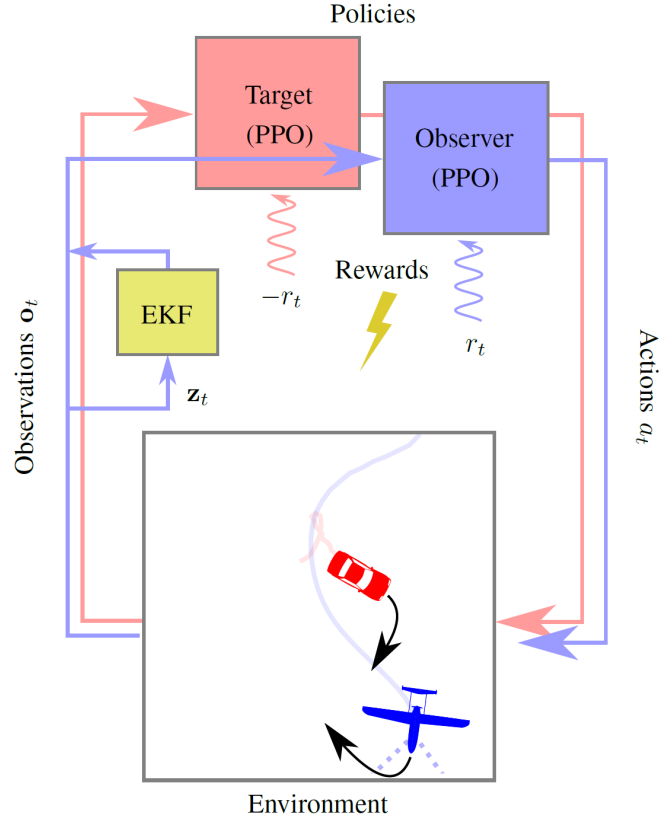


Figure 4.1.: The method’s schematic overview illustrates the process of addressing a classical UAV control task using RL [1]. Two distinct policies, depicted at the top, make action selections based on realistic environment observations shown on the left. To handle noisy sensor measurements, an Extended Kalman Filter (EKF) is employed. The key feature is the use of contradicting reward signals for individual policies, inducing antagonistic behaviors within the learning process.

4.2. Tracking framework

One UAV and one ground-based vehicle are modeled, which can interact within an environment. For the sake of simplicity, height is neglected, resulting in a 2D environment.

The states of the agents, denoted as $\mathcal{A} \in \{\mathcal{O}, \mathcal{T}\}$, are parameterized by their positions $\mathbf{x}_{\mathcal{A},t} \in \mathbb{R}^2$, rotations $\phi_{\mathcal{A},t} \in [-\pi, \pi]$, forward speeds $v_{\mathcal{A},t} \in \mathbb{R}$, and rotational speeds $\omega_{\mathcal{A},t} \in \mathbb{R}$. To simplify the optimization process, a constant value is assumed for $v_{\mathcal{A},t}$, while $\omega_{\mathcal{A},t}$ is utilized to control the agent's steering. The simulation frequency is defined as $\tau > 0$, indicating that $1/\tau$ seconds elapse at each time step. At every time-step $t \mapsto t + 1$, the platform state is updated based on classical mechanics principles.

$$\begin{aligned}\mathbf{x}_{\mathcal{A},t+1} &= \mathbf{x}_{\mathcal{A},t} + \frac{1}{\tau} \begin{pmatrix} \cos(\phi_{\mathcal{A},t}) \\ -\sin(\phi_{\mathcal{A},t}) \end{pmatrix} v_{\mathcal{A},t} \\ \phi_{\mathcal{A},t+1} &= \phi_{\mathcal{A},t} + \frac{1}{\tau} \omega_{\mathcal{A},t}\end{aligned}\tag{4.2.1}$$

To model the observer $\mathcal{O}$ responsible for sensing the target platform $\mathcal{T}$, a simulated radar sensor is employed on the observer platform. This radar generates noisy local measurements $\mathbf{z}_{\mathcal{T},t}$ of the target's position relative to the observer, represented in polar coordinates.

$$\begin{aligned}\Delta \mathbf{x}_t &= \mathbf{x}_{\mathcal{O},t} - \mathbf{x}_{\mathcal{T},t} \\ d_t &= \|\Delta \mathbf{x}_t\|_2 \\ \theta_{\mathcal{T},t} &= \text{atan2}\left((\Delta \mathbf{x}_t)_y, (\Delta \mathbf{x}_t)_x\right) - \phi_{\mathcal{O},t} \\ \mathbf{z}_{\mathcal{T},t} &\sim \mathcal{N}\left([d_t, \theta_{\mathcal{T},t}]; \text{diag}(\sigma_d^2, \sigma_\theta^2)\right)\end{aligned}\tag{4.2.2}$$

The front-facing sensor is subject to a fixed FOV with a constant opening $0 < \alpha \leq 2\pi$, wherein a measurement is only recorded if $|\theta_{\mathcal{T},t}| < \frac{1}{2}\alpha$. Furthermore, $\Delta t_{t,z}$ is defined as the number of time steps since the last recorded measurement.

An EKF is employed to estimate the target's state from the noisy sensor measurements, which uses the true sensor noise matrix and the widely-used piecewise constant white acceleration dynamics model [100]. Consequently, the target track is represented as $([\tilde{\mathbf{x}}_{\mathcal{T},t}, \tilde{\mathbf{v}}_{\mathcal{T},t}], \mathbf{P}_{\mathcal{T},t})$, where $\tilde{\mathbf{x}}_{\mathcal{T},t}$ and $\tilde{\mathbf{v}}_{\mathcal{T},t}$ are estimates of the target's position and velocity, respectively, with corresponding covariance $\mathbf{P}_{\mathcal{T},t}$ in the global frame.

Similar to d_t and $\theta_{\mathcal{T},t}$, the following is defined based on the constructed track, such that

$$\begin{aligned}\Delta\tilde{\mathbf{x}}_t &= \mathbf{x}_{\mathcal{O},t} - \tilde{\mathbf{x}}_{\mathcal{T},t} \\ \tilde{d}_t &= \|\Delta\tilde{\mathbf{x}}_t\|_2 \\ \tilde{\theta}_{\mathcal{T},t} &= \text{atan2}\left((\Delta\tilde{\mathbf{x}}_t)_y, (\Delta\tilde{\mathbf{x}}_t)_x\right) - \phi_{\mathcal{O},t}.\end{aligned}\tag{4.2.3}$$

4.3. Agent models

For this method, the PPO algorithm [101] is employed, known for its high performance in a wide range of tasks, including UAV control [102, 103]. As previously mentioned, the utilization of RL methods necessitates a meticulous definition of the observation and action space, as well as a reward function that accurately reflects the optimization objective.

4.3.1. Observation spaces

To enable the agents to receive a scalar indicator of uncertainty rather than a multidimensional covariance, the entropy is calculated such that

$$S_t = \frac{1}{2} \ln \det(2\pi e \mathbf{P}_{\mathcal{T},t}).\tag{4.3.1}$$

In addition, the time since the last measurement $\Delta t_{t,z}$ is processed using a decay function for a $k_d > 1$, as given in

$$\kappa_t = k_d^{-\Delta t_{t,z}}.\tag{4.3.2}$$

Unlike $\Delta t_{t,z}$, κ_t is bounded, which proves advantageous for ensuring training stability.

Accordingly, the environment state is summarized by projecting the agent's observation with $\theta_{\mathcal{O},t}$ defined analogously to $\theta_{\mathcal{T},t}$, both angles are decomposed into sine and cosine terms, which corrects the discontinuity of the angle representation at $\pm\pi$.

$$\begin{aligned}\mathbf{o}_{\mathcal{O},t} &= [\omega_{\mathcal{O},t}, \kappa_t, \tilde{d}_t, \sin(\tilde{\theta}_{\mathcal{T},t}), \cos(\tilde{\theta}_{\mathcal{T},t}), S_t] \\ \mathbf{o}_{\mathcal{T},t} &= [\omega_{\mathcal{T},t}, d_t, \sin(\theta_{\mathcal{O},t}), \cos(\theta_{\mathcal{O},t})]\end{aligned}\tag{4.3.3}$$

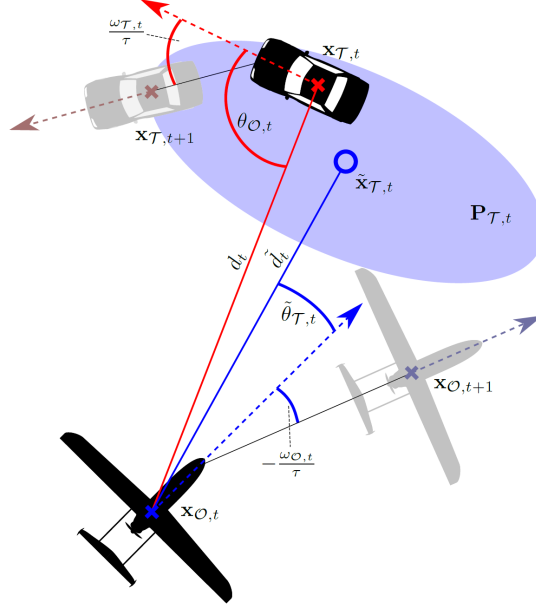


Figure 4.2.: A partial visualization of the observation space [1]. The UAV (bottom left) generates an estimate $\hat{\mathbf{x}}_{T,t}$ of the target (top center) along with its corresponding covariance $\mathbf{P}_{T,t}$. This estimate is then transformed into local polar coordinates $(\tilde{d}_{T,t}, \tilde{\theta}_{T,t})$ before being added to the observation vector. Similarly, the target observation uses a polar coordinate representation of the true UAV position. Both platforms change their heading with respect to their rotational velocities $\omega_{O,t}$ and $\omega_{T,t}$.

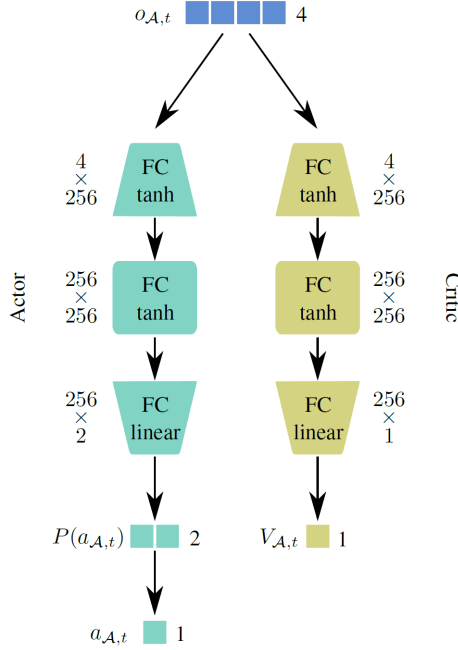


Figure 4.3.: The model architecture for the learned policies [1]. On the left side, the actor network is illustrated, responsible for learning the parameterization of a Gaussian action distribution. On the right side, the critic network is displayed.

This approach results in concise representations of the environment state through observations. For the observer, these observations encompass not only information about the target’s position and platform velocity but also the time-since-measurement decay κ_t and the uncertainty measure S_t . While S_t is crucial for assessing the track uncertainty, κ_t introduces the notion of time to the otherwise time-agnostic observer, enabling more complex behaviors. Although the observer relies on the estimated target position, the target is provided with the true position of the observer. It is essential to note that the goal of this paper is to train an observer policy that remains robust against maneuvering and evading targets. By assuming the worst-case scenario where the target knows the true position of the observer, the target’s role as the antagonist is strengthened.

It is important to note that by normalizing the observations $\mathbf{o}_{\mathcal{A},t}$ with the limits $d_{\max}$ and $S_{\max}$, it is ensured that $\mathbf{o}_{\mathcal{A},t} \in [-1, 1]$. Utilizing normalized values is advantageous as neural networks tend to learn them more effectively, thus enhancing training performance. The parameter $d_{\max}$ represents the maximal range of interest, and $S_{\max}$ is an empirically chosen value higher than any encountered track entropy.

4.3.2. Action spaces

To enable the agents’ navigation in the environment, actions $a_{\mathcal{A},t} \in [-1, 1]$ are accepted, which steer the platform while respecting the velocity limits $\omega_{\mathcal{A},\min}, \omega_{\mathcal{A},\max}$. Indeed, both agents can adapt their rotational velocities to effectively control their movement in the environment.

$$\omega_{\mathcal{A},t} = \frac{1}{2} (\omega_{\mathcal{A},\max} - \omega_{\mathcal{A},\min}) (a_{\mathcal{A},t} + 1) + \omega_{\mathcal{A},\min} \quad (4.3.4)$$

4.3.3. Reward functions

The reward function captures the optimization goals of the RL agents. In this sensor management task, the observer’s objective is to maximize the tracking performance, which is formulated as the minimization of the track entropy S_t . Conversely, the target aims to counteract the observer’s efforts as effectively as possible. Consequently, the rewards are defined as

$$\begin{aligned} r_{\mathcal{O},t} &= 1 - \frac{S_t}{S_{\max}} \\ r_{\mathcal{T},t} &= -r_{\mathcal{O},t}. \end{aligned} \quad (4.3.5)$$

This approach establishes conflicting objectives for the observer $\mathcal{O}$ and the target $\mathcal{T}$. The observer seeks to increase tracking accuracy, while the target aims to reduce it. This antagonistic setup fosters competition between the two agents and allows for the exploration of different strategies and behaviors.

4.3.4. Termination criteria

A single simulation run terminates if any of the following termination criteria are met

$$t > t_{\max} \quad d_t > d_{\max} \quad \tilde{d}_t > d_{\max}. \quad (4.3.6)$$

As is common in the RL literature, each individual simulation run is denoted as an *episode*. The termination criteria contribute to richer training data, as uninteresting situations are not recorded. This effect is particularly noticeable during the early stages of training, where the policies have not yet fully learned their respective goals. It is worth noting that negative rewards provide an incentive to terminate an episode, while positive rewards encourage its continuation. Therefore, it is ensured that $r_{\mathcal{O},t} > 0$ and $r_{\mathcal{T},t} < 0$ hold. This encourages the observer to prefer longer episodes, whereas the target is inclined to end the episode, if possible.

4.4. Policy explanations

As AI techniques continue to advance and find broader applications, the significance of model explanation becomes increasingly pronounced. Neural network models, in particular, exhibit inherent opacity in their reasoning, which raises concerns about trust and poses challenges for their deployment in security-critical systems. Consequently, there has been a surge of research proposing various methods for explaining AI models, and an extensive overview of these approaches can be found in [104]. This paper provides an explanation of the learned policy through post-hoc policy approximation using decision trees, aiming to enhance transparency and interpretability in the AI system. A post-hoc decision tree model of the observer is constructed by generating representative samples of the policy $\Pi : \mathcal{O} \rightarrow \mathcal{A}$. An excerpt of the resulting tree is depicted in Figure 4.4, where the color indicates the action that would be taken if the tree is evaluated until the corresponding node. As is typical with decision trees, nodes in the upper levels of the tree can be regarded as the most critical decision points, while lower levels deal with more specific details of the task.

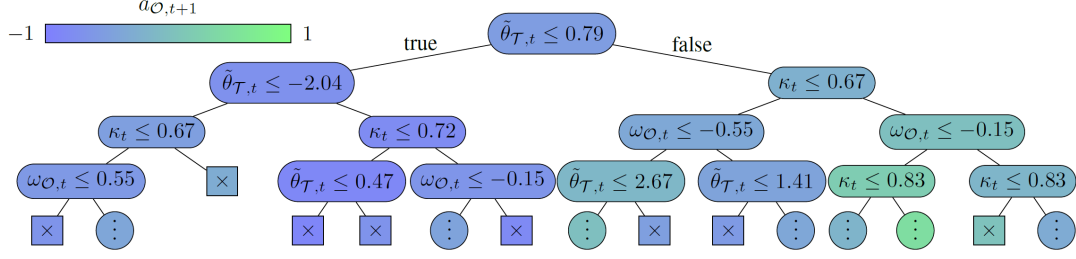


Figure 4.4.: Decision tree generated using artificial observations [1]. Each node in the tree represents a numerical decision criterion, and the node’s color indicates the action that would be taken at that decision point. As traversing the tree downward, the chosen actions more closely mimic the trained policy. Truncated splitting nodes are denoted by vertical dots, while leaf nodes are represented as $\times$.

It is also possible to analyze individual paths in the tree to understand the general decision process of the policy. However, it must be noted that the decision tree is not an exact approximation of the true policy, due to the coarse per-dimension sample size, as well as inherently due to a finite tree. Consequently, it can not be expected to capture the full potential of the agent, prohibiting the exhaustive explanation through this method. Still, the decision tree gives important insight into the general decision process of the otherwise opaque neural network model.

4.5. Novel contributions and advancements to original paper

This paper provides valuable insights into the behavior of agents in an empty environment. In this thesis, the realism and practicality are enhanced by the incorporation of urban maps. Urban maps, comprising buildings and roads, offer a tangible setting. The inclusion of the buildings and roads introduces physical constraints within the environment, which in turn affects the behavior of agents operating within that environment. The buildings constitute obstacles for the movement of the target, as well as they cause difficulties for the observer to track the target. In addition, off-road parts of the map cause the target to slow down. In order to accurately reflect these effects on the agents, simulations of these constraints are implemented.

Therefore, by introducing the context to the environment, interactions between agents and the environment become more intricate. Agents are now to take into account these constraints and conditions of the environment, which are the results of utilizing the urban maps for the representation of the environment. In addition to the already defined observations of the agents, the knowledge about the context provides a different type of information. Hence, this contextual information, termed as the *context knowledge*, is con-

sidered as another set of observations for the agents, presenting information about the map and its components to the agents.

As maps are essentially images, image processing techniques, such as CNN, are employed to extract information about the map and its components. This contextual information about the environment, defined as the context knowledge, is then included as an additional set of observations for the agents, leading to an enhanced awareness of their surroundings.

The utilization of the maps also involves the design and creation of these images, which are then employed for training and testing within the simulation. To ensure effective learning and avoid overfitting, agents are provided with a randomly generated map image in each RL episode. Instead of manually producing urban map images, a technique known as *procedural generation* is implemented and employed. This technique enables the generation of diverse and randomly generated map images based on random sampling while preserving the similarity to the urban maps.

Procedural generation is one of the key elements and research topics of this thesis. It introduces a novel aspect, similar to context knowledge, which complements and enhances the work presented in the original paper.

The subsequent chapters, Chapter 5 and Chapter 6, delve into the concepts of procedural generation and context knowledge, respectively. These chapters present the theoretical foundations and illustrate how these concepts are applied in the context of the tracking and evasion problem.

5. Procedural map generation

In this work, maps come in two types: basic maps with buildings and empty terrain, and more complex maps with non-uniform buildings, roads, and empty terrain. Procedural techniques such as Voronoi diagrams and Perlin noise are used to create the intricate structures in the second type of map. Both map types are generated using procedural image generation, ensuring efficient data generation and diverse environments.

Maps are the representations of the environment. The environment is defined to be enclosed within a square, with its origin placed at the square's midpoint. The maps that are introduced in this chapter have a square shape and their origins are situated at the lower-left corner.

5.1. Basic maps

The aim is to generate a basic grayscale map that contains buildings, and also contain information about the heights of buildings on the map. The following steps outline the generation process for this type of map.

The individual entries of the generating map $\mathbf{M}_g \in [0, 1]^{n \times n}$ are sampled uniformly at random, s.t. $(\mathbf{M}_g)_{i,j} \sim U(0, 1)$ for $i, j = 0, \dots, n - 1$. Based on a threshold β_b , the map $\mathbf{M}_g$ is binarized, resulting in the binary occupancy map $\mathbf{M}_u$ such that

$$(\mathbf{M}_u)_{i,j} = \begin{cases} 1 & \text{if } (\mathbf{M}_g)_{i,j} > \beta_b \\ 0 & \text{otherwise.} \end{cases} \quad (5.1.1)$$

The building threshold β_b determines the density of buildings on the generated map. Increasing β_b leads to a lower building density on the map. This control over β_b allows the generation of various maps with different building densities. The ability to generate maps with varying building densities provides a more diverse environment.

To model the actual height of the buildings, an additional image of the map size is randomly generated in the interval $[h_{\min}, h_{\max}]$, where $h_{\min}$ and $h_{\max}$ refer to the predeter-

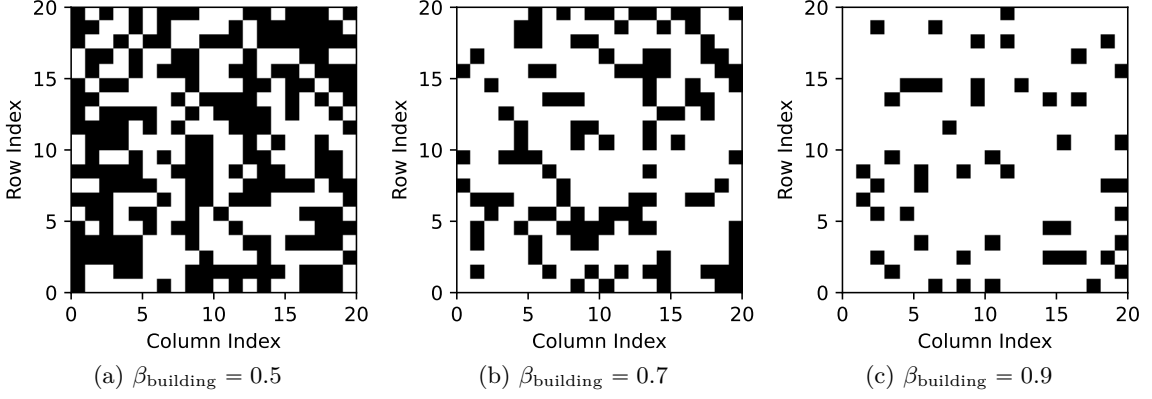


Figure 5.1.: Binary map $\mathbf{M}_{\mathbf{u}}$ with three different configurations. Increasing the building threshold β_b leads to the sampling of fewer buildings, hence lower overall building density in the map.

mined minimum and maximum building height on the map, respectively. This image serves as the building height mask, presented as $\mathbf{H} \in [h_{\min}, h_{\max}]^{n \times n}$. It is sampled uniformly at random, s.t. $(\mathbf{H})_{i,j} \sim U(h_{\min}, h_{\max})$.

The basic map, denoted as $\mathbf{M}_{\mathbf{b}}$, is calculated as the Hadamard product of the binary map $\mathbf{M}_{\mathbf{u}}$ and the building height mask $\mathbf{H}$. This basic map $\mathbf{M}_{\mathbf{b}} \in [0, h_{\max}]$ contains the information of building heights and can be constructed as

$$\mathbf{M}_{\mathbf{b}} = \mathbf{M}_{\mathbf{u}} \odot \mathbf{H}. \quad (5.1.2)$$

A pixel with a value of 0 corresponds to an empty terrain i.e. non-building pixel, whereas a pixel having a nonzero value represents a building. The values of the building pixels correspond to the actual building heights $h \in [h_{\min}, h_{\max}]$.

The normalized version of the basic map, denoted as $\overline{\mathbf{M}}_b \in [0, 1]$, is also introduced such that

$$\overline{\mathbf{M}}_b = \frac{\mathbf{M}_b}{h_{\max}}. \quad (5.1.3)$$

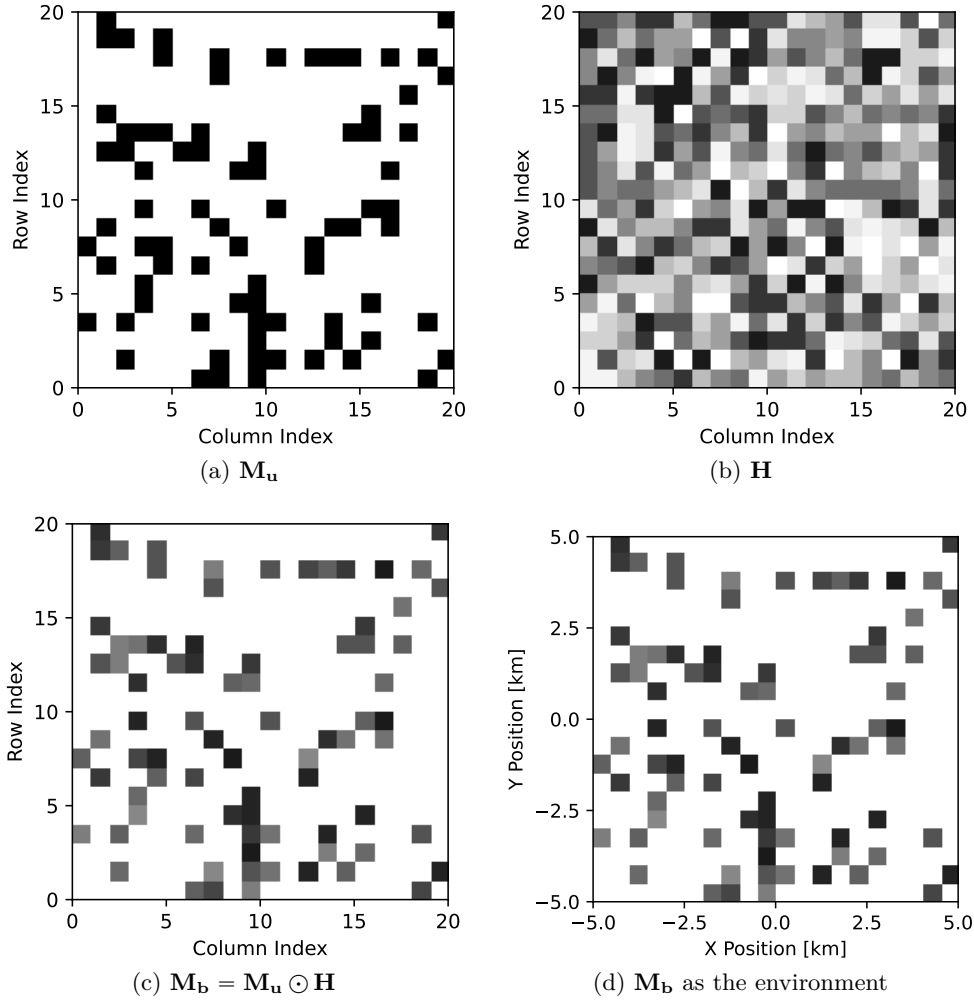


Figure 5.2.: Hadamard product is executed on the binary map $\mathbf{M}_u$ and the height mask $\mathbf{H}$ to obtain the basic map $\mathbf{M}_b$. In (d), representation of the environment with the basic map $\mathbf{M}_b$ is depicted, where the axes represent the x and y positions in kilometers.

5.2. Voronoi diagram-based maps

In order to generate urban maps, Voronoi polygons can be utilized to define a road network, with buildings placed within the polygons. By applying a coherent noise such as simplex noise, obtain natural-looking building maps and road networks are obtained. Voronoi diagrams offer the capability to capture the irregular placement of roads and allow for the configuration of local population density, which aligns well with the characteristics of urban maps [105].

5.2.1. Generating points and partition

Generating points play a crucial role in the construction of Voronoi diagrams, as they determine the overall form and layout of the polygons on the plane. Each generating point serves as the basis of a Voronoi polygon, denoted as P , resulting in a partition of the plane into distinct regions.

The distribution of generating points can be controlled to ensure denser sampling in specific regions representing the city center or densely populated areas, and sparser sampling for the areas representing suburban or countryside regions. By employing such a technique, the procedural generation process can create urban maps with varying population densities and distinct urban features. This approach enables the generation of more diverse and natural urban environments, which are essential for training agents in a more realistic simulation.

The generating points are sampled on a map of length n . The initial step in determining the placement of generating points involves selecting the position of the city center $\mathbf{x}_c \in \mathbb{R}^2$. To ensure diversity and uniqueness in each generated map, the city center's position is sampled as $\mathbf{x}_c \sim U(0, n)$. As a result, the location of the city center varies across different urban maps, contributing to the generation of distinct environments.

The sampling of the generating points $\mathbf{x}_g \in \mathbb{R}^{2m}$ are based on two different distributions, where m of them, denoted as $\mathbf{x}_{g_1} \in \mathbb{R}^m$, are sampled based on a multivariate normal distribution, and the other m , denoted as $\mathbf{x}_{g_2} \in \mathbb{R}^m$, are sampled on a uniform distribution, such that

$$\begin{aligned}\mathbf{x}_{g_1} &\sim \mathcal{N}(\mu, \Sigma), \\ \mathbf{x}_{g_2} &\sim U(0, n), \\ \mathbf{x}_g &= [\mathbf{x}_{g_1}, \mathbf{x}_{g_2}].\end{aligned}\tag{5.2.1}$$

The mean μ of the multivariate distribution corresponds to the city center position and

determines the location where the points are more likely to be concentrated. On the other hand, the covariance matrix Σ controls the spread or sparsity of the point distribution around the city center. By adjusting the mean and covariance matrix of the distribution, the procedural generation process can effectively create urban maps with different levels of density and clustering, simulating diverse urban landscapes that resemble real-world cities with varying population densities and spatial characteristics.

Sampling on two different distributions provides a more balanced and natural distribution, with the city center having higher point density while the rest of the map maintains a more even spread of points. This hybrid approach contributes to the generation of urban maps that better resemble real-world cities, with varied population densities across different regions.

Voronoi polygons are constructed based on these points, partitioning the plane into distinct regions. The information about the vertices, edges, and polygons forms the foundation of the road network and building placement. The vertices represent potential road intersections, the edges define the roads connecting these intersections, and the polygons define the regions for building placement. This sets the stage for generating an urban map with natural and diverse features.

5.2.2. Road network

After generating the Voronoi polygons, the corresponding edges, essentially line segments, are utilized to construct the road network. Unlike continuous vector-based representations, digital images are composed of discrete units known as pixels, each representing a single color or intensity value. Therefore, each road in the road network is in fact a line segment approximation of the corresponding edge. This inherent limitation results in the pixelated representation of roads, where roads are rendered as a series of connected pixels, rather than a smooth and continuous line.

Despite the pixelation effect, the representation of roads through connected pixels is still effective in conveying the structural layout of the road network within the urban map. The pixelated roads provide a clear depiction of road connections and orientations, allowing the map to represent the road network visually.

To avoid antialiasing issues and unwanted artifacts resulting from pixelation, a strategy is employed to mitigate these problems. First, the Voronoi edges, which are the basis of the road network, are first upscaled by a factor of λ . The upscaling factor λ influences the sharpness of the resulting map. By using the upscaled versions, the roads are constructed using line segment approximations based on the Voronoi edges. This results in a road network map with enhanced precision. Finally, the obtained road network map is downscaled back to the original dimensions of the map, preserving the overall map structure while

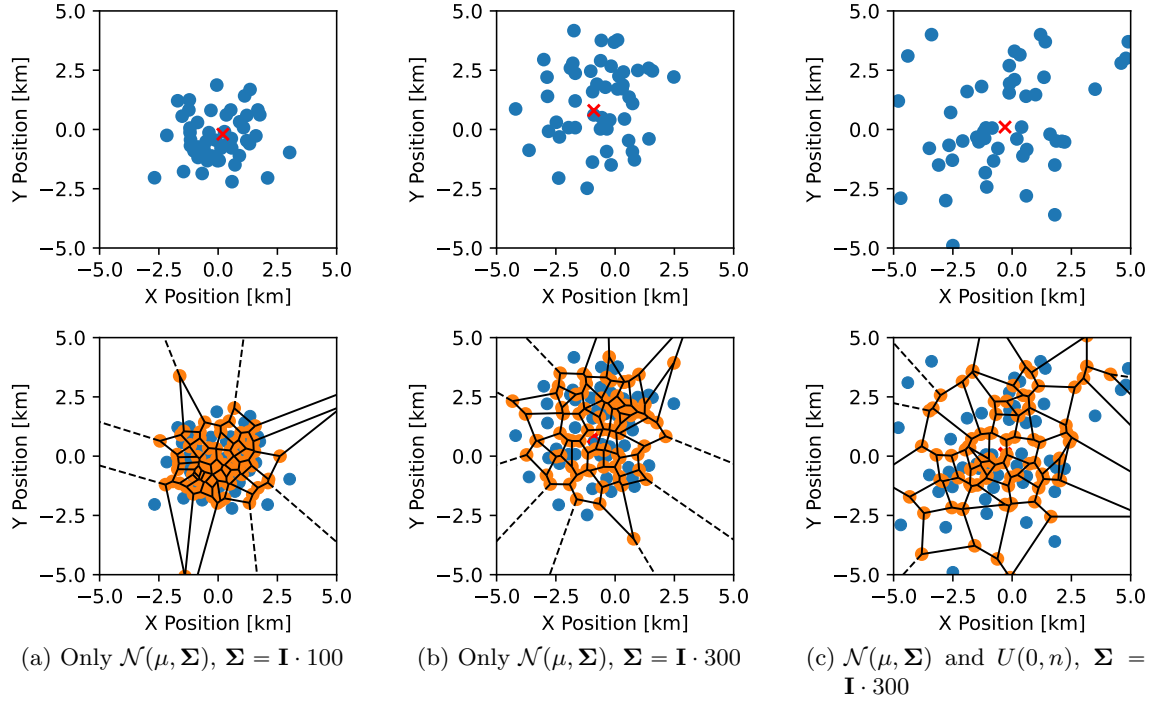


Figure 5.3.: Three different generating point sampling. Blue points represent generating points, the red cross denotes the dense region center, and the vertices of Voronoi polygons are marked with orange points. In (a) and (b), all the $2m$ points are sampled on a multivariate normal $\mathcal{N}(\mu, \Sigma)$, whereas in (c), m points are sampled on a $\mathcal{N}(\mu, \Sigma)$ and m points on a $U(0, n)$. Smaller covariance matrix Σ values result in tighter clustering around the center, emphasizing higher population density, whereas higher values lead to sparser distribution and reduced density. A hybrid approach of $\mathcal{N}(\mu, \Sigma)$ and $U(0, n)$ yields a more natural representation of urban and suburban population distributions. The bottom figures represent the Voronoi diagram of the related generating point distribution.

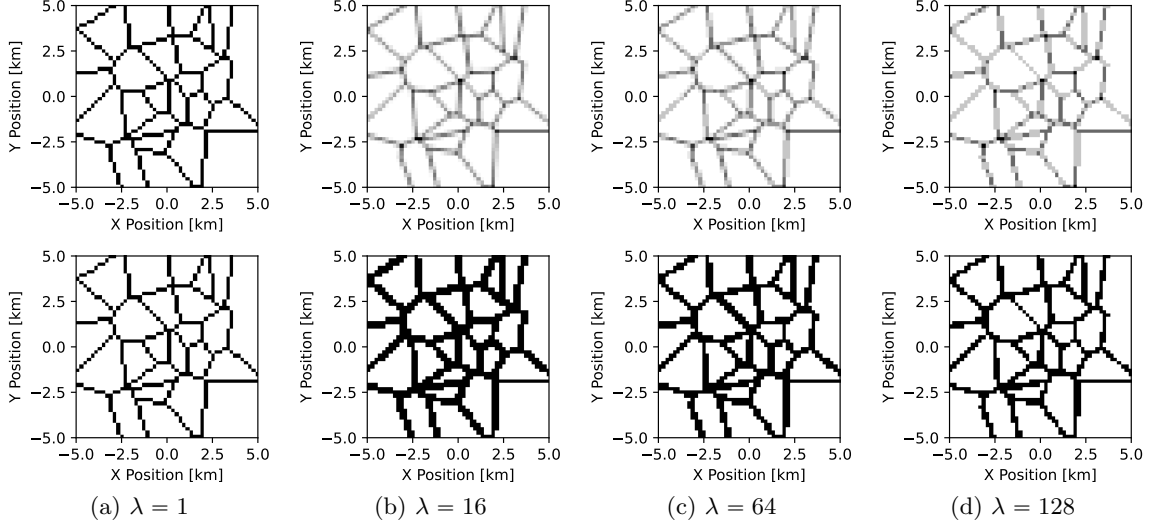


Figure 5.4.: Effect of upscaling factor λ on line segment approximation. $\lambda = 1$ means no upscaling is applied. In the process of upscaling, due to the descaling of the upscaled road network map into the actual map size, the road network $\mathbf{M}_{\mathbf{r}}$ is converted into the range $[0, 1]$. Images on top reflect the actual road network maps, whereas their binary versions are added on the bottom, for the sake of clarity.

achieving a higher level of detail. This road network map is denoted as $\mathbf{M}_{\mathbf{r}} \in [0, 1]^{n \times n}$.

Drawing roads relatively thin in the generated maps enhances the similarity to real urban maps. This choice ensures that the target agent can move along the roads without encountering unnecessary restrictions while maintaining a natural appearance. Additionally, reducing the width of the roads aids in creating a clear distinction between the road network and other elements on the map, such as buildings or open spaces. This clarity is vital in the context of RL scenarios involving navigation tasks.

5.2.3. Distance-based building sampling with simplex noise

The final step in obtaining Voronoi diagram-based urban maps is to sample buildings inside each Voronoi polygon to obtain the building map $\mathbf{M}_{\mathbf{v}} \in [0, 1]^{n \times n}$.

Adaptive building density

In reality, the building distance varies with respect to the distance to the city center. Therefore, the sampling density should vary based on the distance to the city center, so

5. Procedural map generation

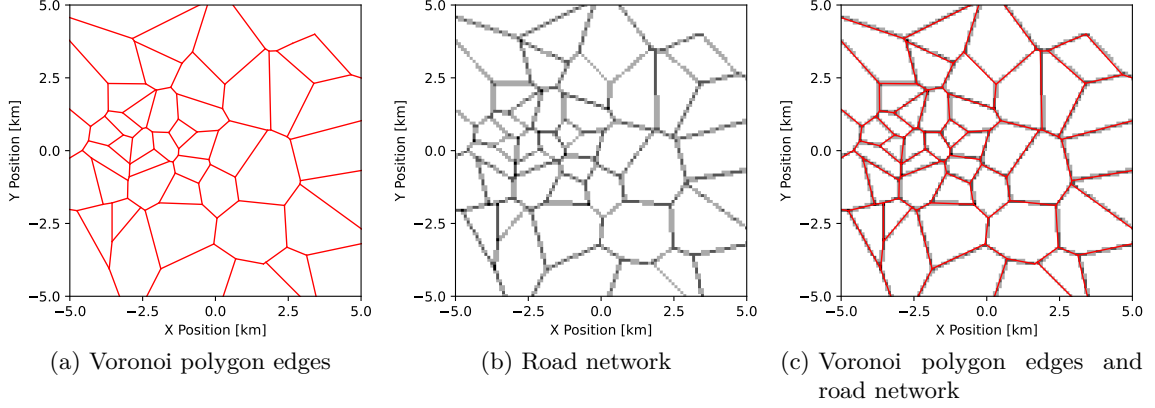


Figure 5.5.: Voronoi edges and the resulting road network. In (c), Voronoi polygon edges and the road network are plotted together, highlighting the process of constructing the road network based on Voronoi edges.

that the polygons closer to the city center should have a higher probability of containing buildings, while the density of buildings gradually decreases towards the suburban and countryside areas. By adapting the sampling density in this manner, the resulting urban map exhibits a more natural and plausible building distribution, similar to what is observed in real-life urban environments.

Let $\mathbf{x}_c \in \mathbb{R}^2$ represent the position of the city center and $\mathbf{x}_p \in \mathbb{R}^2$ the centroid of the polygon P (see (2.1.2) for the calculation of $\mathbf{x}_p$). The distance between the centroids of the polygons and the city center can be used to set a threshold, called *polygonwise noise threshold* $\beta_p(\mathbf{x}_p, \mathbf{x}_c) \in [0, 1]$. This threshold is used to set the building density in the polygon P .

There are several methods to decide on the value of $\beta_p(\mathbf{x}_p, \mathbf{x}_c)$ for each polygon P in the Voronoi diagram to establish the expected density behavior. One way can be manually setting the value $\beta_p(\mathbf{x}_p, \mathbf{x}_c)$ for each polygon. Another way is to divide the map into several subregions, according to the closeness of the polygons in each subregion, and consecutively set $\beta_p(\mathbf{x}_p, \mathbf{x}_c)$ for each subregion. Both these methods require manual effort and therefore, do not fit into the context of procedural generation. In addition, considering the second method, the change in the density of the buildings may look discrete instead of a continuous change in density. This discrete behavior may manifest in the actual map as a noncontinuous change in building density on the consecutive subregions. Therefore, a smoother and natural-looking density change in the map can be achieved by basing the $\beta_p(\mathbf{x}_p, \mathbf{x}_c)$ on a continuous function. This way, a smoother transition of building densities between the city center and the countryside is obtained.

The polygonwise noise threshold $\beta_p(\mathbf{x}_p, \mathbf{x}_c)$ depends on the Euclidean distance $\|\mathbf{x}_p - \mathbf{x}_c\|_2$

and it is calculated based on the hyperbolic tangent function $\tanh$ which allows for a smooth and continuous transition of building densities from the city center to the countryside. Then, $\beta_p(\mathbf{x}_p, \mathbf{x}_c)$ is calculated such that

$$\beta_p(\mathbf{x}_p, \mathbf{x}_c) = \frac{1}{2} \left(\tanh \left(\frac{\|\mathbf{x}_p - \mathbf{x}_c\|_2 - p_{\text{inf}}}{\psi} \right) + 1 \right), \quad (5.2.2)$$

where p_{inf} is the inflection point and ψ represents the sharpness. By adjusting the parameters p_{inf} and ψ , the shape of this function can be altered accordingly, as can be seen in Figure 5.6.

Calculating $\beta_p(\mathbf{x}_p, \mathbf{x}_c)$ this way provides smooth building density changes between polygons, in terms of the closeness to the city center. As these thresholds are obtained on a continuous function, smoother distribution of buildings is assured and an unnatural forming on the map is prevented.

Simplex noise for map generation

Unlike the basic map $\mathbf{M}_b$ where all the buildings are sampled on a uniform distribution (see Section 5.1), buildings in each Voronoi polygon are sampled on a simplex noise distribution ψ . This distribution is in the value range $[-0.5, 0.5]$. As simplex noise provides a locally correlated distribution, building locations are more closely related, resulting in a visually coherent urban landscape.

Given a point $\mathbf{x}_e$ in a polygon P , the sampled simplex noise for this point is denoted as $s_e \sim S$. This noise $s_e \in [-0.5, 0.5]$ is first normalized into $\bar{s}_e \in [0, 1]$ by $\bar{s}_e = s_e + 0.5$, in order to ensure coherence with the range of $\beta_p(\mathbf{x}_p, \mathbf{x}_c) \in [0, 1]$. The normalized noise $\bar{s}_e \in [0, 1]$ is then compared to $\beta_p(\mathbf{x}_p, \mathbf{x}_c)$ such that

$$x_e = \begin{cases} \bar{s}_e & \text{if } \bar{s}_e > \beta_p(\mathbf{x}_p, \mathbf{x}_c) \\ 0 & \text{otherwise.} \end{cases} \quad (5.2.3)$$

If the noise $\bar{s}_e$ of the point $\mathbf{x}_e$ is bigger than the polygonwise noise threshold $\beta_p(\mathbf{x}_p, \mathbf{x}_c)$, then the value of this point is set to the $\bar{s}_e$, corresponding to the building height on the point $\mathbf{x}_e$. Therefore, the simplex noise distribution is used for both the sampling of the building positions and the corresponding building heights.

Consider two polygons P_1 and P_2 , with centroids $\mathbf{x}_{p_1}$ and $\mathbf{x}_{p_2}$, respectively. If P_1 is closer to the city center $\mathbf{x}_c$ than P_2 , then based on the steps explained above, $\beta_p(\mathbf{x}_{p_1}, \mathbf{x}_c) < \beta_p(\mathbf{x}_{p_2}, \mathbf{x}_c)$, meaning that the probability of a building to be sampled in P_1 is greater than that in P_2 . This consequently leads to a higher building density in P_1 .

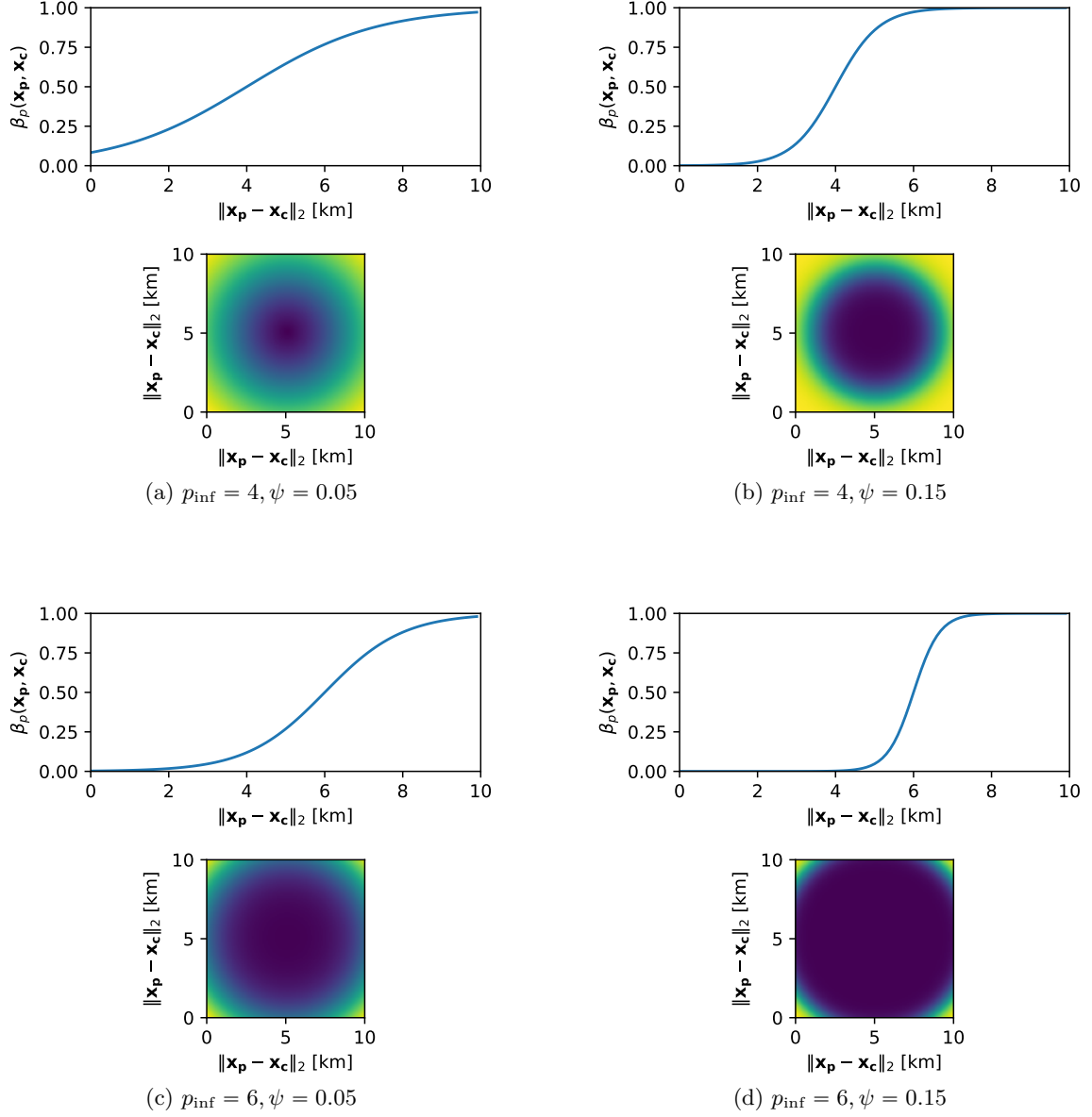


Figure 5.6.: Scalar and 2D views of the polygonwise noise threshold function. The sharpness ψ influences the rate at which the threshold increases or decreases with respect to the distance input. As the sharpness ψ increases, the curve has steeper slopes around inflection point p_{inf} and more closely resembles a step function. Meanwhile, the inflection point p_{inf} controls the point of maximum curvature on the threshold curve.

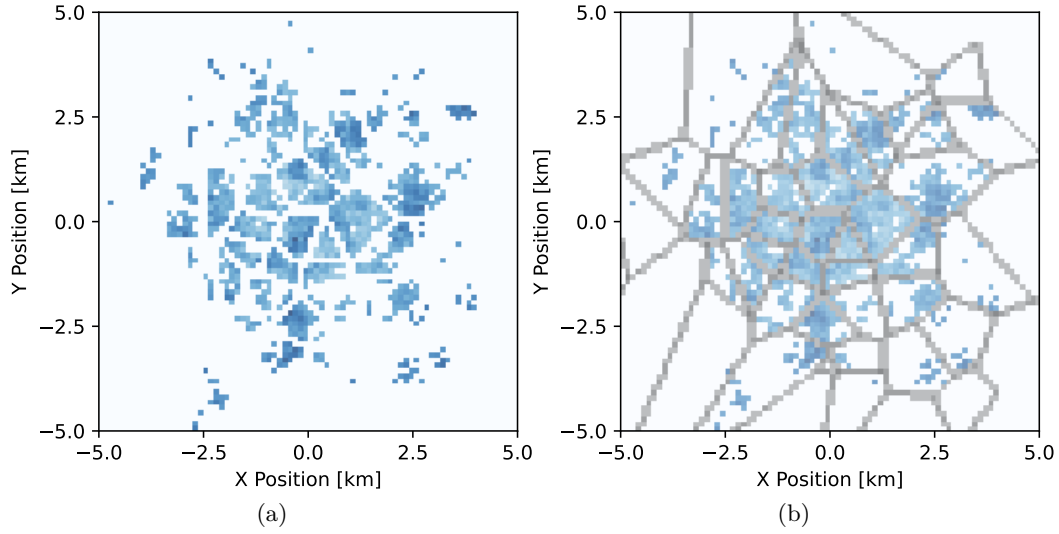


Figure 5.7.: Visualization of the sampled buildings. In (a), only the sampled buildings are present. In (b), the buildings are displayed together with the road network.

After processing all the points and polygons using (5.2.2) and (5.2.3), the building map $\mathbf{M}_v \in [0, 1]^{n \times n}$ containing non-uniform buildings is ultimately obtained. A summary of steps to procedurally generate complex urban maps with non-uniform buildings and road networks is given below.

1. Sample generating points $\mathbf{x}_g$ and consequently partition the plane using Voronoi diagrams $\rightarrow$ Section 5.2.1
2. Construct the road network $\mathbf{M}_r$ based on the edges of the Voronoi polygons $\rightarrow$ Section 5.2.2
3. Compute polygonwise noise threshold $\beta_p(\mathbf{x}_p, \mathbf{x}_c)$ of each polygon P and sample buildings in each polygon using the simplex noise distribution S to obtain the building map $\mathbf{M}_v \rightarrow$ Section 5.2.3

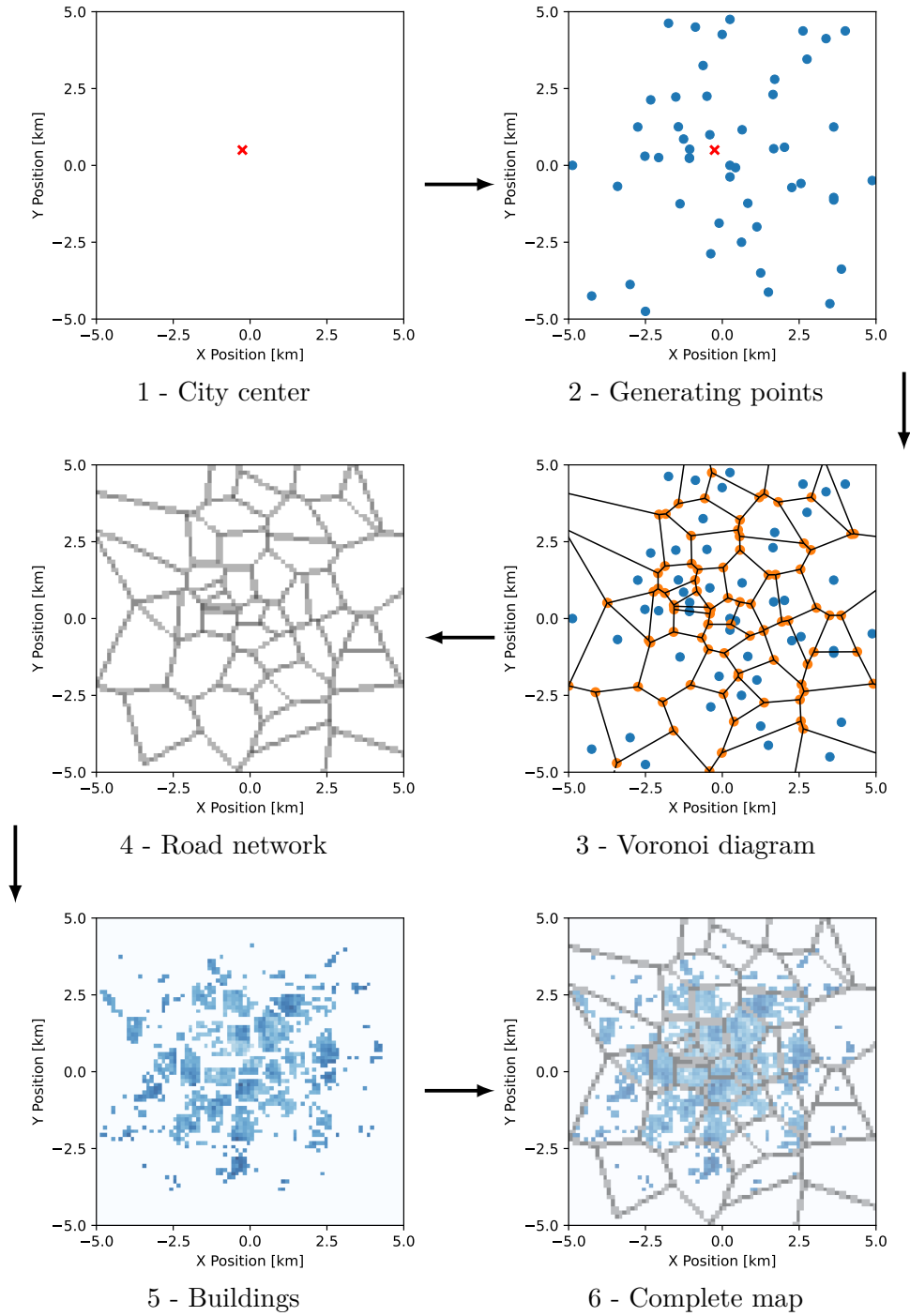


Figure 5.8.: Voronoi diagram-based map generation pipeline. Firstly, the city center and generating points are sampled. Then, Voronoi polygons are obtained based on these points. Using the edges of the Voronoi polygons, the road network is constructed. In the final step, buildings are sampled inside the Voronoi polygons. The last figure shows the sampled buildings and the road network together.

6. Exploiting context knowledge and awareness in MARL

The maps introduced in the previous chapter contain information that gives rise to inherent knowledge, known as context knowledge. Alongside the state-related observations discussed in Section 4.3.1, this context knowledge leads to a new category of observations. This chapter elaborates on the concept of context knowledge and demonstrates how this knowledge is utilized by agents within the framework of tracking and evasion.

6.1. Fundamentals of context knowledge

This section begins by introducing the characteristics of the environment that form the basis of RL interactions. It then explains how the agents are adapted to the newly introduced maps. Finally, the fundamental functionalities and effects of the maps on the agents, such as building collision and occlusion, are presented.

6.1.1. Coordinate system transformation

The environment is confined in a square with sides of length l in meters, which can be represented as a 2D Cartesian coordinate system, where the origin is situated at the midpoint of the square. A point $\mathbf{x}_{\text{env}} \in \mathbf{E}$ in the environment $\mathbf{E} = [-\frac{l}{2}, \frac{l}{2}]^2$ can be represented relative to this coordinate system.

The maps are computer-generated graphics, as explained in Chapter 5. These maps have a square shape of size $(n \times n)$. The origin of the map's coordinate system is situated at the lower-left corner of the image.

While the environment points can have float values to represent continuous positions in the environment, the map is represented by pixels, which require integer point representation. The presence of two coordinate systems, one describing the environment in meters and the other representing the map using pixel coordinates, requires a coordinate system transformation.

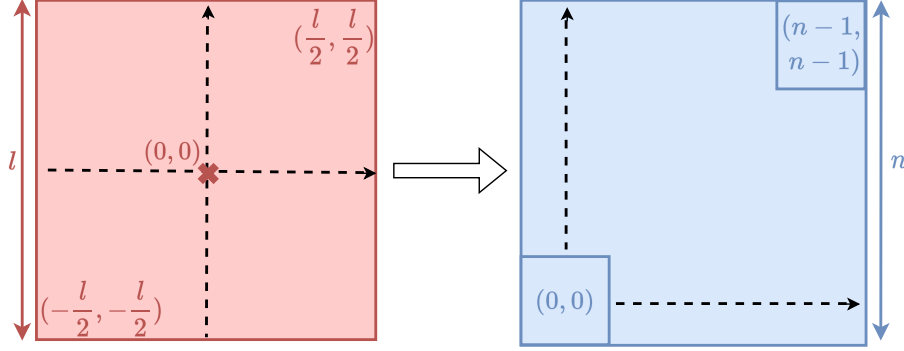


Figure 6.1.: Environment and map. The left plot depicts the environment with its origin located at the midpoint, while the right plot illustrates the map with its origin situated at the lower-left corner.

A pixel position in the map is represented as $\mathbf{x}_{\text{map}} = (x_{\text{map}}, y_{\text{map}}) \in \{0, \dots, n-1\}^2$, where x_{map} and y_{map} are the horizontal and vertical pixel distances from the origin $(0,0)$, respectively. An environment point $\mathbf{x}_{\text{env}}$ can be transformed into a map pixel position $\mathbf{x}_{\text{map}}$.

It is first shifted by $\frac{l}{2}$ to align the origins of the map and environment. Then, it is scaled down by dividing by $\frac{l}{n-1}$ to fit within the size range of the map. The last step is to round the result to the nearest integer, as the pixel indices on the map can only have integer values.

$$\mathbf{x}_{\text{map}} = \text{round} \left(\left(\mathbf{x}_{\text{env}} + \frac{l}{2} \right) \frac{n-1}{l} \right) \quad (6.1.1)$$

Agent positions

The environment positions of the observer and target at the time t are denoted as $\mathbf{x}_{\mathcal{O},t} = (x_{\mathcal{O},t}, y_{\mathcal{O},t}) \in \mathbb{R}^2$ and $\mathbf{x}_{\mathcal{T},t} = (x_{\mathcal{T},t}, y_{\mathcal{T},t}) \in \mathbb{R}^2$, respectively. For the sake of simplicity, the altitudes of both agents, indicating their vertical positions with respect to the surface, are fixed. For this reason, the z -axis components of the agents are not explicitly given in their point definitions, but rather presented as scalars $h_{\mathcal{O}}$ and $h_{\mathcal{T}}$.

6.1.2. Environment boundary constraint

An episode terminates if any of the following criteria are satisfied (see Section 4.3.4)

$$t > t_{\max} \quad d_t > d_{\max} \quad \tilde{d}_t > d_{\max}, \quad (6.1.2)$$

where $t_{\max}$ is the maximum episode length, $d_{\max}$ is the maximum target distance, $\tilde{d}_t$ is the estimation of the distance between the observer and target, and d_t is the actual distance between the observer and target.

An additional criterion is introduced to prevent agents from navigating outside the valid environment space. If either of the agents ventures beyond the defined environment boundaries explained in Section 6.1.1 such that the condition below is met

$$\mathbf{x}_{\mathcal{O},t} \notin \mathbf{E} \quad \vee \quad \mathbf{x}_{\mathcal{T},t} \notin \mathbf{E}, \quad (6.1.3)$$

then the episode is terminated.

A notable concern arises with respect to the estimated position of the target, which is based on the EKF estimations of the observer. Due to the inherent noise in the estimation process, the estimated target position does not align precisely with the actual target position. This creates a challenge in preventing the estimated target from appearing out of the environment.

To address this issue, it would be possible to check the estimated target position against the environment boundary, analogous to (6.1.3). However, this approach may lead to cases where the target is still inside the environment but the estimated position has erroneously crossed the boundary. This enables the target to trick the observer into tracking the estimated target and lure it outside the environment, resulting in early termination, which is advantageous for the target due to the defined negative reward.

To prevent this exploitation, the estimated target position is clipped to the valid environment space. Consequently, the estimated target position always lies within the environment boundaries and the target can no longer deceive the observer. This ensures that the target learns the expected evasive behavior rather than adopting deceptive strategies.

6.1.3. Building collision

The target is modeled as a ground vehicle platform. In the context of an urban-like map consisting of buildings, roads, and open terrain, one critical aspect is building collision.

In the realm of RL, agents are often not explicitly instructed on how to avoid disadvantageous situations. Instead, they are expected to learn from the consequences of such

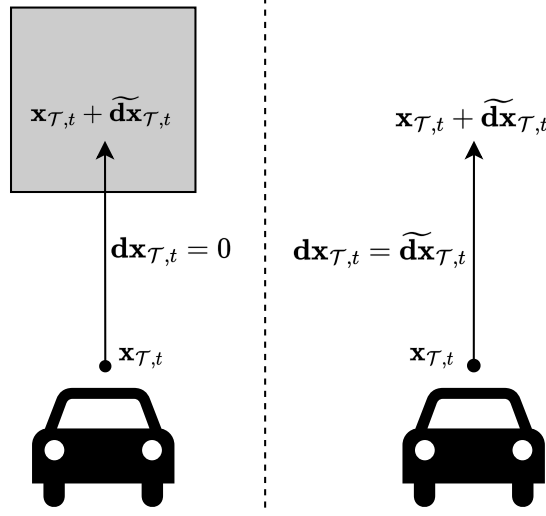


Figure 6.2.: Building collision. When the target encounters a building, its movement is halted. Conversely, if the subsequent position falls within empty terrain, the target continues its motion.

situations, adapt their behavior based on the reward, and eventually develop policies that help them steer clear of unfavorable circumstances. In this regard, the target's response to building collision is simulated by letting it come to a halt when encountering buildings. Note that the observer is designed to fly at a height greater than the tallest building on the map, ensuring that it remains unaffected by building collisions.

The position of an agent $\mathcal{A}$ at time step $t + 1$ is denoted as $\mathbf{x}_{\mathcal{A},t+1}$, which is computed by adding the displacement $\mathbf{d}\mathbf{x}_{\mathcal{A},t}$ calculated at time t to the previous position $\mathbf{x}_{\mathcal{A},t}$. To compute $\mathbf{d}\mathbf{x}_{\mathcal{A},t}$, the target computes the change in position $\widetilde{\mathbf{d}}\mathbf{x}_{\mathcal{A},t}$ based on (4.2.1), that is the change that the agent supposed to have in between time t and $t + 1$.

If the next position of the target $\mathbf{x}_{T,t} + \widetilde{\mathbf{d}}\mathbf{x}_{T,t}$ is inside a building, the target is forced to stop by setting $\mathbf{d}\mathbf{x}_{T,t} = 0$. If the next position represents empty terrain, then $\mathbf{d}\mathbf{x}_{T,t}$ is set to $\widetilde{\mathbf{d}}\mathbf{x}_{T,t}$, such that

$$\mathbf{d}\mathbf{x}_{T,t} = \begin{cases} 0 & \text{if } \mathbf{x}_{T,t} + \widetilde{\mathbf{d}}\mathbf{x}_{T,t} \text{ is in a building} \\ \widetilde{\mathbf{d}}\mathbf{x}_{T,t} & \text{otherwise.} \end{cases} \quad (6.1.4)$$

6.1.4. Adaptive target speed

In Section 5.2.2 it was shown how Voronoi diagrams can be utilized to generate road maps. As in real life, off-road terrain typically impedes the movement of a ground vehicle and thus causes the vehicle to slow down. Therefore, the speed of the target is adaptive, in relation to the roads, such that its speed is reduced if the target goes off the road.

Similar to the building collision explained in Section 6.1.3, the next position of the target $\mathbf{x}_{\mathcal{T},t+1}$ and the change in position $\widetilde{\mathbf{d}\mathbf{x}}_{\mathcal{T},t}$ are computed based on (4.2.1). If the next position of the target $\mathbf{x}_{\mathcal{T},t} + \widetilde{\mathbf{d}\mathbf{x}}_{\mathcal{T},t}$ corresponds to off-road, then $\mathbf{d}\mathbf{x}_{\mathcal{T},t}$ is set to scaled down $\widetilde{\mathbf{d}\mathbf{x}}_{\mathcal{T},t}$ by a scalar $\alpha_{\text{slow}} \in [0, 1]$ and this results in reduced speed. If the next position of the target is on-road, then $\mathbf{d}\mathbf{x}_{\mathcal{T},t}$ is set to $\widetilde{\mathbf{d}\mathbf{x}}_{\mathcal{T},t}$, such that

$$\mathbf{d}\mathbf{x}_{\mathcal{T},t} = \begin{cases} \widetilde{\mathbf{d}\mathbf{x}}_{\mathcal{T},t} \cdot \alpha_{\text{slow}} & \text{if } \mathbf{x}_{\mathcal{T},t} + \widetilde{\mathbf{d}\mathbf{x}}_{\mathcal{T},t} \text{ is off-road} \\ \widetilde{\mathbf{d}\mathbf{x}}_{\mathcal{T},t} & \text{otherwise.} \end{cases} \quad (6.1.5)$$

6.1.5. Initial positions

In maps consisting of only buildings and empty terrain (see Section 5.1), the initial position of the target is sampled on the empty terrain. For the complex maps including buildings, roads, and empty terrain (see Section 5.2), the target's initial position is set on a road pixel. The observer is also decided to be initially located at the exact center of the map. This restricts the initialization of valid configurations of the environment.

Another modification involves the provision of exact target initial position information to the observer. In the original work described in Chapter 4, the observer only possessed knowledge of the estimated initial position of the target, and the initializations were solely based on a normal distribution, as explained in Section 4.2. However, as maps are involved in this work, this method sometimes led to the estimated target initial position being sampled outside the map. This caused the observer to go out of the environment. To address this problem, the observer henceforth knows the exact initial position of the target. It should be noted that beginning from the first measurement, the exact and estimated target positions are no longer exactly the same.

6.1.6. Occlusion

The target is tracked using a radar sensor that the observer is equipped with. In the original work described in Chapter 4, it is explained that the observer is equipped with

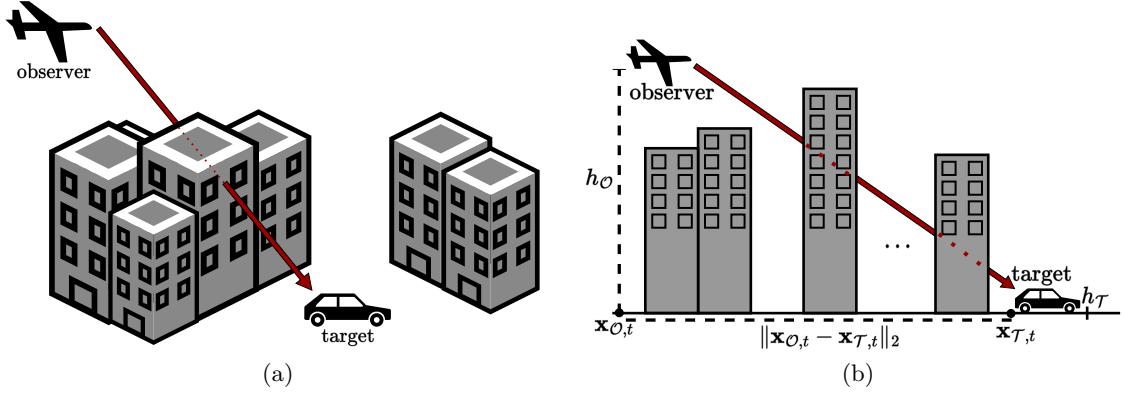


Figure 6.3.: 2D and 3D illustrations of the occlusion. If the buildings in between the agents block the line of sight of the target, then it is occluded.

a range-bearing sensor that is restricted in its FOV, as explained in Section 4.1. In conjunction with an EKF, the FOV radar sensor allows the observer to estimate the range and bearing of the target relative to its own position.

In this work, the simulated sensor is extended with an occlusion model. This change was driven by the inclusion of buildings, which may block the direct line of sight to the sensor. As this increases the complexity of the sensor model, it is further assumed the sensor has an unlimited FOV.

Occlusion occurs when one object or part of an object obstructs the view of another object, leading to partial or complete hiddenness of the obstructed object. In visual scenes, occlusion is a natural occurrence that can impact the perception of the environment.

As the overall goal of the observer is to track the target and keep it in its line of sight, occlusion can greatly affect the tracking performance of the observer. Furthermore, it should be noted that the reward function, as detailed in Section 4.3.3, is intricately linked to the observer's tracking performance. This reward function is designed to incentivize the observer to maximize its tracking capabilities. When occlusion occurs and hinders the observer's tracking by causing disruptions in the sensing of the target, it directly affects the reward earned by the observer and subsequently influences its tracking performance.

Mathematical foundations of occlusion

Based on the calculations using the 3D positions of the observer $\hat{\mathbf{x}}_{\mathcal{O},t} = (\mathbf{x}_{\mathcal{O},t}, h_{\mathcal{O}}) \in \mathbb{R}^3$ and the target $\hat{\mathbf{x}}_{\mathcal{T},t} = (\mathbf{x}_{\mathcal{T},t}, h_{\mathcal{T}}) \in \mathbb{R}^3$, it is possible to determine whether occlusion is present.

In the 3D scenery, the slope u_t of the line of sight at time t can be computed as

$$u_t = \frac{h_{\mathcal{O}} - h_{\mathcal{T}}}{\|\mathbf{x}_{\mathcal{O},t} - \mathbf{x}_{\mathcal{T},t}\|_2}, \quad (6.1.6)$$

where $h_{\mathcal{O}} - h_{\mathcal{T}}$ is the altitude difference of the agents and $\|\mathbf{x}_{\mathcal{O},t} - \mathbf{x}_{\mathcal{T},t}\|_2$ is the 2D Euclidian distance between $\mathbf{x}_{\mathcal{O},t}$ and $\mathbf{x}_{\mathcal{T},t}$,

Only the buildings on the points that are lying along the line of sight may intersect the line, thus blocking the sight of the observer. For this reason, considering only these points is more efficient than considering all the points on the map.

The *occlusion threshold* of a point, denoted as β_o , is defined based on the slope u_t and the distance to the agents. For a point that lies along the line of sight, if the height of the building on that point exceeds the corresponding occlusion threshold β_o , then this building intersects the line of sight of the observer and this results in occlusion. In other words, the occlusion threshold of a point is the maximum allowable height of the building located on this point so that this building does not block the line of sight.

Given a point $\hat{\mathbf{x}}_{\mathbf{p}} \in \mathbb{R}^3$ in the map with the 2D projection onto the xy-plane $\mathbf{x}_{\mathbf{p}}$, the occlusion threshold of this point $\beta_o(\hat{\mathbf{x}}_{\mathbf{p}})$ can be calculated using the slope u_t , such that

$$\beta_o(\hat{\mathbf{x}}_{\mathbf{p}}) = u_t \|\mathbf{x}_{\mathbf{p}} - \mathbf{x}_{\mathcal{T},t}\|_2 + h_{\mathcal{T}}. \quad (6.1.7)$$

Consider M points $(\hat{\mathbf{x}}_1, \dots, \hat{\mathbf{x}}_M)$ lying along the line of sight having the respective building heights $(h_1, \dots, h_M)$ and the occlusion thresholds $(\beta_o(\hat{\mathbf{x}}_1), \dots, \beta_o(\hat{\mathbf{x}}_M))$. Then, occlusion occurs if any of the buildings on these points is taller than the occlusion threshold of its point. This can be expressed as a logical condition, which is satisfied when occlusion is present.

$$(h_1 > \beta_o(\hat{\mathbf{x}}_1)) \vee \dots \vee (h_M > \beta_o(\hat{\mathbf{x}}_M)) = \bigvee_{i=1}^M (h_i > \beta_o(\hat{\mathbf{x}}_i)) \quad (6.1.8)$$

The ratio between the height of the buildings and the altitude of the observer plays a critical role in the occurrence of occlusion. If the buildings are tall and close to the altitude of the observer, occlusion can happen frequently, making it challenging for the observer to detect and track the target effectively. On the other hand, if the buildings are short in comparison to the altitude of the observer, occlusion may rarely or never occur, as there is a lower chance of obstruction. In this case, the observer can track the target relatively easily.

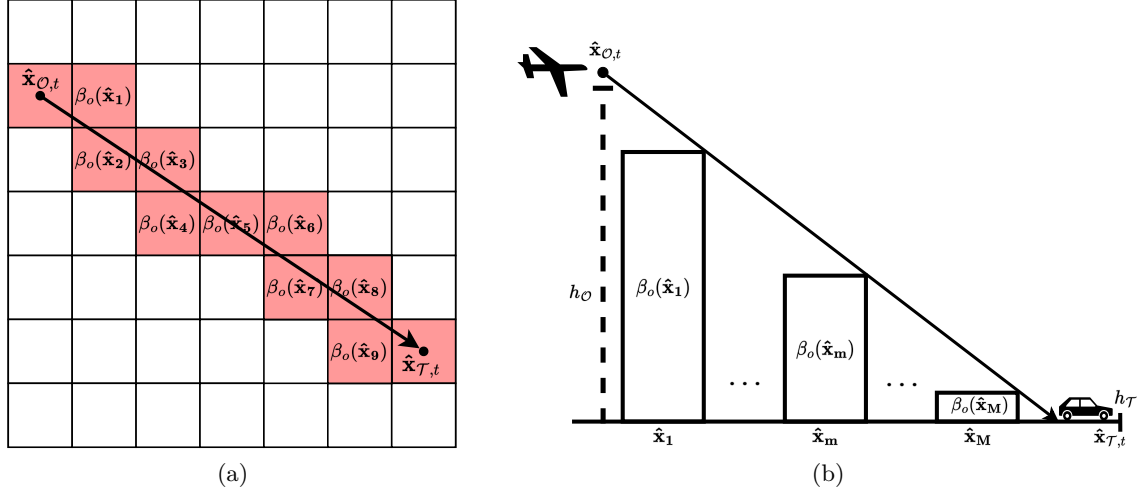


Figure 6.4.: Illustrations of occlusion thresholds. Considering the observer and target as points in 3D space, the line of sight connects the two points. In (a), a top-down grid view of an exemplary positioning of agents, the line of sight, and exemplary M points ($\hat{\mathbf{x}}_1, \dots, \hat{\mathbf{x}}_M$) between agents (painted in red) are illustrated. In (b), the line of sight and the occlusion thresholds of these exemplary M points between agents are depicted.

6.2. Expanding observation spaces

This section starts with the adaptation and expansion of the observation spaces. Subsequently, the construction of a CNN model is detailed, illustrating its role in processing the image input data to generate map feature vectors.

6.2.1. State-related observations

As described in Section 4.3.1, the original observation spaces of both agents $\mathbf{o}_{A,t}$ contain state-related observations. Some of these observations are inherent to the nature of the agent itself, while others are derived from the state of the other agent and their relative positions and interactions. These observation spaces from (4.3.3) are represented below as a reminder.

$$\begin{aligned}\mathbf{o}_{O,t} &= [\omega_{O,t}, \kappa_t, \tilde{d}_t, \sin(\tilde{\theta}_{T,t}), \cos(\tilde{\theta}_{T,t}), S_t] \\ \mathbf{o}_{T,t} &= [\omega_{T,t}, d_t, \sin(\theta_{O,t}), \cos(\theta_{O,t})]\end{aligned}\tag{6.2.1}$$

One of the modifications made to the agent model, as detailed in the original work, in-

volves modifying the action spaces. Instead of maintaining a continuous action space as explained in Section 4.3.2, the action spaces are now discretized. These discrete action spaces comprise three possible actions, denoted as $a_{\mathcal{A},t} \in \{0, 1, 2\}$. The motivation behind this change is to simplify the action spaces and facilitate better control over the rotational velocity $\omega_{\mathcal{A},t}$, which is calculated based on the selected action such that

$$\omega_{\mathcal{A},t} = (a_{\mathcal{A},t} - 1) \omega_{\mathcal{A},\max}, \quad (6.2.2)$$

where $\omega_{\mathcal{A},\max}$ refers to the maximal rotational velocity limit of the agent. It is ensured that $\omega_{\mathcal{A},t}$ is still in the range $[-1, 1]$.

Furthermore, an additional inclusion to the original work involves incorporating the global positions $\mathbf{x}_{\mathcal{A},t}$ and rotations $\phi_{\mathcal{A},t} \in [-\pi, \pi]$ of the agents on the map. This addition serves the purpose of localizing agents within a global map context. Consequently, a new set of observations should be incorporated into the existing observations, taking into account the actual agent positions and rotations on the map.

The rotation $\phi_{\mathcal{A},t}$ is decomposed into sine and cosine terms, as similarly done in the original work as explained in Section 4.3.1, such that

$$\phi_{\mathcal{A},t} = \begin{pmatrix} \cos(\phi_{\mathcal{A},t}) \\ \sin(\phi_{\mathcal{A},t}) \end{pmatrix}, \quad (6.2.3)$$

addressing the discontinuity of the angle representation at $\pm\pi$. The resulting representation is continuous and thus more suitable for learning [1].

In addition, $\mathbf{x}_{\mathcal{A},t} \in [-l, l]^2$ is normalized into $\bar{\mathbf{x}}_{\mathcal{A},t} \in [-1, 1]^2$ before it is integrated into the observation space, such that

$$\bar{\mathbf{x}}_{\mathcal{A},t} = \frac{\mathbf{x}_{\mathcal{A},t} + l}{l} - 1, \quad (6.2.4)$$

State-related observations, denoted as $\mathbf{s}_{\mathcal{O},t} \in [-1, 1]^{10}$ and $\mathbf{s}_{\mathcal{T},t} \in [-1, 1]^8$, are then formed such that

$$\begin{aligned} \mathbf{s}_{\mathcal{O},t} &= [\mathbf{o}_{\mathcal{O},t}, \bar{x}_{\mathcal{O},t}, \bar{y}_{\mathcal{O},t}, \sin(\phi_{\mathcal{O},t}), \cos(\phi_{\mathcal{O},t})] \\ \mathbf{s}_{\mathcal{T},t} &= [\mathbf{o}_{\mathcal{T},t}, \bar{x}_{\mathcal{T},t}, \bar{y}_{\mathcal{T},t}, \sin(\phi_{\mathcal{T},t}), \cos(\phi_{\mathcal{T},t})]. \end{aligned} \quad (6.2.5)$$

It should be noted that the termination criteria, presented in Section 6.1.2, prevent the observations from exceeding their definition space.

6.2.2. Context-related observations

In addition to the state-related observations described in (6.2.5), the maps introduced in Chapter 5 and this section, lead to a new set of observations, called *context-related observations* $\mathbf{c}_{A,t}$.

Building maps

The main components of the context-related observations are the basic maps $\overline{\mathbf{M}}_b$ from Section 5.1 and Voronoi diagram-based building maps $\mathbf{M}_v$ from Section 5.2.

Position maps

In addition to building maps, using the position maps $\mathbf{M}_{pos,A,t}$ of the agent positions $\mathbf{x}_{A,t}$ contributes to the knowledge about the environment. Therefore, visual representations of the positions of the agents on the map are constructed.

Rather than representing the agent's position as a singular point on the map, the positions of agents are depicted on the position maps, where higher values are assigned to closer points to the agent's position, and lower values to more distant points. These position maps serve as visual representations indicating the relative distance of surrounding pixels from the agent's position. This approach contributes to creating smoother visualizations.

Given the position of an agent $\mathbf{x}_{A,t}$ and a point $\mathbf{x}_r \in \mathbb{R}^2$ on an empty map, the Euclidean distance between $\mathbf{x}_{A,t}$ and all the points $\mathbf{x}_r$ on this empty map are calculated. Then, a min operation is applied on these distances and the scalar κ_p , so that only the distances smaller than κ_p are considered. Consequently, the resulting values are normalized into $[0, 1]$ by dividing by κ_p . In the last step, the normalized values are inverted.

$$\mathbf{M}_{pos,A,t} = 1 - \frac{\min(\|\mathbf{x}_{A,t} - \mathbf{x}_r\|_2, \kappa_p)}{\kappa_p} \quad (6.2.6)$$

Even though the building maps and road maps do not change throughout an episode, the positions of the agents change at each episode step, rendering the position maps dynamic.

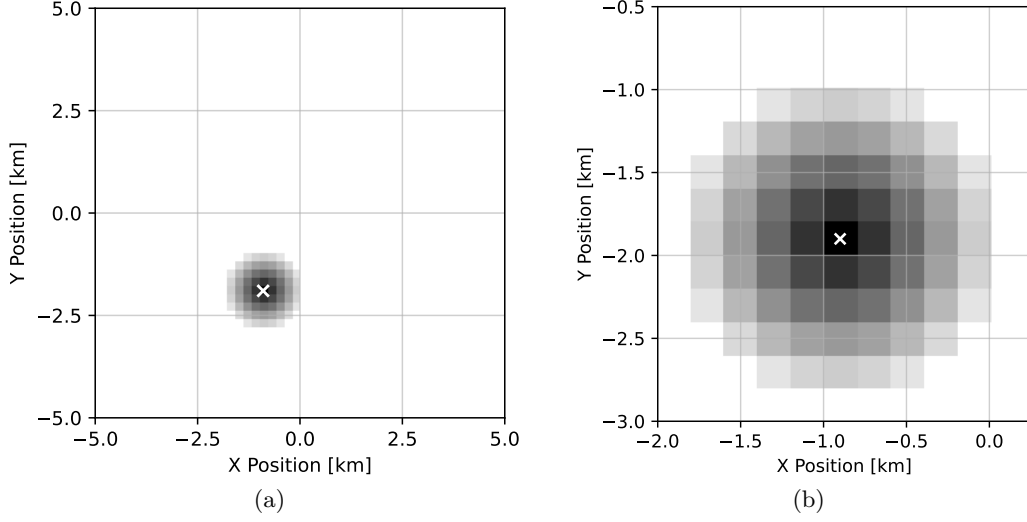


Figure 6.5.: An exemplary agent position map is depicted. The white cross represents the position of the agent. In (a), the map represents the spatial distribution of distances from the agent's position to the surrounding pixels. The surrounding pixels exhibit lower values compared to those closer to the agent's position. A close-up version of the position map from (a) is shown in (b).

Rotation maps

Rotation maps $\mathbf{M}_{rot,A,t}$ contain a ray that indicates the rotation and position of the agent. The ray starts from the position of the agent $\mathbf{x}_{A,t}$ and extends in the direction of the rotation of the agent $\phi_{A,t}$. Path of the ray, denoted as $\mathbf{g}_t$, can be described using the agent's position $\mathbf{x}_{A,t}$ and the rotation $\phi_{A,t}$, such that

$$\mathbf{g}_t = \{\mathbf{x}_{A,t} + t \begin{pmatrix} \cos(\phi_{A,t}) \\ \sin(\phi_{A,t}) \end{pmatrix} \mid t \geq 0\}, \quad (6.2.7)$$

where $\begin{pmatrix} \cos(\phi_{A,t}) \\ \sin(\phi_{A,t}) \end{pmatrix}$ is the decomposition of the rotation $\phi_{A,t}$, as explained in (6.2.3).

6.2.3. Forming context-related observations

The maps that are introduced so far, form the context-related observations $\mathbf{c}_{A,t}$. The maps are stacked in order to construct $\mathbf{c}_{A,t} \in [0, 1]^{n \times n \times k}$, where k is the number of maps forming $\mathbf{c}_{A,t}$.

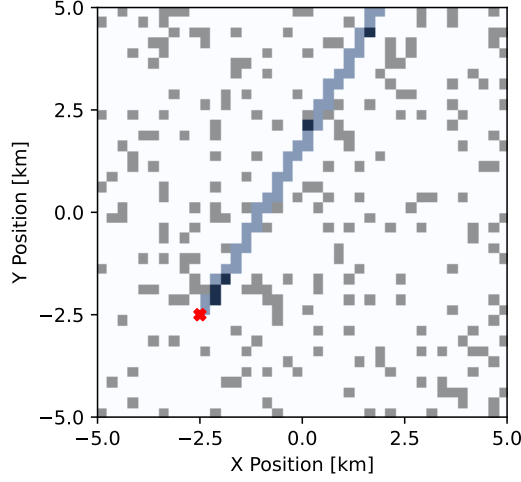


Figure 6.6.: Rotation map. The ray, painted blue, starts from the agent’s position and has a direction aligning with the rotation of the agent. It is plotted together with the map $\bar{\mathbf{M}}_b$, and the position of the agent, which is marked as the red cross.

The list of these maps is shown below.

1. Basic map $\bar{\mathbf{M}}_b \rightarrow$ Section 5.1
2. Voronoi-based building map $\mathbf{M}_v \rightarrow$ Section 5.2.3
3. Road network map $\mathbf{M}_r \rightarrow$ Section 5.2.2
4. Agent position maps $\mathbf{M}_{pos,\mathcal{O},t}$, $\mathbf{M}_{pos,\mathcal{T},t}$ and $\tilde{\mathbf{M}}_{pos,\mathcal{T},t} \rightarrow$ Section 6.2.2
5. Agent rotation maps $\mathbf{M}_{rot,\mathcal{O},t}$ and $\mathbf{M}_{rot,\mathcal{T},t} \rightarrow$ Section 6.2.2

It should be noted that the building maps $\bar{\mathbf{M}}_b$ and $\mathbf{M}_v$, and the road network map $\mathbf{M}_r$ stay same through the whole episode. On the other hand, the agent position maps $\mathbf{M}_{pos,\mathcal{O},t}$, $\mathbf{M}_{pos,\mathcal{T},t}$ and $\tilde{\mathbf{M}}_{pos,\mathcal{T},t}$, as well as the agent rotation maps $\mathbf{M}_{rot,\mathcal{O},t}$ and $\mathbf{M}_{rot,\mathcal{T},t}$ change at each time step t due to the change in agent positions and rotations.

Consequently, the observation spaces of the agents consist of both the state-related observations $\mathbf{s}_{\mathcal{A},t}$, described in (6.2.5), and the context-related observations $\mathbf{c}_{\mathcal{A},t}$.

The next section explains how these observations are used by agents to develop policies in order to perform tracking and evasion.

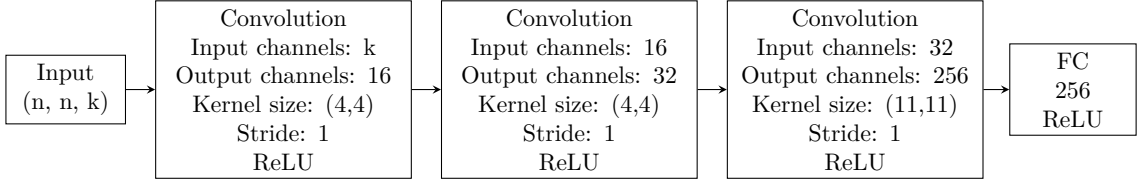


Figure 6.7.: Architecture of the CNN. This network takes $\mathbf{c}_{\mathcal{A},t}$ of size (n, n, k) as input, and applies three convolutions to the input images in order to extract hierarchical features. These features are then processed by an FC layer to produce a feature vector of size 256.

6.3. Utilizing CNN in actor-critic RL model

6.3.1. Processing context-related observations with a CNN

The conventional approach for agents to make use of the observations involves directly inputting the complete observations into the actor-network model representing the policy function, similar to the method employed in the original work with the model depicted in Figure 4.3. As the current observation spaces are significantly larger than the original observation spaces, this leads to a high number of trainable parameters in the network, thus the training process becomes time-consuming and computationally inefficient.

CNNs are proven to be effective in extracting knowledge from the images [9], [11], [15], [73], and generating feature vectors, which are the compressed representations of the input data. Therefore, the maps in the context-related observations $\mathbf{c}_{\mathcal{A},t}$ can be processed by a CNN in order to generate a feature vector that represents $\mathbf{c}_{\mathcal{A},t}$. Processing the maps with a CNN also preserves locality, as it processes only a kernel-sized portion of the map locally.

The architecture of the CNN is shown in Figure 6.7. It is utilized to extract a feature vector, which represents a compressed representation of the input $\mathbf{c}_{\mathcal{A},t}$, from the context-related observations $\mathbf{c}_{\mathcal{A},t}$.

6.3.2. CNN-integrated RL model

Numerous studies explored the integration of CNNs into RL models, as detailed in the literature review in Section 3.1.4. In this work, the CNN is integrated into the RL model in order to process context-related observations $\mathbf{c}_{\mathcal{A},t}$. Furthermore, the state-related observations $\mathbf{s}_{\mathcal{A},t}$ are processed with two FC layers using the ReLU activation function. After $\mathbf{c}_{\mathcal{A},t}$ and $\mathbf{s}_{\mathcal{A},t}$ are processed, the outputs are concatenated. Based on the concatenated vector, the action distribution and the related value are computed by the PPO algorithm.

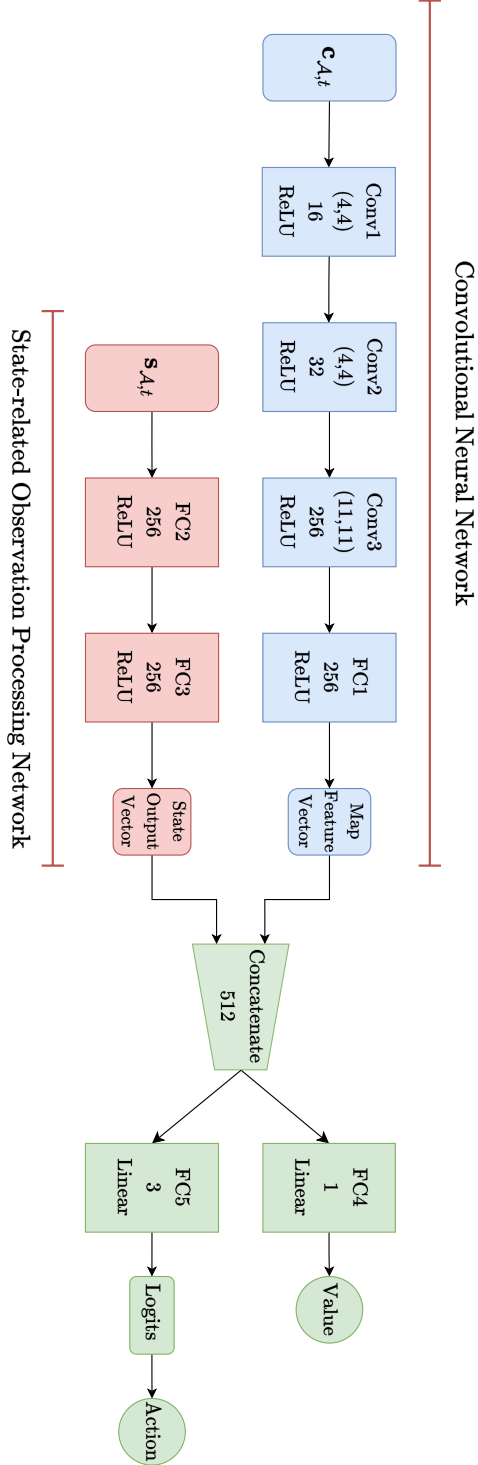


Figure 6.8.: CNN-integrated actor-critic RL model. The model consists of the CNN and the state-related observation processing network. Outputs of these networks are combined through concatenation, leading to the computation of the value function by the critic network and the extraction of action logits by the actor network. The parameters of the convolutional layers such as kernel sizes, output channels, and activation functions, are presented within the respective layer nodes. Furthermore, information about the output sizes and activation functions of the fully connected layers is provided below their corresponding layer names.

7. Results and evaluations

In this chapter, the results are displayed, and the evaluations and discussions are presented. In the first section, Section 7.1, the procedural map generation technique is evaluated, and a comparison between basic map generation and Voronoi-based map generation is provided.

The remainder of this chapter focuses on the RL experiments. In Section 7.2, the escape task and the tracking and evasion task are introduced, and the related scenarios are defined. In Section 7.3, the results of these tasks are presented, accompanied by evaluations and discussions.

7.1. Procedural map generation

The maps utilized in this thesis are automatically generated through procedural map generation, featuring randomly generated content. In contrast to manual map design and creation, procedural map generation enables the automatic generation of maps throughout the training process, without necessitating human intervention.

An example basic map and Voronoi diagram-based map are given in Figure 7.1. In the basic map, the buildings and building heights are randomly generated based on the uniform distribution.

Meanwhile, the Voronoi diagram-based map contains irregular building placement, and the road network, and follows adaptive building sampling density. The buildings in this map are sampled so that more buildings are placed around the city center, whereas the countryside is not as populated as the city center. These characteristics of this map are influenced by urban maps, as the aim is to obtain maps that contain similar characteristics as the typical urban map, as explained in Section 3.2.

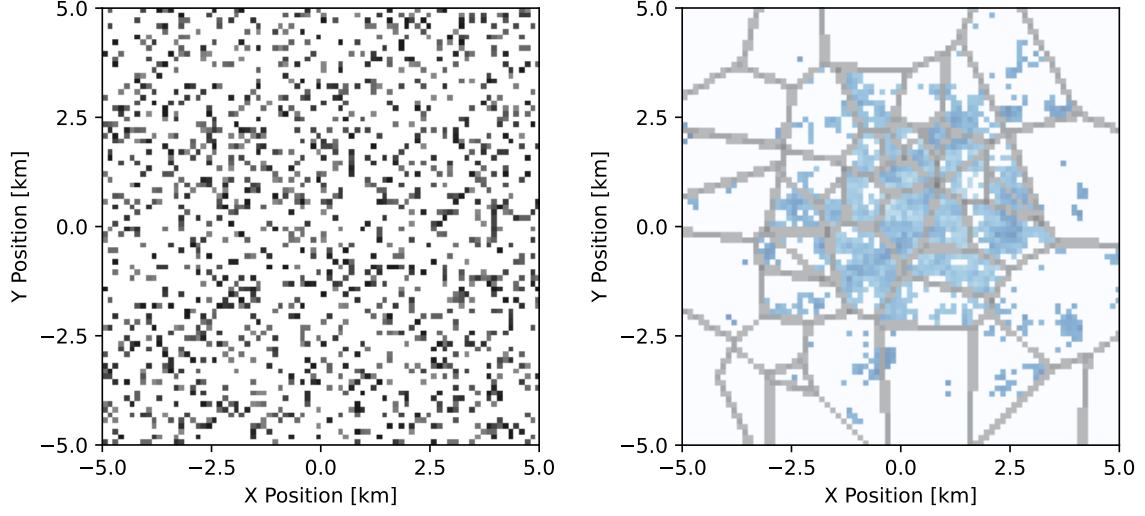


Figure 7.1.: Basic map (left) and Voronoi diagram-based map (right).

7.2. Descriptions of tasks

7.2.1. Task A: Escape

The escape task is designed to encourage the target agent to move as far away as possible from its initial position, effectively escaping from it. This task serves as an evaluation of how the context knowledge is employed by the target in order to accomplish the goal, which involves the navigation of the target on the map.

The reward for the target in this task is defined to encourage the target to increase its Euclidean distance from its initial position. This also takes into account its Euclidean distance from the initial position in the previous step, such that

$$r_{\mathcal{T},t} = \|\mathbf{x}_{\mathcal{T},t} - \mathbf{x}_{\mathcal{T},t=0}\|_2 - \|\mathbf{x}_{\mathcal{T},t-1} - \mathbf{x}_{\mathcal{T},t=0}\|_2, \quad (7.2.1)$$

where $\mathbf{x}_{\mathcal{T},t=0}$ denotes the initial position of the target, at $t = 0$. This reward also penalizes the target for returning to or staying in the same position it was in at the previous time step. Consequently, it encourages the target to keep moving away from the initial position, aligning with the goal of escaping from it.

The escape task can be regarded as a form of evasion behavior, as the target’s objective is to move away from its initial position while taking into account the contextual conditions.

Experiments focus on two cases: one where the target is only provided the context-related

observations $\mathbf{c}_{\mathcal{T},t}$, and the other where the target has access to both the state-related $\mathbf{s}_{\mathcal{T},t}$ and context-related observations $\mathbf{c}_{\mathcal{T},t}$.

In the first case, the objective is to assess the performance and behavior of the target in the escape task when relying solely on context knowledge. Given that only context-related observations are available to the target, and it does not have access to any state-related observation, the results serve as an indicator of how the target utilizes the context knowledge.

In the second case, the target utilizes both state-related and context-related observations. To evaluate the impact of different observation spaces provided to the target on its performance and behavior, comparative analyses are presented. Various scenarios are defined to introduce diversity into the experiments and evaluations.

The formulation of context-related observations is tailored to the configuration of each scenario and is detailed within the scenario descriptions. State-related observations are already defined in (6.2.5).

Scenario A.1: Basic map

This scenario utilizes the basic maps $\bar{\mathbf{M}}_b$ introduced in Section 5.1. In addition to the basic map, the target has also access to its rotation map $\mathbf{M}_{pos,\mathcal{T},t}$ and initial position map $\mathbf{M}_{rot,\mathcal{T},t=0}$. The target does not have access to any state-related observation. It can only utilize context-related observations, which are formulated as

$$\mathbf{c}_{\mathcal{T},t} = \begin{bmatrix} \bar{\mathbf{M}}_b \\ \mathbf{M}_{pos,\mathcal{T},t=0} \\ \mathbf{M}_{rot,\mathcal{T},t} \end{bmatrix}. \quad (7.2.2)$$

Scenario A.2: Voronoi diagram-based map

This scenario utilizes the Voronoi diagram-based maps $\mathbf{M}_v$ introduced in Section 5.2. It is further divided into four scenarios.

A.2.1: The road map $\mathbf{M}_r$ is not involved, thus the speed adaptation (see Section 6.1.4) does not apply. The target does not have access to any state-related observation, it can only utilize context-related observations.

A.2.2: The road map is not involved, thus the speed adaptation does not apply. As opposed to A.2.1, both the context and state-related observations are provided to the target.

The context-related observations for A.2.1 and A.2.2 are formulated as

$$\mathbf{c}_{\mathcal{T},t} = \begin{bmatrix} \mathbf{M}_v \\ \mathbf{M}_{pos,\mathcal{T},t=0} \\ \mathbf{M}_{rot,\mathcal{T},t} \end{bmatrix}. \quad (7.2.3)$$

A.2.3: The road map is also included in the context-related observations and the speed adaptation applies. The target does not have access to any state-related observation, it can only utilize context-related observations.

A.2.4: The road map is also included in the context-related observations and the speed adaptation applies. Both the context and state-related observations are provided to the target.

The context-related observations for A.2.3 and A.2.4 are formulated as

$$\mathbf{c}_{\mathcal{T},t} = \begin{bmatrix} \mathbf{M}_v \\ \mathbf{M}_r \\ \mathbf{M}_{pos,\mathcal{T},t=0} \\ \mathbf{M}_{rot,\mathcal{T},t} \end{bmatrix}. \quad (7.2.4)$$

7.2.2. Task B: Tracking and evasion

Tracking and evasion is the core task of this work. It is the extension to the original work, as described in Section 4.5. Unlike Task A: Escape, which involves only the target agent, this task includes both the observer and target agents.

The aim of this task is to evaluate the tracking and evasion behavior of the observer and target, where the context knowledge, which is formed into the context-related observations $\mathbf{c}_{\mathcal{A},t}$, are provided to the agents. The rewards that are utilized in this task are already defined in Section 4.3.3.

It should be noted that in all scenarios, the observer does not know the exact position of the target but rather estimates it, as described in (4.2.3). Therefore, the observer is provided with the position map of the estimated target position $\widetilde{\mathbf{M}}_{pos,\mathcal{T},t}$. On the other hand, the target has access to the exact position and rotation of the observer due to the setting of this task, as defined in Section 4.3.1.

Scenario B.1: Basic map

This scenario utilizes the basic maps $\overline{\mathbf{M}}_b$, and is divided into two scenarios, based on the observation spaces of the agents.

B.1.1: Agents do not have access to any state-related observation $\mathbf{s}_{\mathcal{A},t}$, they are only provided the context-related observations $\mathbf{c}_{\mathcal{A},t}$.

B.1.2: Agents are provided both the state-related and the context-related observations.

The context-related observations of the agents for B.1.1 and B.1.2 are formulated as

$$\mathbf{c}_{\mathcal{O},t} = \begin{bmatrix} \overline{\mathbf{M}}_b \\ \widetilde{\mathbf{M}}_{pos,\mathcal{T},t} \\ \mathbf{M}_{rot,\mathcal{O},t} \end{bmatrix}, \quad \mathbf{c}_{\mathcal{T},t} = \begin{bmatrix} \overline{\mathbf{M}}_b \\ \mathbf{M}_{pos,\mathcal{T},t=0} \\ \mathbf{M}_{rot,\mathcal{O},t} \\ \mathbf{M}_{rot,\mathcal{T},t} \end{bmatrix}. \quad (7.2.5)$$

Scenario B.2: Voronoi diagram-based map

This scenario utilizes the Voronoi diagram-based maps $\mathbf{M}_v$ and it is divided into two scenarios.

B.2.1: Agents do not have access to any state-related observation, they are only provided the context-related observations.

B.2.2: Agents are provided both the state-related observations and the context-related observations.

The context-related observations of the agents for B.2.1 and B.2.2 are formulated as

$$\mathbf{c}_{\mathcal{O},t} = \begin{bmatrix} \mathbf{M}_v \\ \mathbf{M}_r \\ \widetilde{\mathbf{M}}_{pos,\mathcal{T},t} \\ \mathbf{M}_{rot,\mathcal{O},t} \end{bmatrix}, \quad \mathbf{c}_{\mathcal{T},t} = \begin{bmatrix} \mathbf{M}_v \\ \mathbf{M}_r \\ \mathbf{M}_{pos,\mathcal{T},t=0} \\ \mathbf{M}_{rot,\mathcal{O},t} \\ \mathbf{M}_{rot,\mathcal{T},t} \end{bmatrix}. \quad (7.2.6)$$

7. Results and evaluations

Scenario	Map type	Road map	$\mathbf{s}_{\mathcal{T},t}$	Scenario	Map type	Road map	$\mathbf{s}_{\mathcal{A},t}$
A.1	Basic	-	-	B.1.1	Basic	-	-
A.2.1	Voronoi	-	-	B.1.2	Basic	-	+
A.2.2	Voronoi	-	+	B.2.1	Voronoi	+	-
A.2.3	Voronoi	+	-	B.2.2	Voronoi	+	+
A.2.4	Voronoi	+	+				

Table 7.1.: Escape scenarios (left) and tracking and evasion scenarios (right). Various scenarios are defined based on the type of map being used, the inclusion of the road map, and the availability of state-related observations $\mathbf{s}_{\mathcal{A},t}$ for the agents.

7.3. Results

The experiments are performed on a dedicated computing machine in INDA, with an Intel i9-10900K CPU @ 3.70GHz, 64 GB RAM, and Nvidia Quadro RTX 5000 GPU. The *Ray RLlib* framework is used, which is an open-source RL framework that utilizes fine-grained nested parallelism to achieve high-performance results in a wide variety of RL tasks. RLlib provides PPO algorithm and handles all necessary interactions for policy updates and evaluation [106].

It should be noted that the trainings require significant amount of time. For instance, training the scenario A.2.1 for 40 million time steps requires approximately 26 hours. Due to time constraints in this thesis, not all scenarios were trained for an equal and lengthy duration. This aspect of comprehensive training is deferred to future research endeavors.

7.3.1. Task A: Escape

In each scenario, randomly selected two trajectory plots from the last episodes are presented. These plots illustrate the behavior and navigation of the target within the environment. The green curve represents the target’s trajectory, with the green dot marking its initial position. Red crosses are used to indicate points where the target collides with a building or the border, causing it to stop, as described Section 6.1.3.

In addition to the trajectory plots, collision rate and distance plots are also provided. The collision rate plot illustrates the percentage of collisions over time. The distance plots show the mean distance between the last and initial positions of the target in an episode, measured over time. Colliding with a building causes the target to stop, and this constitutes a negative effect on the target in the context of the escape task.

Stopping prevents the target from increasing its reward because the target’s reward only increases when it moves away from its initial position, as described in (7.2.1). Hence, the

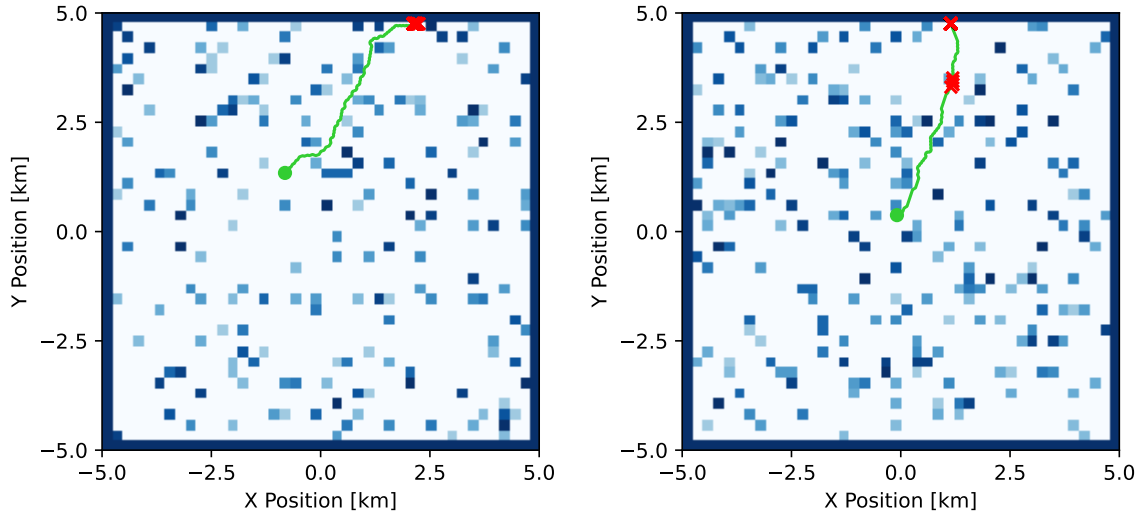


Figure 7.2.: Trajectory plots of A.1: Basic map with context-related observations. The presented trajectories are randomly selected from the last episodes. In each plot, the green curve represents the target’s trajectory, with the green dot marking its initial position. Red crosses are used to indicate the points where the target collides with a building or the border.

collision rate plots for each scenario are assessed with this consideration in mind. Furthermore, since the reward is closely tied to the distance covered by the agent, the distance to initial point metric plays a significant role in evaluating the target’s performance.

Training durations may differ for each scenario. The trajectory plots, as well as collision rate and distance plots for individual scenarios, involve the complete training duration, and they are intended to display the overall behavior. In order to assure uniform and fair comparisons, the comparison plots present the values obtained in the same training duration. Therefore, the actual numerical value comparisons are conducted where the same training durations for the scenarios are considered.

A.1: Basic map with context-related observations

The collision rate and distance plots presented in Figure 7.3 reveal a decrease in collision rates and an increase in the distance the target travels from its initial position over time. These plots show that, as the target is trained longer, fewer collisions happen and the target manages to go further from its initial position. As only the context-related observations are provided to the target, it can be claimed that the target is able to utilize these observations in order to maximize its reward.

In addition, analyzing the trajectory plots given in Figure 7.2, it can be seen that the target manages to evade the buildings, thus the collision rate decreases over time. This

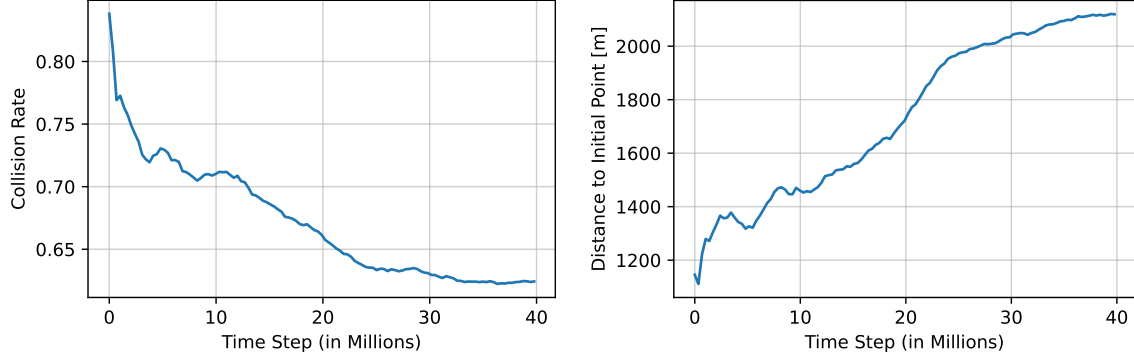


Figure 7.3.: Collision rate (left) and distance (right) plots of A.1. The collision rate plot displays the percentage of the collisions, evaluated over time. The distance to initial point is also plotted over time and they show the mean distance between the last and initial positions of the target in an episode.

observation implies that the target exhibits the ability to navigate around buildings rather than colliding with them, as it executes its escape behavior.

A.2.1: Voronoi diagram-based map with context-related observations

Comparing this scenario with A.1, the only difference is the type of building map. Unlike A.1 which uses the basic map, A.2.1 utilizes the Voronoi diagram-based map. Same as A.1, only the context-related observations are provided to the target.

In the collision rate and distance plots given in Figure 7.5, it can be observed that the collision rate decreases and the distance to the initial point increases over time. Hence, the distance plot illustrates that the target is trained to increase its distance from the initial position. Therefore, as similarly presented in the evaluations of A.1, the target is able to perform the escape task and maximize its reward based on context-related observations.

The building evasion behavior is also visible in the trajectory plot on the left, given in Figure 7.4. Nevertheless, the target is not expected to evade all the buildings that it is confronting. For instance, in the trajectory plot on the right, the target can not evade the building and eventually sticks in front of it, and therefore, can not go away from its initial position any longer.

A.2.2: Voronoi diagram-based map with context-related and state-related observations

In this scenario, the configuration closely resembles that of A.2.1, with the key difference being the inclusion of state-related observations alongside the context-related ones. In the collision rate and distance plots given in Figure 7.7, it can be seen that the collision rate decreases and distance increases over time. Therefore, the distance plot shows that as the

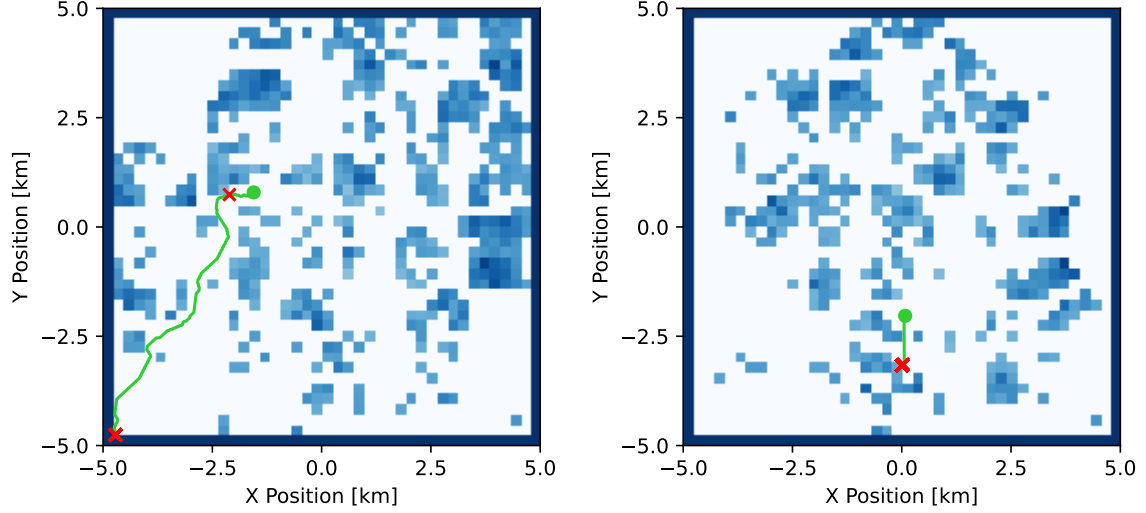


Figure 7.4.: Trajectory plots of A.2.1: Voronoi diagram-based map with context-related observations. The presented trajectories are randomly selected from the last episodes. In each plot, the green curve represents the target's trajectory, with the green dot marking its initial position. Red crosses are used to indicate the points where the target collides with a building or the border.

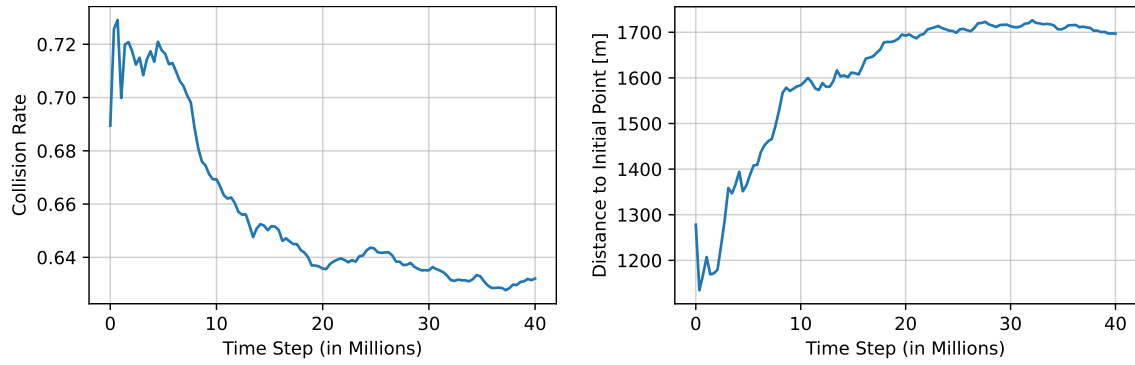


Figure 7.5.: Collision rate (left) and distance (right) plots of A.2.1. The collision rate plot displays the percentage of the collisions, evaluated over time. The distance to initial point is also plotted over time and they show the mean distance between the last and initial positions of the target in an episode.

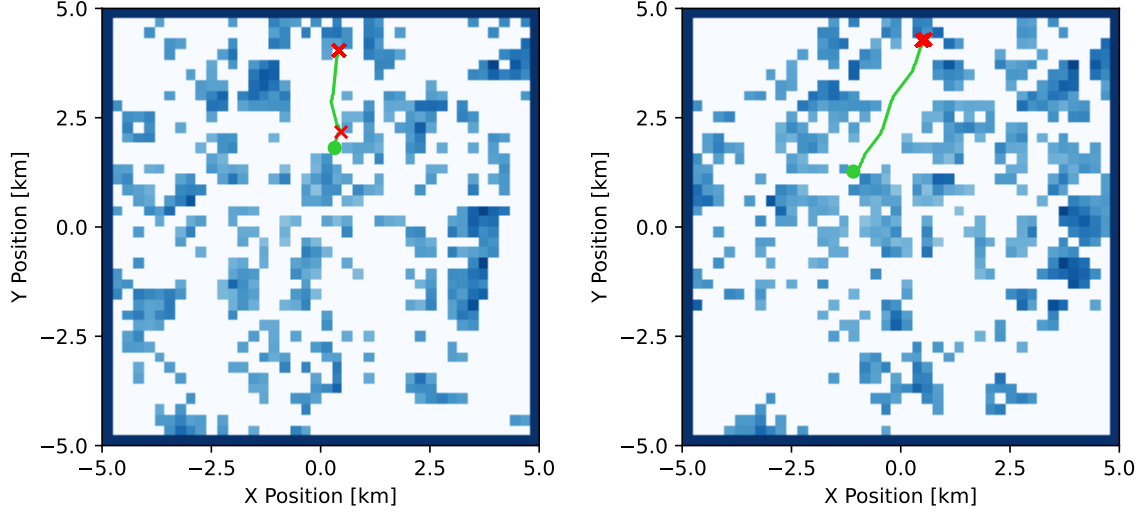


Figure 7.6.: Trajectory plots of A.2.2: Voronoi diagram-based map with context-related and state-related observations. The presented trajectories are randomly selected from the last episodes. In each plot, the green curve represents the target's trajectory, with the green dot marking its initial position. Red crosses are used to indicate the points where the target collides with a building or the border.

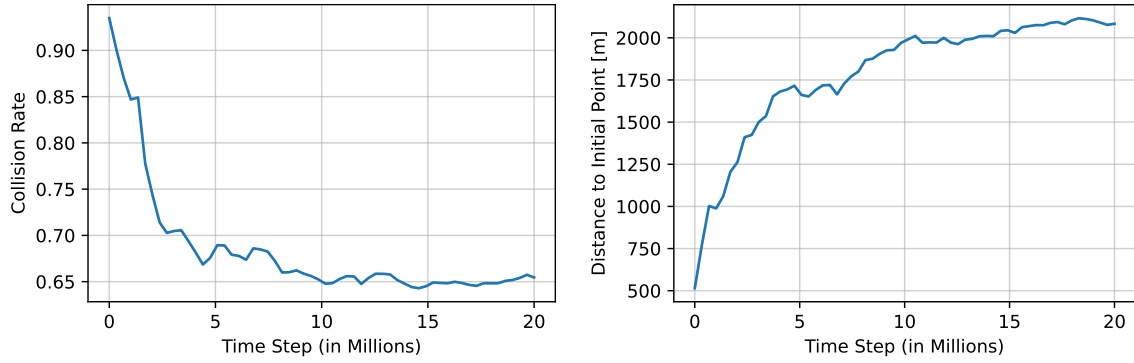


Figure 7.7.: Collision rate (left) and distance (right) plots of A.2.2. The collision rate plot displays the percentage of the collisions, evaluated over time. The distance to initial point is also plotted over time and they show the mean distance between the last and initial positions of the target in an episode.

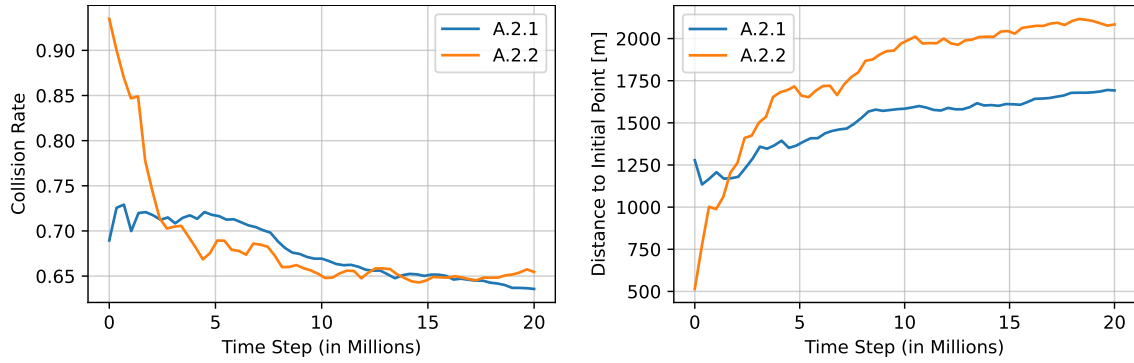


Figure 7.8.: Collision rate (left) and distance (right) plots of A.2.1 and A.2.2. The collision rate plot displays the percentage of the collisions, evaluated over time. The distance to initial point is also plotted over time and they show the mean distance between the last and initial positions of the target in an episode. In A.2.1, only the context-related observations are provided to the target, whereas in A.2.2, the target has access to both types of observations.

target is trained, it is able to go further from its initial position, similar to the results of A.2.1.

The collision rate and distance plots of A.2.1 and A.2.2 are given together in Figure 7.8. As in A.2.2 the target is trained for 20 million time steps and in A.2.2 this duration is 40 million time steps, for the sake of comparison, the result for the first 20 million steps of A.2.2 is plotted. The distance value for A.2.1 converges to approximately 1750 meters for its complete training duration, which is already shown in Figure 7.5.

According to the plots, it can be seen that the results for both scenarios are relatively close to each other. It is worth mentioning that in A.2.2, the distance to the initial point exceeds the same metric of A.2.1. As the only difference between these two scenarios is the integration of the state-related observations, this increase in distance metric can be explained over the utilization of both types of observations together. One of the possible reasons can be that, the information incorporated by the state-related observations, used together with the context-related observations, may be helpful for the target in performing the escape task, thus the distance metric has a higher score than the same metric of A.2.1.

It can be inferred that the inclusion of state-related observations provides valuable information to the target, which enhances its performance in the escape task. This may explain the higher distance metric in this scenario compared to the same metric in A.2.1.

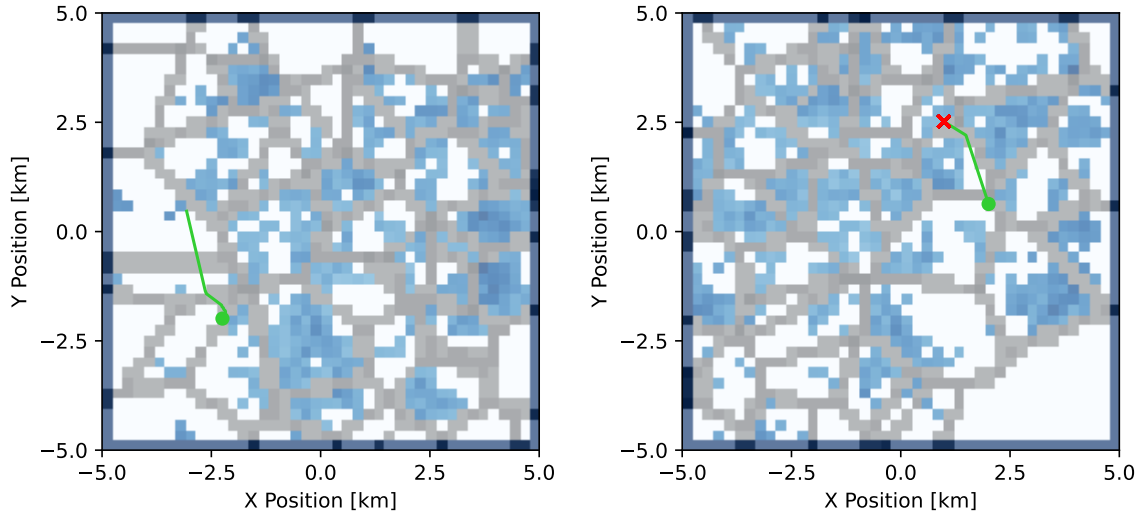


Figure 7.9.: Trajectory plots of A.2.3: Voronoi diagram-based map and the road map, with context-related observations. The presented trajectories are randomly selected from the last episodes. In each plot, the green curve represents the target’s trajectory, with the green dot marking its initial position. Red crosses are used to indicate the points where the target collides with a building or the border.

A.2.3: Voronoi diagram-based map and road map with context-related observations

Compared to the context-related observations utilized in A.2.1 and A.2.2, this scenario also contains the road map. As the road map is involved, the speed adaptation, as described in Section 6.1.4 also applies. According to the speed adaptation, if the target goes off the road, its speed decreases, therefore the distance it can take also reduces. Same as A.2.1, only the context-related observations are provided to the target.

The collision plot given in Figure 7.10 shows that the collision rate decreases at the first several million time steps, then starts to increase until approximately 10 million steps. Taking a look at the behavior of the collision rate plot, it can be observed that the collision rate stays around 0.65 and does not show any significant increase.

Moreover, in the distance plot, the distance to the initial point increases up to 1100 meters and then stays around 1050 meters. Similar to the collision rate behavior, the distance metric also does not show any significant increase. Comparing these results with the collision rate and distance plots of A.2.1 given in Figure 7.5, which has the same setting except the inclusion of the road map, the target in A.2.3 can not perform escape task as performant as it does in A.2.1. The notable increase in the collision rate and relatively low distance to the initial point values in this scenario may suggest that the target struggles

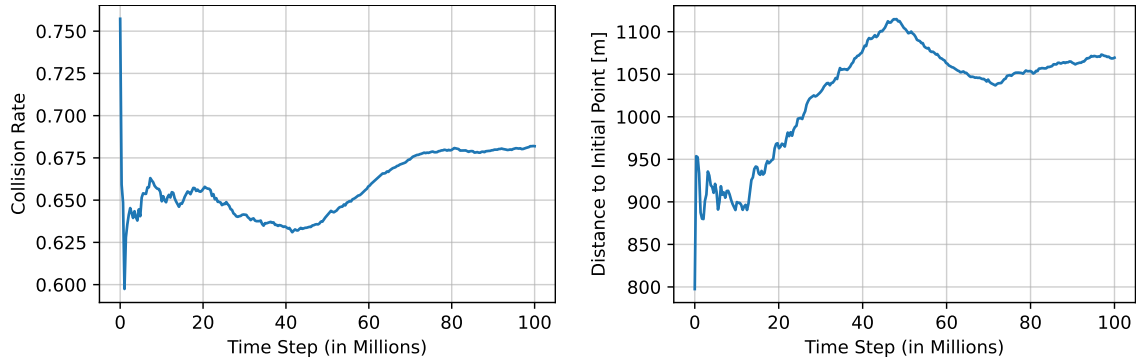


Figure 7.10.: Collision rate (left) and distance (right) plots of A.2.3. The collision rate plot displays the percentage of the collisions, evaluated over time. The distance to initial point is also plotted over time and they show the mean distance between the last and initial positions of the target in an episode.

to effectively utilize the provided observations, which consist solely of context-related observations.

Given that the difference between scenarios A.2.1 and A.2.3 is the inclusion of the road map in the context-related observations, which activates the speed adaptation, these results are assessed with these distinctions in mind.

With the inclusion of the road map, the observation space gets more complex. As the target uses these observations to perform its task, it may be difficult for the target to understand and utilize this complex observation space. Furthermore, the target's inability to effectively utilize the road map observations results in a lack of awareness regarding the road network on the map. When the speed adaptation is in effect, going off the road leads to a decrease in speed and, consequently, a reduction in the distance the target can travel. If the target uses off-road terrain instead of the roads on the map, its achievable distance is reduced. Therefore, the relatively low value in the distance plot and the increase in the collision plot may be attributed to the complexity of the observation space, which the target may not be using effectively to perform the escape task.

A.2.4: Voronoi diagram-based map and road map with context-related and state-related observations

Compared to A.2.3, both the context-related and state-related observations are provided to the target in this scenario. Analyzing the collision rate and distance plots given in Figure 7.12, it can be seen that the collision rate decreases, and the distance to the initial point increases over time, as expected. This indicates that the target is trained to be able to increase its distance from the initial position, over time.

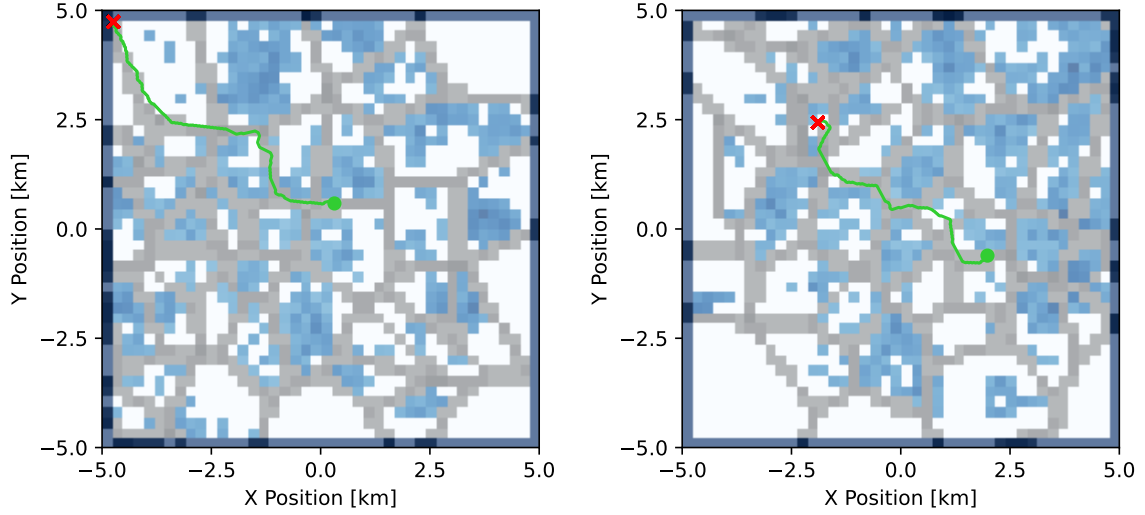


Figure 7.11.: A.2.4: Voronoi diagram-based map and road map with context-related and state-related observations. The presented trajectories are randomly selected from the last episodes. In each plot, the green curve represents the target’s trajectory, with the green dot marking its initial position. Red crosses are used to indicate the points where the target collides with a building or the border.

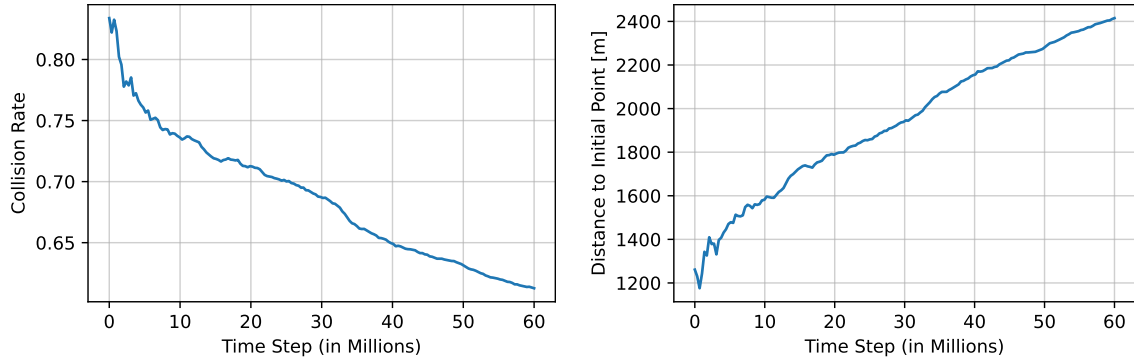


Figure 7.12.: Collision rate (left) and distance (right) plots of A.2.4. The collision rate plot displays the percentage of the collisions, evaluated over time. The distance to initial point is also plotted over time and they show the mean distance between the last and initial positions of the target in an episode.

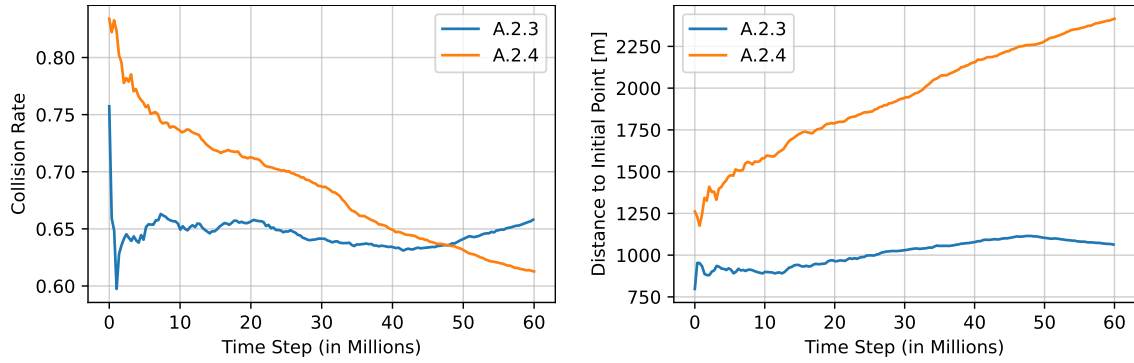


Figure 7.13.: Collision rate (left) and distance (right) plots of A.2.3 and A.2.4. The collision rate plot displays the percentage of the collisions, evaluated over time. The distance to initial point is also plotted over time and they show the mean distance between the last and initial positions of the target in an episode. Both scenarios are based on the Voronoi diagram-based map. The road map is also involved. In A.2.3, only the context-related observations are provided to the target, whereas in A.2.4, the target has access to both types of observations.

Additionally, the trajectory plots provided in Figure 7.11 also offer insights into the target’s behavior. These plots reveal that the target is able to evade buildings.

Moreover, by analyzing these trajectory plots, it may be inferred that the target utilizes the roads to escape from its initial position. This may show that the target is able to utilize the road map and thus aware of the road network. Therefore it may prefer to stay on the road in order to maintain its speed as much as possible in order to prevent speed decrease.

These results are then compared to A.2.4. As A.2.4 is trained for 60 million time steps and A.2.3 for 100 million time steps, for the sake of comparison, the results for the first 60 million time steps of A.2.3 are used. The collision rate and distance plots of A.2.3 and A.2.4 are given together in Figure 7.13.

In the distance comparison plot, the values of the distance to the initial point curve for scenario A.2.4 are consistently higher than those for A.2.3 across all time steps. In addition, these distance values in A.2.4 increase over time, whereas the distance plot of A.2.3 does not show a prominent increase. These results are then evaluated considering that in A.2.4, in addition to the context-related observations, the state-related observations are also provided to the target. Hence, the utilization of both types of observations together results in an improved performance of the target when compared to using only the context-related observations.

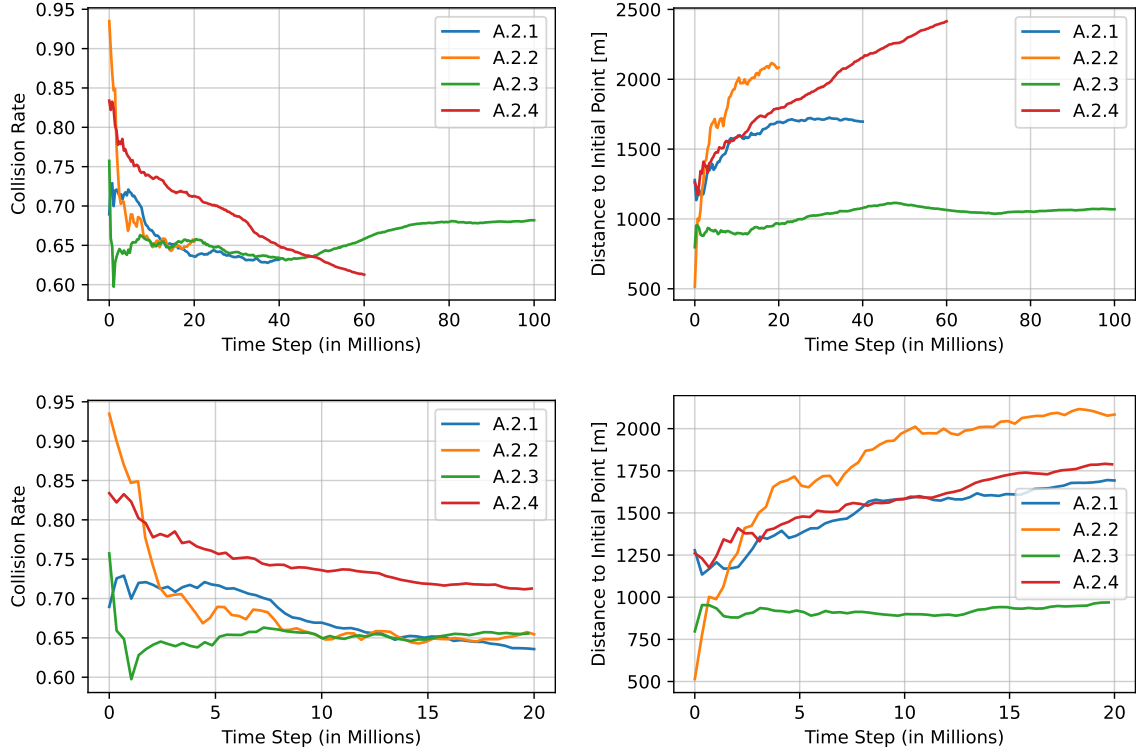


Figure 7.14.: Collision rate (left) and distance (right) plots of A.2.1, A.2.2, A.2.3 and A.2.4. The collision rate plot displays the percentage of the collisions, evaluated over time. The distance to initial point is also plotted over time and they show the mean distance between the last and initial positions of the target in an episode. The top two plots contain the complete data for the collision rate and distance results of the four scenarios, which are trained for different time steps. In the bottom two plots, for the sake of comparison, results for the first 20 million time steps of each scenario are plotted. All these scenarios are based on the Voronoi diagram-based map. In A.2.1, only the context-related observations are provided to the target, whereas in A.2.2, the target has access to both types of observations. In A.2.3 and A.2.4, the road map, thus the speed adaptation, is also considered. In A.2.3, only the context-related observations are provided to the target, whereas in A.2.4, the target has access to both types of observations.

	Collision rate	Distance (in meters)
A.2.1	0.635	1692.412
A.2.2	0.654	2083.579
A.2.3	0.655	968.976
A.2.4	0.712	1788.091

Table 7.2.: Collision rate and distance results obtained after 20 million time steps of training for A.2.1, A.2.2, A.2.3 and A.2.4.

The collision rate and distance comparisons between all the scenarios of A.2 are presented in Figure 7.14. In the first two plots, the collision rate and distance plots for the complete training time, which are already plotted under the relevant scenario results, are plotted together. In the last two plots, for the sake of comparison, the results for the first 20 million steps of all the scenarios are plotted together.

The top plots in Figure 7.14 give insight into how the plots behave when they are trained for longer. For instance, all the curves except the one for A.2.3 exhibit a decreasing trend in the collision rate plot, and an increasing trend in the distance plot. In addition, the curves for A.2.4 and A.2.2 increase to higher distance to initial point values, which can be visible in these top two plots. In addition, it can be observed that they exhibit a trend of increase as they are trained longer.

For the sake of comparison, the collision rate and distance results obtained after the first 20 million time steps of training for each scenario are given in Table 7.2. It should be noted that, as A.2.3 and A.2.4 consider speed adaptation, which implies slowing down the target when it goes off the road, the distance results for them should be separately evaluated. In Table 7.2, it can be observed that the distance results for scenarios A.2.1 and A.2.2 are relatively close to each other. For this reason, the performance of the target in A.2.1, utilizing only the context-related observations, is close to its performance in A.2.2. One possible explanation for the higher collision rates observed in A.2.2 and A.2.4, despite the increased distance from the initial point hence improved escape behavior, may be related to collisions occurring at the map’s borders. Since the target moves towards the borders while escaping from its initial point, it may have a higher likelihood of colliding with the border as it gets closer to it. This phenomenon might explain the observed increase in the collision rate.

Another comparison can be made between the distance results of A.2.3 and A.2.4. The increase in distance result in A.2.4 may be explained through the utilization of both the state-related and context-related observations in A.2.4. Therefore, it can be stated that the state-related observations provide helpful information to the target, which in turn increases the performance of the target in the task.

To sum up, in A.1, the target is able to evade buildings, decrease collisions, and increase its distance to the initial point by utilizing context-related observations. Similar comments can be made by analyzing the results of A.2.1. The distance and collision rate results of A.2.1 and A.2.2 are relatively close to each other, therefore utilizing only the context-related observations, the target can perform similarly to how it performs in A.2.2. When analyzing the results of A.2.3, the collision rate and distance plots start to differ from the expectations for a performant escape behavior of the target. Therefore, the integration of the road map may be the reason for this decrease in performance, as it makes the observation space more complex. Finally, it is inferred that using both the state-related and context-related observations in A.2.4 increases the distance result of A.2.3, which shows that using state-related observations provides helpful knowledge to the target in performing its task.

7.3.2. Task B: Tracking and evasion

In each scenario, randomly selected two trajectory plots from the last episodes are presented. These plots illustrate the behavior and navigation of the observer and the target within the environment, with the task of tracking and evasion. In these plots, the green curve represents the trajectory of the target and the green dot represents its initial position. Meanwhile, the orange curve represents the trajectory of the observer and the orange dot represents its initial position.

All the scenarios employing the Voronoi diagram-based map, which are B.2.1 and B.2.2, already include the road map and speed adaptation. Therefore, unlike task A, the road map is not an element of comparison in this task. Road map is not directly related to the tracking performance of the observer, but rather it affects the target. The comparisons focus on the utilized observation spaces.

Mean reward plots of the observer are presented together with the smoothed versions, which provide a smoother view of how the mean reward of the observer changes over time. The observer reward, as defined in (4.3.5), is directly related to the tracking performance of the observer and therefore, the observer reward plots are used to assess the tracking performance of the observer.

In addition to the reward plots, occlusion rate plots are also provided. In these plots, the occlusion rate, meaning the percentage of the occlusions, is plotted over time. As the occlusion prevents the observer from tracking the target, these plots show how the tracking performance of the observer changes over time. In the comparative analysis, the observer mean rewards plots serve as valuable metrics. Moreover, the insights and rationales drawn from Task A in Section 7.3.1 are leveraged to evaluate the outcomes and draw conclusions in this task.

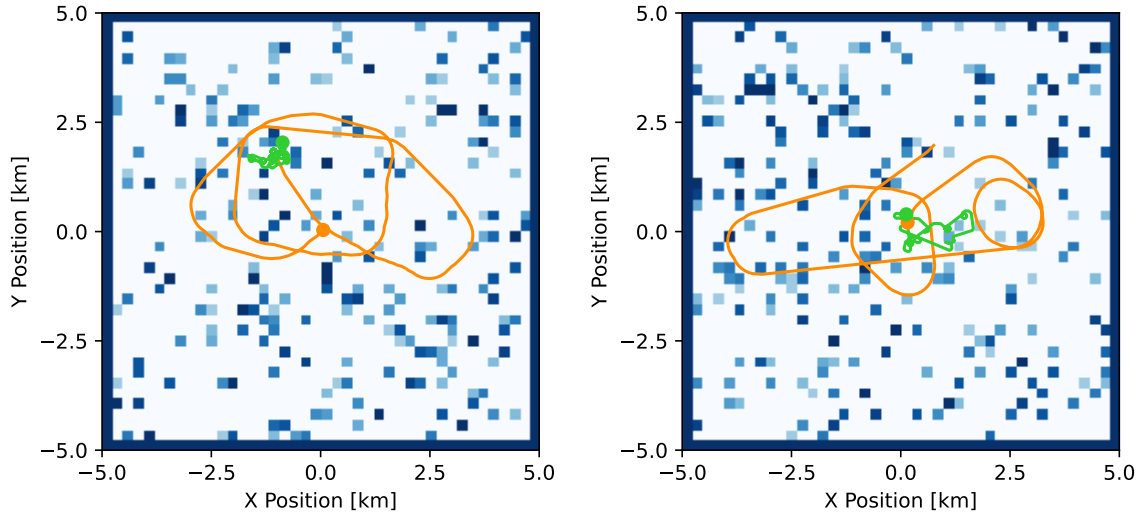


Figure 7.15.: Trajectory plots of B.1.1: Basic map with context-related observations. The presented trajectories are randomly selected from the last episodes. In each plot, the green curve represents the trajectory of the target and the green dot represents its initial position. Meanwhile, the orange curve represents the trajectory of the observer and the orange dot represents its initial position.

Similarly explained in Task A, the training durations for this task may also differ for each scenario. The trajectory plots, as well as observer mean rewards, and occlusion rate plots for individual scenarios, involve the complete training duration. They are intended to display the overall behavior of the metrics. In order to assure uniform and fair comparisons, the comparison plots present the values obtained within the same training duration. Therefore, the actual numerical value comparisons are conducted where the same training durations for the scenarios are considered.

B.1.1: Basic map with context-related observations

In the reward and occlusion rate plots given in Figure 7.16, it can be seen that the observer reward seems to decrease within the first approximately 10 million time steps, then it increases over time. One of the possible reasons for the decrease is, in this first approximately 10 million time steps, the observer struggles to understand the map only using context-related observations and to find the target on the map. As the observer is not given the exact position of the target but rather uses EKF to predict its position, in the first approximately 10 million time steps, the observer may not be able to properly estimate the position of the target, as it has to locate the target in order to track it. Again in the reward plot, it can be observed that the observer reward increases and converges to around 95. Meanwhile, the occlusion rate exhibits a decreasing trend.

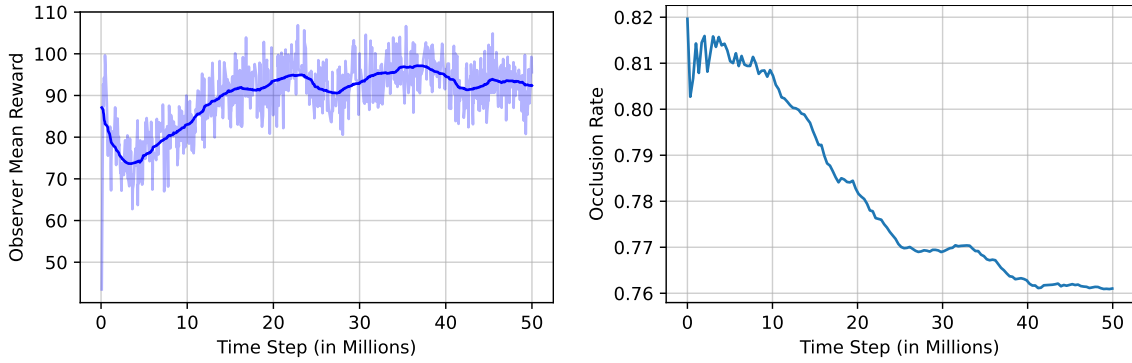


Figure 7.16.: Mean observer reward (left) and occlusion rate (right) plots of B.1.1. The observer mean reward is plotted together with its smoothed version, with different opacities. The occlusion rate plot displays the percentage of the occlusions, evaluated over time.

B.1.2: Basic map with context-related and state-related observations

Compared to B.1.1 which is based on only the context-related observations, this scenario involves utilizing the state-related and context-related observations together. By looking at the reward and occlusion rate plots given in Figure 7.18, it can be seen that the reward increases and the occlusion rate decreases over time, therefore the observer policy improves in performing the tracking task.

The behavior of the observer can be compared in the trajectory plots of B.1.1 and B.1.2. The observer in the trajectories of B.1.1 seems to travel across the map and explores the map in order to localize and keep the target in sight, in order to track it. This may be caused by the observer may struggle to use the context-related observations in order to understand the provided maps and thus, track the target. Meanwhile, the observer in the trajectories of B.1.2 is in close contact with the target and it seems that it is able to keep the target in its sight.

The observer mean reward plots of B.1.1 and B.1.2 are given together in Figure 7.19. It can be seen that both the plot displays increase at first and then the values stabilize, over time. In addition, the observer mean rewards of B.1.2 are higher than those of B.1.2.

The only difference between these two scenarios is the observation spaces provided to the agents. One of the possible reasons for the behavior difference seen in the trajectories and the increase in observer mean reward value in B.1.2 may be related to the utilization of the state-related observations in B.1.2, in addition to the context-related observations. As also discussed in A.2.2 and A.2.4, using additional state-related observations increases the performance and leads to policy improvement.

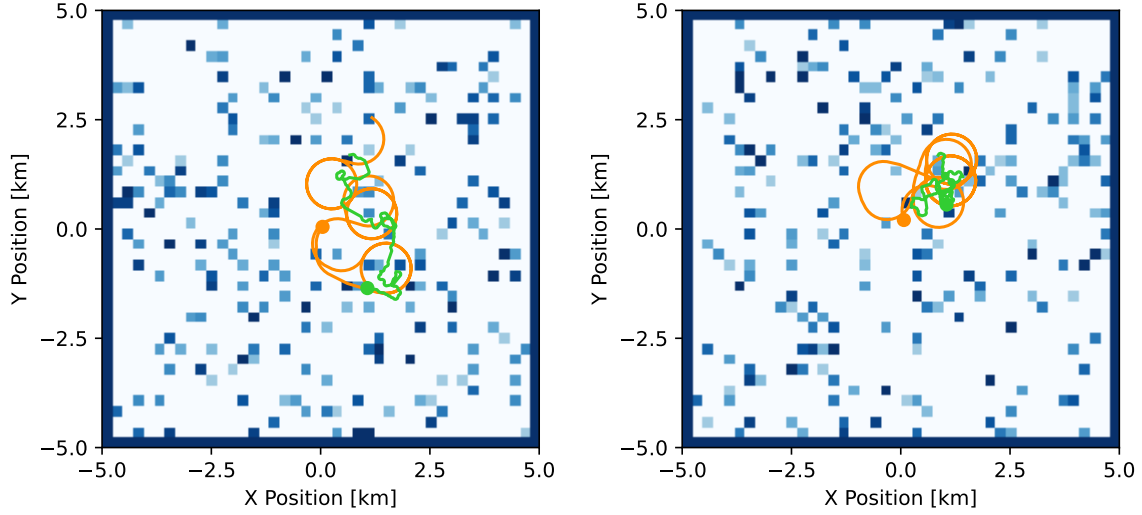


Figure 7.17.: Trajectory plots of B.1.2: Basic map with context-related and state-related observations. The presented trajectories are randomly selected from the last episodes. In each plot, the green curve represents the trajectory of the target and the green dot represents its initial position. Meanwhile, the orange curve represents the trajectory of the observer and the orange dot represents its initial position.

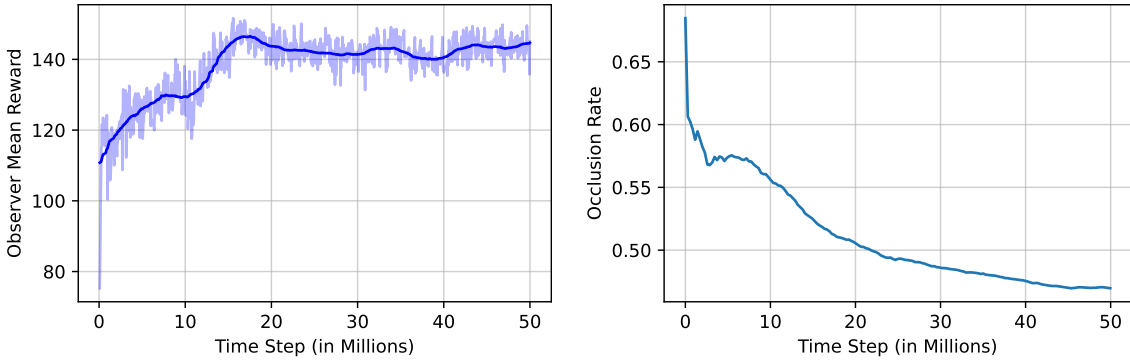


Figure 7.18.: Mean observer reward (left) and occlusion rate (right) plots of B.1.2. The observer mean reward is plotted together with its smoothed version, with different opacities. The occlusion rate plot displays the percentage of the occlusions, evaluated over time.

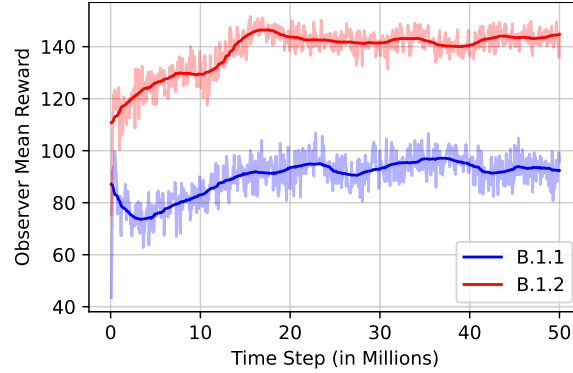


Figure 7.19.: Mean observer reward plots of B.1.1 and B.1.2. The observer mean rewards are plotted together with their smoothed versions, with different opacities.

Another reasoning can be applied to the results of B.1.1. From the results of A.1 which is also based on the basic map, it is discussed that the target is aware of the buildings and can navigate in the environment, based on the context-related observations. Using these results from A.1, one of the reasons why the observer performs worse in B.1.1 than in B.1.2 may be related to the performance of the target. As the tracking and evasion tasks involve competing agents, the individual performances of agents are related to the other agent's performance, as well. Considering these, in B.1.1, the target may be outperforming the observer by evading well, so that the observer can not improve its tracking policy and thus the observer's mean reward does not increase to a relatively higher value. Coming to B.1.2, now that both the agents have access to the state-related observations, a reason for the increasing reward behavior of the observer is that the state-related observations provide helpful information to the observer, especially about the target. For this reason, the observer policy improves over time and the observer is able to track the target better, compared to B.1.2.

B.2.1: Voronoi diagram-based map with context-related observations

In the reward and occlusion rate plots given in Figure 7.21, the increase in the observer reward and decrease in the collision rate can be observed. In A.2.3, the same setting is used which involves the Voronoi diagram-based map, road map and using only the context-related observations. The comments on the results of A.2.3 presented that, in the mentioned scenario setting, the policy of the target does not improve as well as it does in A.2.1, which is based on the basic map and utilizes only context-related observations. Considering this, one possible inference can be made that, the observer may take advantage of this inadequate performance of the target in this mentioned scenario setting, therefore the observer mean reward increases and converges to a relatively high value. Another possible reason is that the observer may understand the maps provided with context-related observations and may make use of them in order to increase its tracking performance.

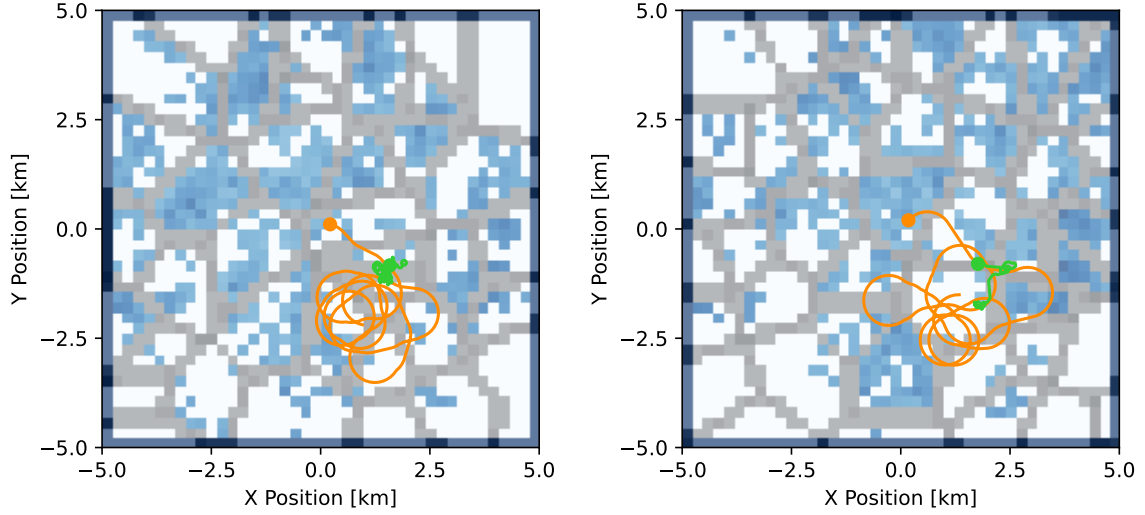


Figure 7.20.: Trajectory plots of B.2.1: Voronoi diagram-based map with context-related observations. The presented trajectories are randomly selected from the last episodes. In each plot, the green curve represents the trajectory of the target and the green dot represents its initial position. Meanwhile, the orange curve represents the trajectory of the observer and the orange dot represents its initial position.

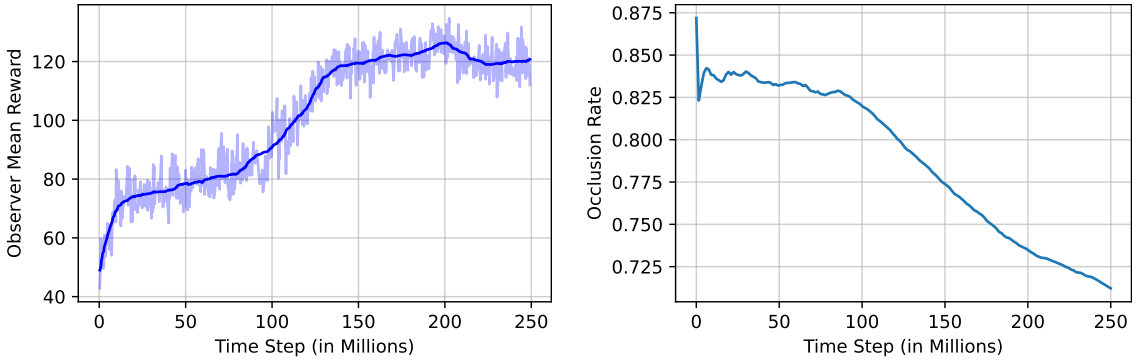


Figure 7.21.: Mean observer reward (left) and occlusion rate (right) plots of B.2.1. The observer mean reward is plotted together with its smoothed version, with different opacities. The occlusion rate plot displays the percentage of the occlusions, evaluated over time.

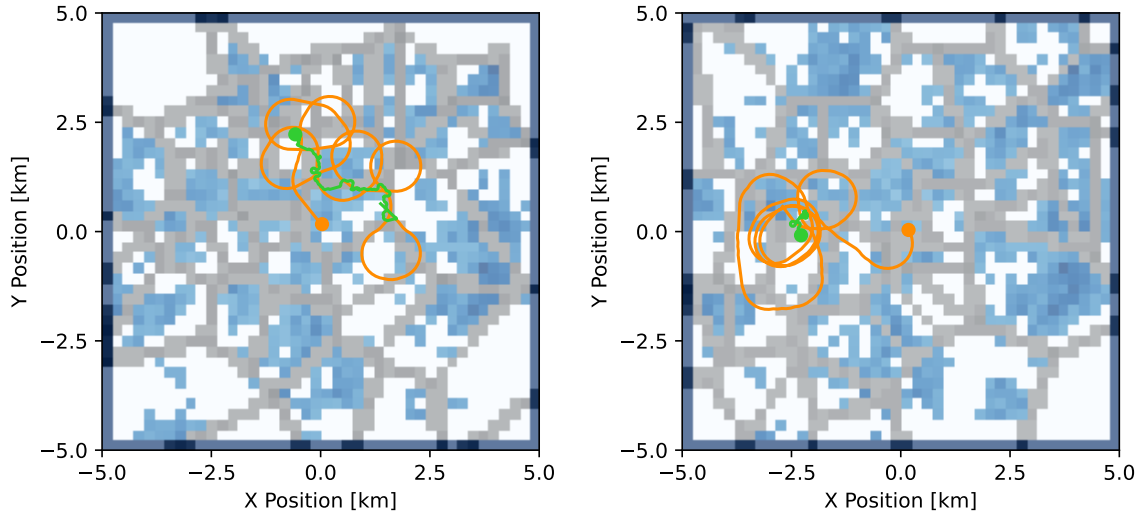


Figure 7.22.: Trajectory plots of B.2.2: Voronoi diagram-based map with context-related and state-related observations. The presented trajectories are randomly selected from the last episodes. In each plot, the green curve represents the trajectory of the target and the green dot represents its initial position. Meanwhile, the orange curve represents the trajectory of the observer and the orange dot represents its initial position.

B.2.2: Voronoi diagram-based map with context-related and state-related observations

This scenario involves utilizing the state-related and context-related observations together, as different from B.2.1. Analyzing the reward and occlusion rate plots given in Figure 7.23, the increase in the observer reward and decrease in the collision rate can be observed, which implies the improvement in the observer policy in the tracking task.

Comparing the observer mean reward plots of B.2.1 and B.2.2 presented in Figure 7.24, the similar behavior in Figure 7.19 is visible. As the only difference between B.2.1 and B.2.2 is the observation spaces, it can be stated that using the state-related observations together with the context-related observations leads to improvement in the observer policy.

In Figure 7.25, the collision rate and distance plots of the target for all the scenarios in this task are presented, in order to evaluate the tracking and evasion from the perspective of the target. Although the collision and distance to initial point metrics do not have a direct relation with the evasion behavior, unlike the escape task, they are presented for the sake of giving an insight into the performance of the target in this task, besides the observer. Future work may involve presenting more related metrics in order to better and more accurately assess the evasion performance of the target.

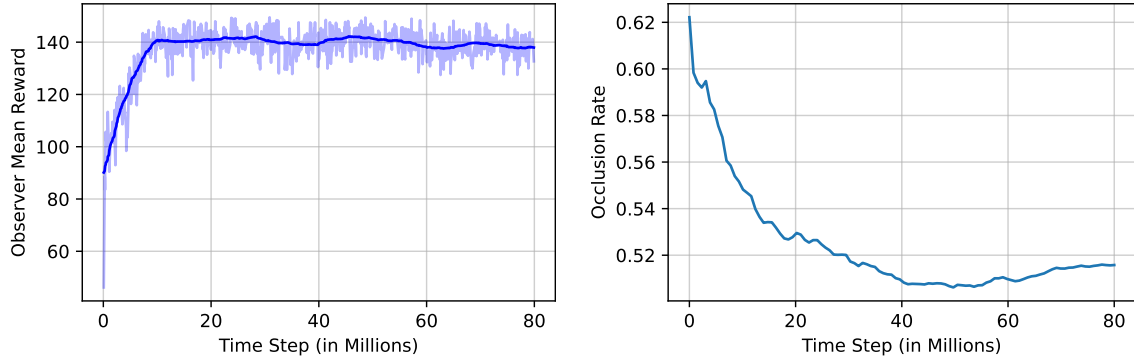


Figure 7.23.: Mean observer reward (left) and occlusion rate (right) plots of B.2.2. The observer mean reward is plotted together with its smoothed version, with different opacities. The occlusion rate plot displays the percentage of the occlusions, evaluated over time.

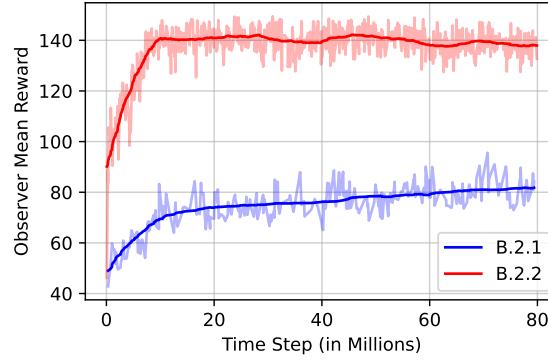


Figure 7.24.: Mean observer reward plots of B.2.1 and B.2.2. The observer mean rewards are plotted together with their smoothed versions, with different opacities.

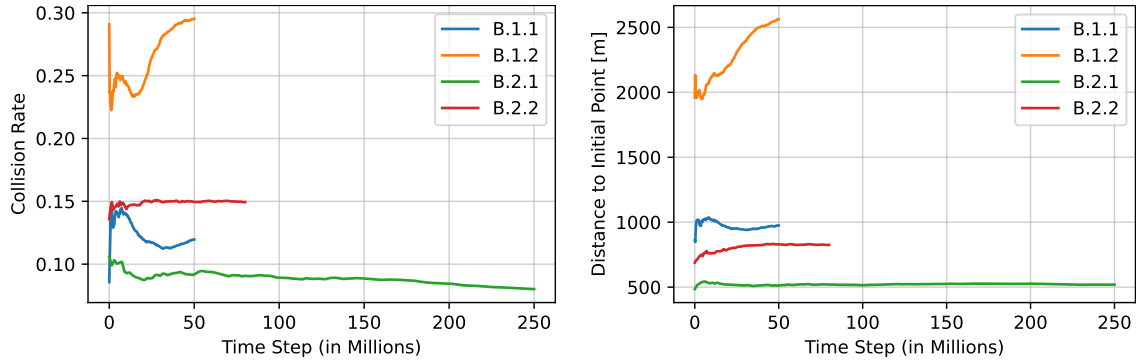


Figure 7.25.: Collision rate (left) and distance (right) plots of the target in B.1.1, B.1.2, B.2.1 and B.2.2. The collision rate plot displays the percentage of the collisions, evaluated over time. The distance to initial point is also plotted over time and they show the mean distance between the last and initial positions of the target in an episode.

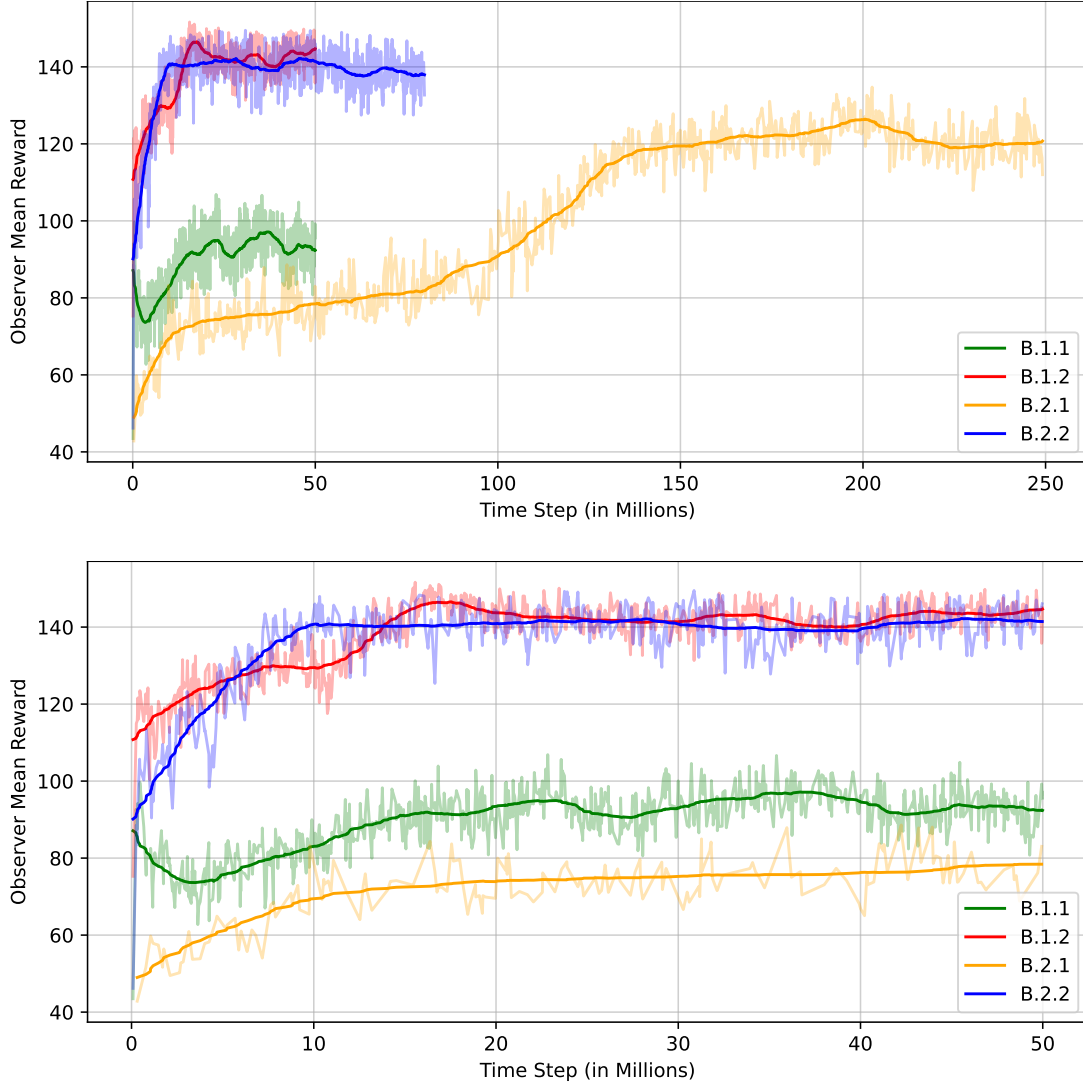


Figure 7.26.: Mean observer reward plots of B.1.1, B.1.2, B.2.1 and B.2.2. The observer mean rewards are plotted together with their smoothed versions, with different opacities. The top two plots contain the complete data for the observer mean reward results of these four scenarios, which are trained for different time steps. In the bottom two plots, for the sake of comparison, results for the first 50 million time steps of each scenario are plotted. In B.1.1 and B.2.1 only the context-related observations are provided to the agents, whereas in B.1.2 and B.2.2, agents have access to both types of observations. Moreover, B.1.1 and B.1.2 utilize the basic map, whereas B.2.1 and B.2.2 are based on the Voronoi diagram-based map, also including the road map.

	Observer mean reward
B.1.1	95.583
B.1.2	145.015
B.2.1	83.037
B.2.2	146.053

Table 7.3.: Observer mean rewards obtained at the end of first 50 million time steps of training of B.1.1, B.1.2, B.2.1 and B.2.2.

To compare and evaluate all the scenarios in this task, both the reward plots showing the complete training results for each scenario (top two plots in Figure 7.26), as well as the plots (bottom two plots in Figure 7.26) and values (Table 7.3) which are obtained considering the same amount of training time for each scenario, are used.

Upon analyzing the reward values presented in Table 7.3, it can be observed that the scenarios utilizing both context-related and state-related observations yield higher observer rewards when compared to scenarios relying solely on context-related observations. It can be therefore inferred that in addition to the context knowledge, which is structured into context-related observations, the state-related observations provide also helpful information to the observer in performing the tracking. Therefore, in order to obtain an improved tracking performance, both state-related and context-related observations should be provided to the agents.

The highest observer mean reward is obtained in B.2.2. By considering the performance of both the observer and the target, one possible reason can be hypothesized by deducing from the evaluations for the results of A.2.2 of A.2.4. As shown in Table 7.2, target performs better in A.2.2 compared to A.2.4. One possible reason for this was explained as the increased complexity of the observation space with the inclusion of the road map and the activation of the speed adaptation. As the performances of each agent are related to the others, the observer may be taking advantage of this in order to improve its policy and thus track performance. Nevertheless, as the results for B.1.2 and B.2.2 are relatively close, it is difficult to come up with exact reasons for these results.

The final remark involves the training durations and how the metrics change for longer trainings. The overall behavior of the observer reward values for the complete training in each scenario, given in the top two plots in Figure 7.26, show that each of the observer mean reward plots increases and eventually converges to a value. Therefore, it can be concluded that through the training, the observer improves its policy in order to track and keep the target within its field of view, to maximize its reward. Moreover, as it can be seen for B.2.1, the observer reward exhibits an increasing trend as it is trained longer. Therefore, longer training durations may increase the overall tracking performance of all the scenarios. It may also be hypothesized that if trained longer, the agents may utilize the observations better and thus increase their tracking and evasion performance. Extensive

7. Results and evaluations

and sufficiently long training for each scenario can be used to prove or disprove these hypotheses, and these experimentations may be part of a future work.

8. Conclusion and future work

The original work [1] of Brandenburger et al., which is presented in Chapter 4, focuses on a MARL problem, where the observer and target agents engage in tracking and evasion behaviors. The main goal of this work is the optimization of tracking performance and the trajectory of the observer. This is achieved through the adoption of a parallel training approach, wherein the policy of the evading target is simultaneously trained alongside the policy of the tracking observer. While the observer aims to maximize its tracking performance, the target’s goal is to minimize it. An empty environment is considered for this tracking and evasion behavior.

In this thesis, the environment is represented by maps that incorporate urban elements such as buildings and roads. These elements are placed on the map in a manner influenced by the typical appearance of urban maps. Providing agents with lots of diverse map data helps them build more robust, general, and effective policies. While humans can manually create these maps, this process can be labor-intensive and time-consuming, particularly when a substantial amount of data is needed. Instead of relying on manual urban map image production, a technique known as procedural map generation, which randomly generates maps adhering to predefined design procedures, is introduced and employed. In order to procedurally generate maps containing urban map content, this technique utilizes Voronoi diagrams such that the Voronoi polygons and edges are used to construct building and road maps.

Then, the contextual information derived from these maps, denoted as context knowledge, is described. To enable the MARL framework mentioned earlier to harness this context knowledge, the CNN is introduced to process the procedurally generated maps, which are inherently images. Subsequently, the integration of this CNN into the actor-critic RL model, enabling agents to leverage context knowledge in their tracking and evasion tasks, is elucidated.

In the results and evaluations section, various scenarios encompassing different maps and observation spaces are presented. In the escape task, the performance of the target utilizing context knowledge is evaluated, and the impact of context knowledge utilization has been discussed. Collision rate and distance to initial point metrics are presented in order to evaluate the results of each scenario, as well as trajectories of the target are plotted in order to analyze the behavior of the target. The results demonstrated that based only on the context-related observations, in the scenarios based on the basic maps and the Voronoi

diagram-based maps containing only buildings, the target is able to evade buildings and increase its distance to the initial point, while also decreasing the collision rate. This shows that the target is able to utilize the context knowledge in these scenarios, in order to perform the escape task. When the road maps are also considered and integrated into context-related observations, along with the speed adaptation, combining both the state-related and context-related observations leads to enhanced performance compared to utilizing only the context-related observations. One of the possible inferences discussed in the evaluations is that, despite the increased complexity of the observation space due to the road maps, the performance of the target improves due to the valuable information offered by the state-related observations together with the context-related observations.

In the tracking and behavior task, various scenarios are presented and experiments are conducted to explore the impact of context knowledge on agent performance. Observer reward and occlusion rate metrics are used to evaluate the tracking performance of the observer. Furthermore, how the performances of the agents change when the observation space gets more complex is evaluated. For each of the scenarios, the mean reward of the observer consistently increases and converges, over time. In addition, the occlusion rate decreases for all the scenarios. These show that using the context knowledge, the policy of the observer for the tracking task improves over time. Similarly to the escape task, it is also observed in this task that utilizing both state-related and context-related observations results in higher observer rewards compared to providing only context-related observations to the agents. The results and discussion from the escape task are also used to evaluate various results of the experiments in this task.

Two types of future work emerge from this work. The first one is about developing new techniques in the evaluation of the components presented in this work. For instance, gradient analysis of the CNN and the generation of saliency maps may provide insights into the CNN’s efficacy in extracting feature vectors from context knowledge. Saliency maps may shed light on the policies of the agents and their comprehension of the context knowledge, by illustrating the regions on the maps where the agents focus their concentration [107].

The second type of future work aims at improving the tracking and evasion performances of the agents. As context knowledge consists of maps, employing different maps may improve the agent’s understanding of this knowledge. One such approach may involve centered maps, as it was implemented in [74], where each agent remains at the center of their respective maps. In this setup, buildings and other map elements are displayed relative to the agent’s position, eliminating the need for agents to understand their positions on the map. This approach may facilitate better comprehension of map content.

Another future work, involving the evasion behavior of the target, can be assigning the target a task so that instead of moving in a small area, it is encouraged to move away from its position and navigate longer distances, while also considering its evasion task. One of the possible ways may be to assign the target to always move in the opposite direction of the observer’s movement. This also introduces new challenges for the observer, which

may in turn improve its tracking policy.

Future work can also explore modifications to the CNN architecture. This involves adjustments such as adding or removing layers and fine-tuning hyperparameters. Such optimizations in image processing through CNNs can potentially enhance the overall performance of the agents, due to the better representation of the context knowledge by the context-related observations.

A. Appendix

Layer	Input ch.	Output ch.	Size	Kernel size	Activ.	Stride	Padding
$\mathbf{c}_{A,t}$	-	-	(n,n,k)	-	-	-	-
Conv1	k	16	-	(4,4)	ReLU	1	0
Conv2	16	32	-	(4,4)	ReLU	1	0
Conv3	32	256	-	(11,11)	ReLU	1	0
FC1	-	-	256	-	ReLU	-	-
$\mathbf{s}_{A,t}$	-	-	8,10	-	-	-	-
FC2	-	-	256	-	ReLU	-	-
FC3	-	-	256	-	ReLU	-	-
Concat	-	-	512	-	-	-	-
FC4	-	-	3	-	Linear	-	-
FC5	-	-	1	-	Linear	-	-

Table A.1.: RL model parameters

Train batch size	16384	l	10000	hz	0.5
SGD minibatch size	4096	n	[42, 42]	$t_{\max}$	600
SGD iter. number	60	$h_{\min}$	15	$\phi_{A,t}$ limits	$[-\pi, \pi]$
Learning rate	0.001	$h_{\max}$	25	$S_{\max}$	0.03
Gradient clip	5	Basic		Target	
Value func. clip	300	β_b	0.85	α_{slow}	0.3
Entropy coeff.	0.025	Voronoi		$\omega_{\mathcal{T},\max}$	0.349
KL coeff.	0.5	m	25	$h_{\mathcal{T}}$	0
KL target	0.01	Σ	200	$v_{\mathcal{T},t}$	20.0
Use GAE	True	p_{inf}	15	ω limits	$[-0.75, 0.75]$
Gamma	0.925	ψ	0.015	Observer	
Lambda	0.85	λ	128	$\omega_{\mathcal{O},\max}$	0.087
Eval. interval	20	Position		$h_{\mathcal{O}}$	26
Workers	19	κ_p	10	$v_{\mathcal{O},t}$	50.0
				ω limits	$[-0.1, 0.1]$

Table A.2.: PPO parameters Table A.3.: Map parameters Table A.4.: Agent parameters

A. Appendix

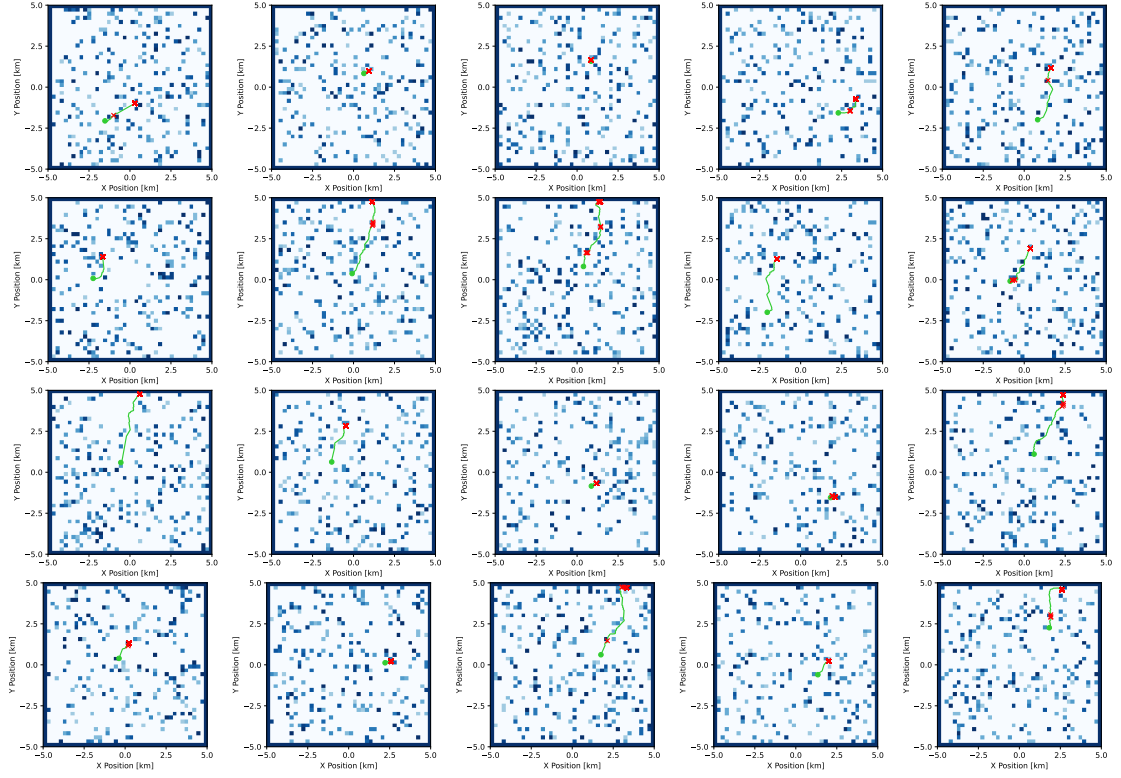


Figure A.1.: More trajectory plots of the scenario A.1.

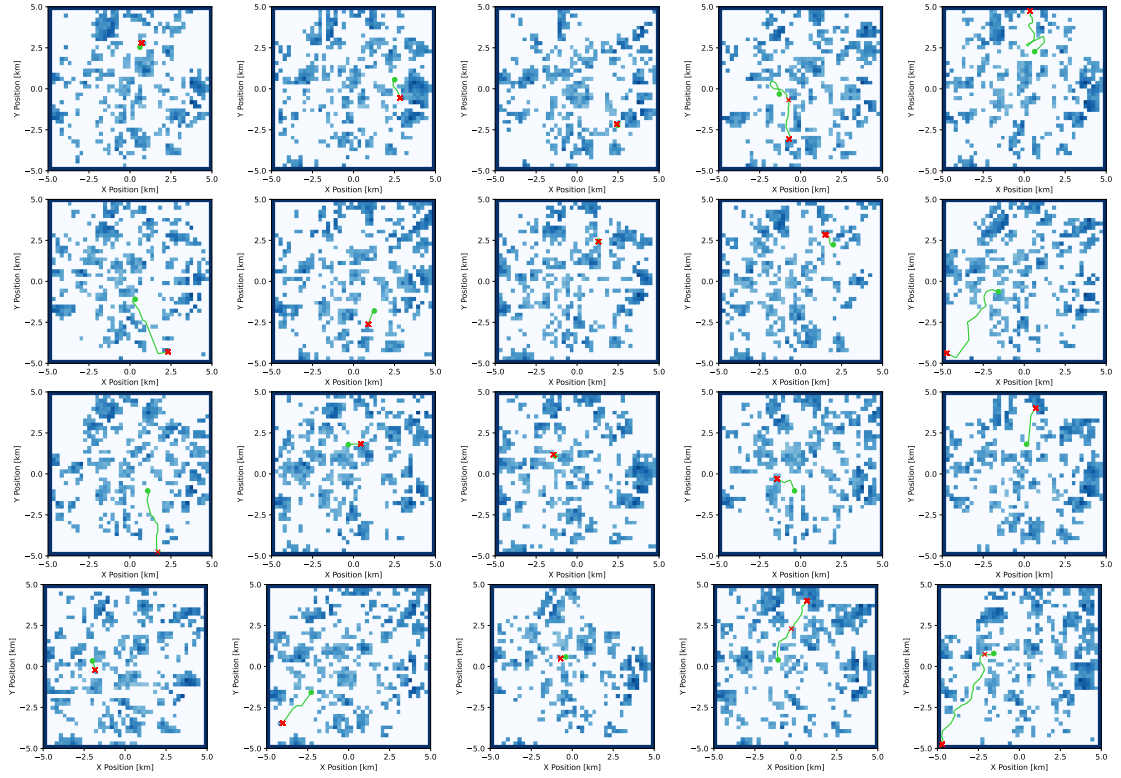


Figure A.2.: More trajectory plots of the scenario A.2.1.

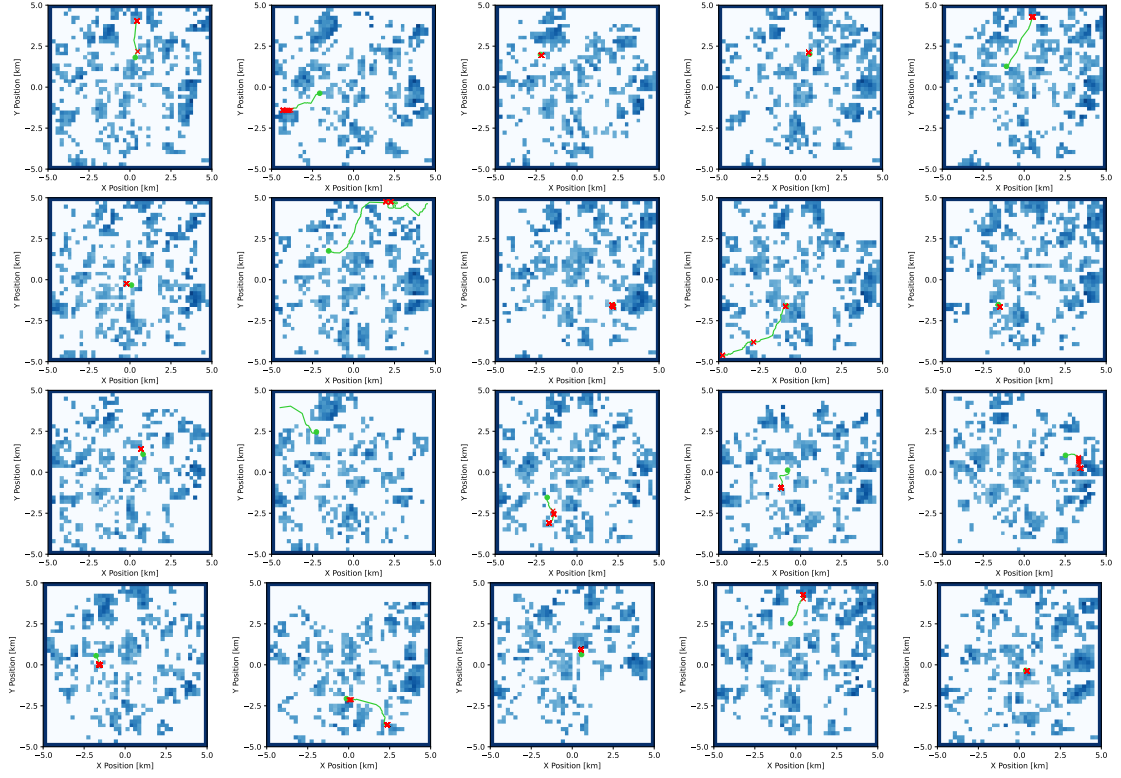


Figure A.3.: More trajectory plots of the scenario A.2.2.

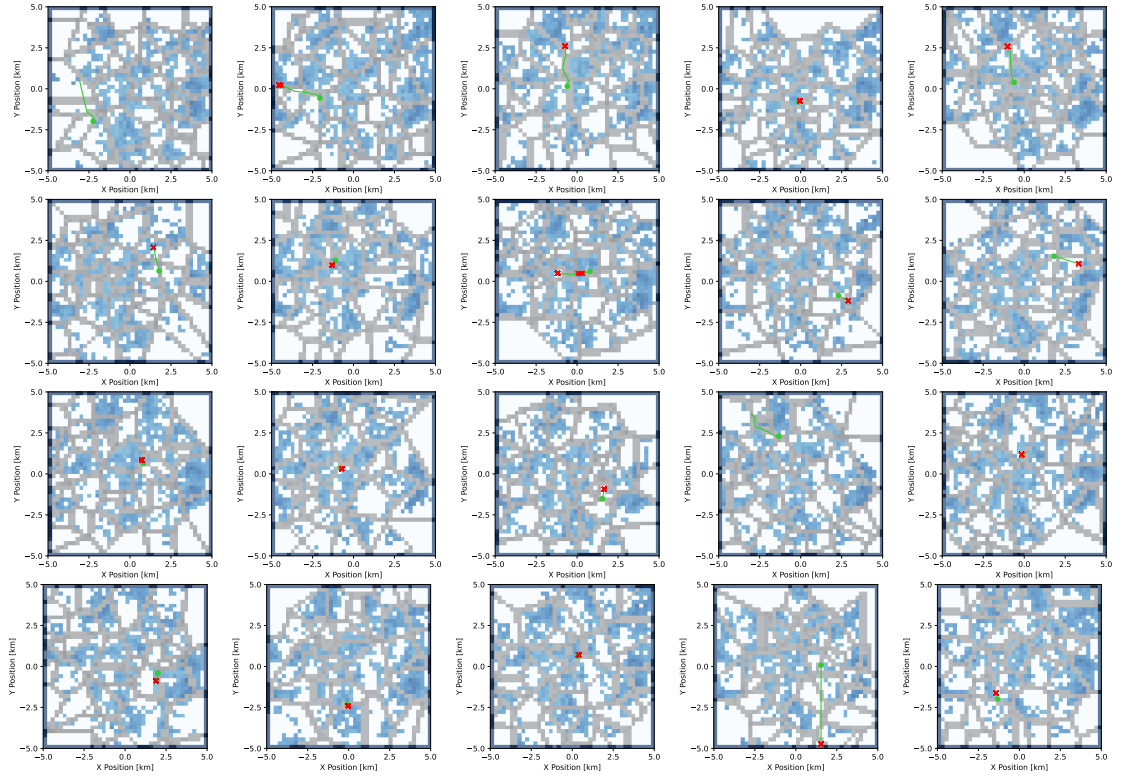


Figure A.4.: More trajectory plots of the scenario A.2.3.

A. Appendix

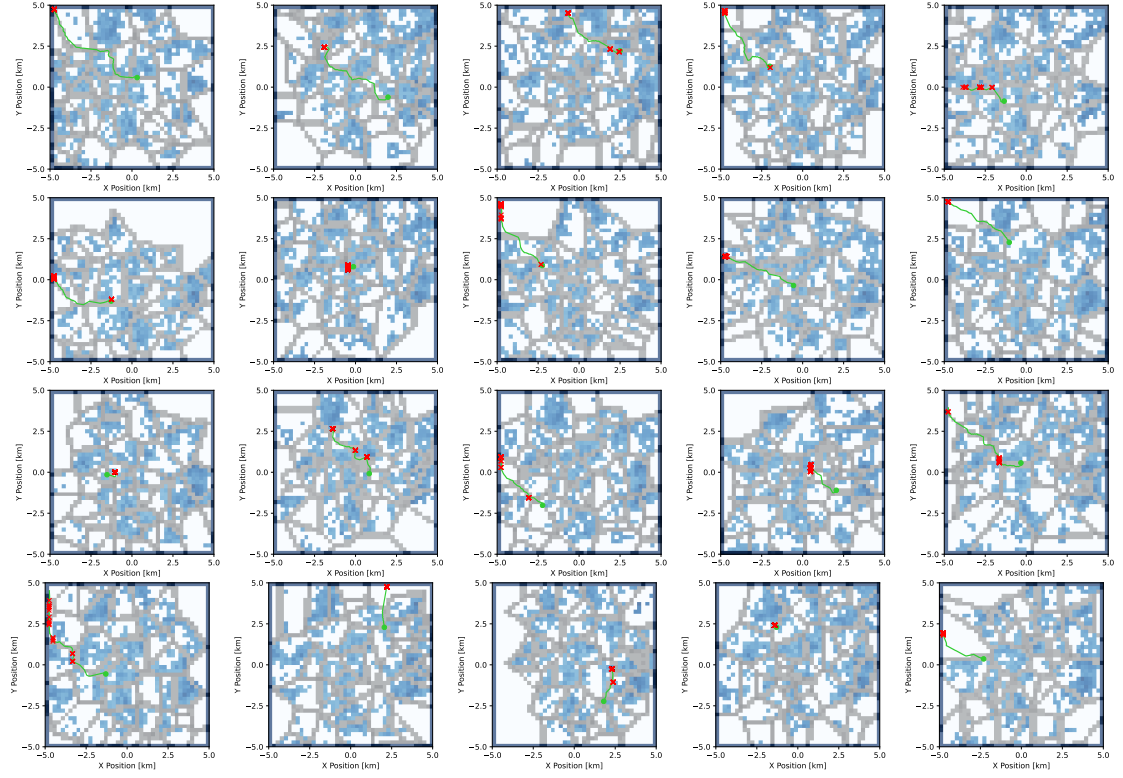


Figure A.5.: More trajectory plots of the scenario A.2.4.

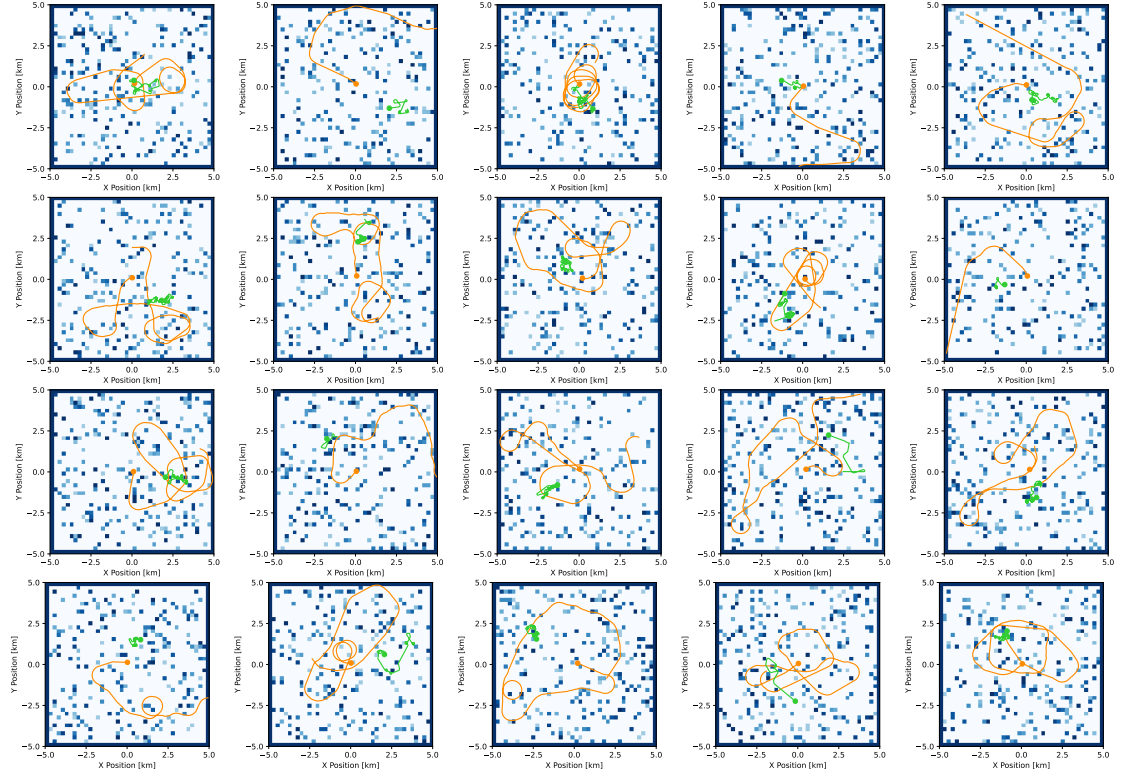


Figure A.6.: More trajectory plots of the scenario B.1.1.

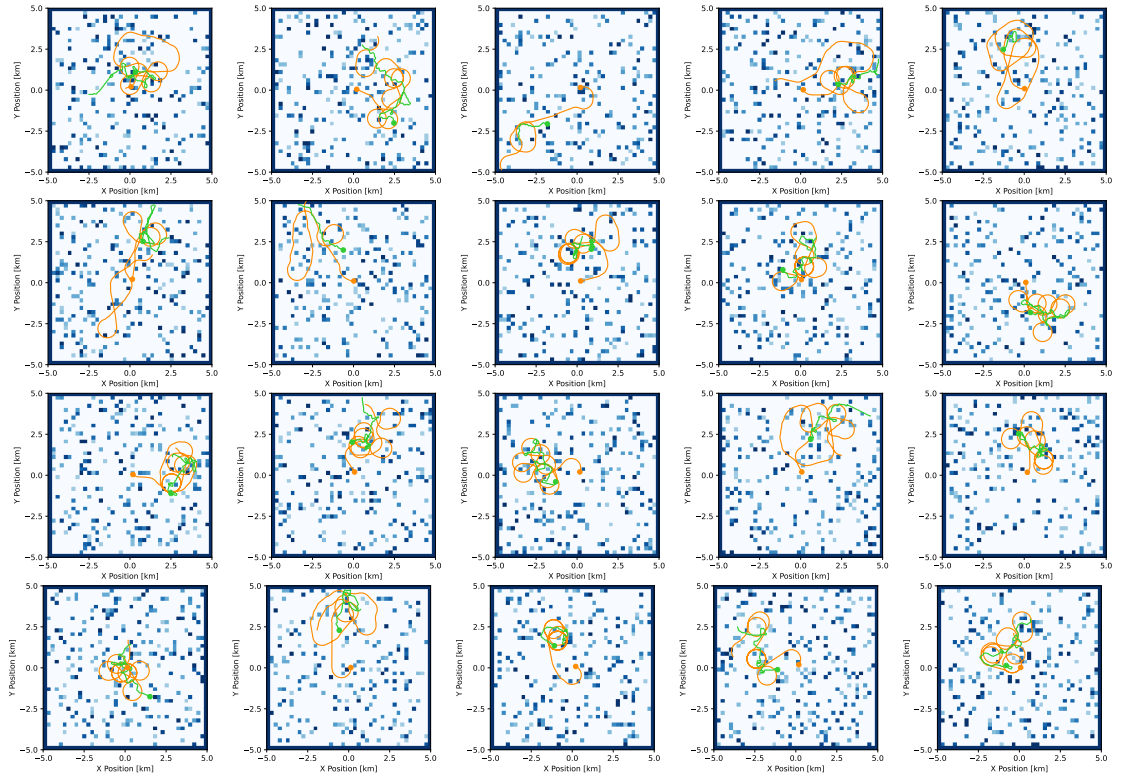


Figure A.7.: More trajectory plots of the scenario B.1.2.

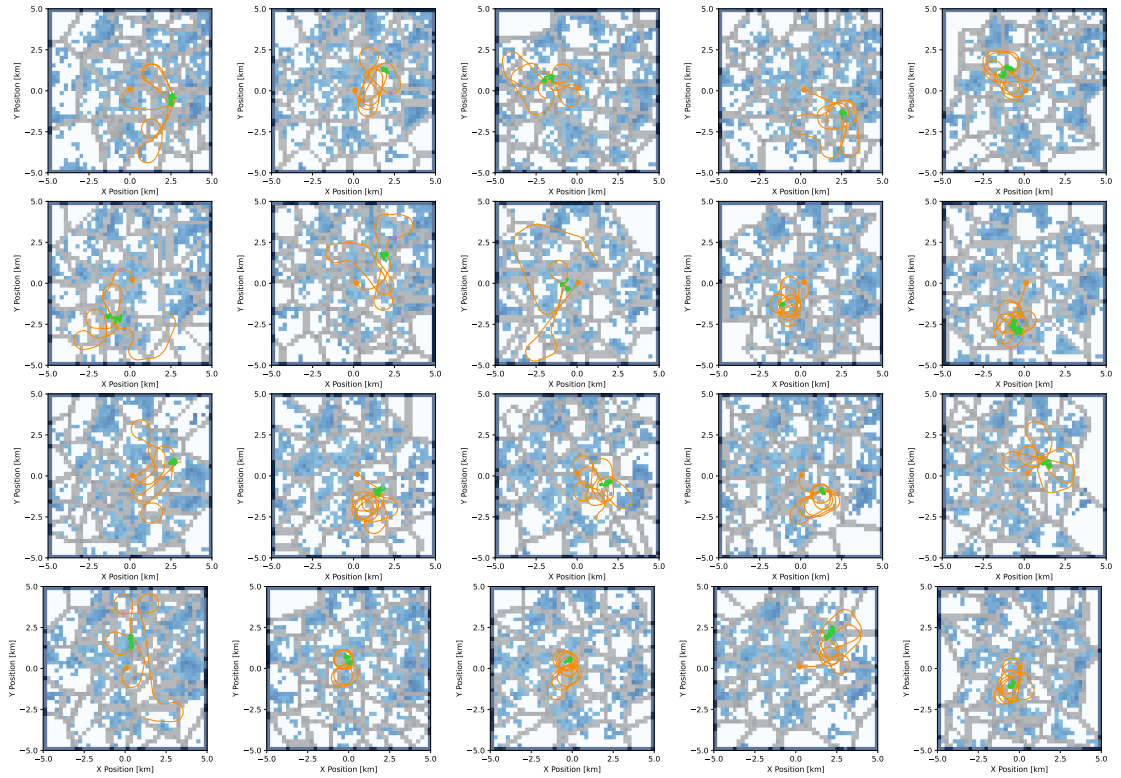


Figure A.8.: More trajectory plots of the scenario B.2.1.

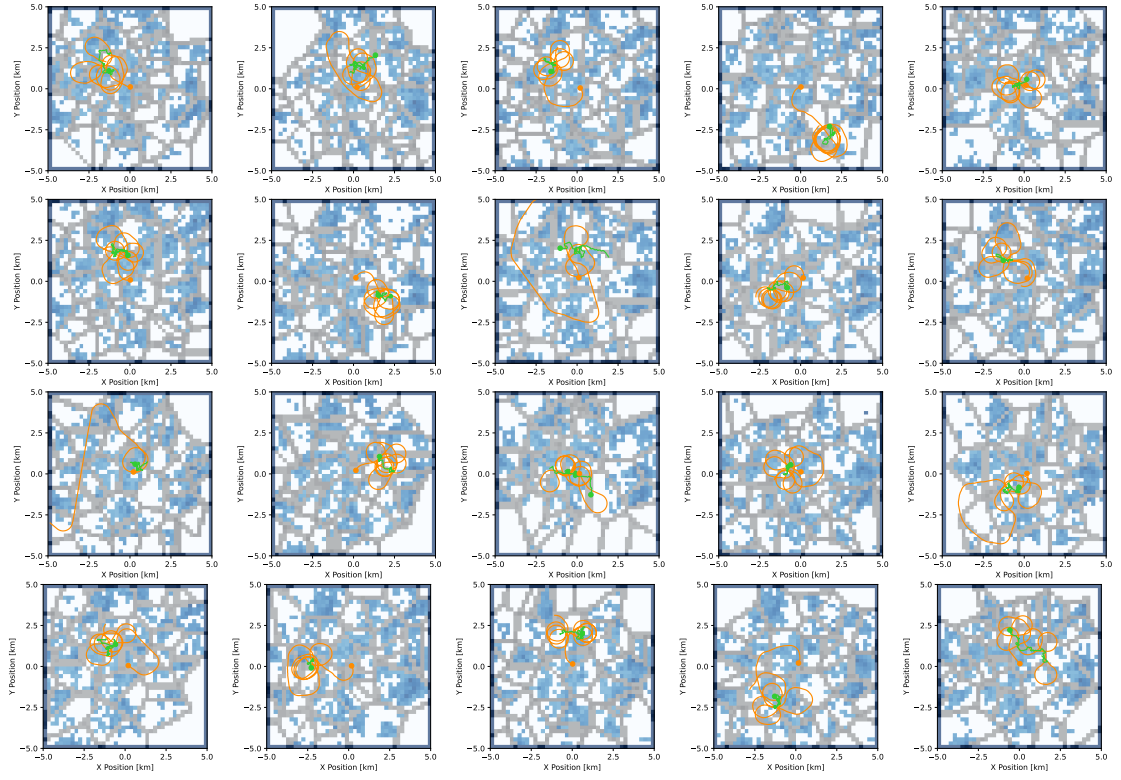


Figure A.9.: More trajectory plots of the scenario B.2.2.

Bibliography

- [1] A. Brandenburger, F. Hoffmann, and A. Charlish, “Co-training an observer and an evading target,” in *2021 IEEE 24th International Conference on Information Fusion (FUSION)*. IEEE, Nov. 2021. [Online]. Available: <https://doi.org/10.23919%2Ffusion49465.2021.9627024>
- [2] P. Bourke, “Polygons, meshes,” <https://paulbourke.net/geometry/polygonmesh/>, (accessed Jul. 16, 2023).
- [3] M. Coskun, O. Yildirim, A. Ucar, and Y. Demir, “An overview of popular deep learning methods,” *European Journal of Technique*, vol. 7, pp. 165–176, Dec. 2017.
- [4] F. Rosenblatt, “The perceptron: a probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, pp. 386–408, 1958.
- [5] X. Du, Y. Cai, S. Wang, and L. Zhang, “Overview of deep learning,” in *2016 31st Youth Academic Annual Conference of Chinese Association of Automation (YAC)*, 2016, pp. 159–164.
- [6] J. Schmidhuber, “Deep learning in neural networks: An overview,” *Neural Networks*, vol. 61, pp. 85–117, Jan. 2015. [Online]. Available: <https://doi.org/10.1016%2Fj.neunet.2014.09.003>
- [7] C. Shao, “A quantum model for multilayer perceptron,” *arXiv: Quantum Physics*, 2018. [Online]. Available: <https://api.semanticscholar.org/CorpusID:119427804>
- [8] J. Yosinski, J. Clune, A. M. Nguyen, T. J. Fuchs, and H. Lipson, “Understanding neural networks through deep visualization,” *CoRR*, vol. abs/1506.06579, 2015. [Online]. Available: <http://arxiv.org/abs/1506.06579>
- [9] J. Fan, C. Ma, and Y. Zhong, “A selective overview of deep learning,” *Statistical Science*, vol. 36, no. 2, pp. 264–290, May 2019.
- [10] F. Li, J. Johnson, and S. Yeung, “Lecture 6: Training neural networks, part I,” http://cs231n.stanford.edu/slides/2017/cs231n_2017_lecture6.pdf, 2017.

- [11] K. O'Shea and R. Nash, "An introduction to convolutional neural networks," *CoRR*, vol. abs/1511.08458, 2015. [Online]. Available: <http://arxiv.org/abs/1511.08458>
- [12] T. Matschak, C. Prinz, F. Rampold, and S. Trang, "Show me your claims and I'll tell you your offenses: Machine learning-based decision support for fraud detection on medical claim data," in *Proceedings of the 55th Hawaii International Conference on System Sciences (HICSS 2022)*, Jan. 2022, pp. 3729–3737.
- [13] A. H. Reynolds, "Convolutional neural networks (CNNs)," <https://anhreynolds.com/blogs/cnn.html>, (accessed Aug. 11, 2023).
- [14] R. Nirthika, S. Manivannan, A. Ramanan, and R. Wang, "Pooling in convolutional neural networks for medical image analysis: a survey and an empirical study," *Neural Computing and Applications*, vol. 34, no. 7, pp. 5321–5347, Apr. 2022. [Online]. Available: <https://doi.org/10.1007/s00521-022-06953-8>
- [15] "CS231n: Convolutional Neural Networks for Visual Recognition," <https://cs231n.github.io/convolutional-networks/>, (accessed Aug. 11, 2023).
- [16] H. Aghdam and E. Heravi, *Guide to Convolutional Neural Networks: A Practical Application to Traffic-Sign Detection and Classification*. Springer, Jan. 2017.
- [17] G. Habib and S. Qureshi, "Optimization and acceleration of convolutional neural networks: A survey," *Journal of King Saud University - Computer and Information Sciences*, vol. 34, no. 7, pp. 4244–4268, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1319157820304845>
- [18] Y. Yu, L. Mroueh, S. Li, and M. Terré, "Multi-agent Q-learning algorithm for dynamic power and rate allocation in lora networks," in *2020 IEEE 31st Annual International Symposium on Personal, Indoor and Mobile Radio Communications*, 2020, pp. 1–5.
- [19] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: A Bradford Book, 2018.
- [20] S. Trimpe and F. Solowjow. (2023) Reinforcement learning and learning-based control. [PowerPoint slides]. Institute for Data Science in Mechanical Engineering, RWTH Aachen University.
- [21] R. Bellman, "A Markovian decision process," *Journal of Mathematics and Mechanics*, vol. 6, no. 5, pp. 679–684, 1957. [Online]. Available: <http://www.jstor.org/stable/24900506>
- [22] Y.-F. Zhang, Q.-F. Zhang, and R.-H. Yu, "Markov property of Markov chains and

- its test,” in *2010 International Conference on Machine Learning and Cybernetics*, vol. 4, 2010, pp. 1864–1867.
- [23] L. Buşoniu, R. Babuška, and B. De Schutter, “Multi-agent reinforcement learning: An overview,” in *Innovations in Multi-Agent Systems and Applications – 1*, ser. Studies in Computational Intelligence, D. Srinivasan and L. Jain, Eds. Berlin, Germany: Springer, 2010, vol. 310, ch. 7, pp. 183–221.
- [24] K. Zhang, Z. Yang, and T. Başar, *Multi-Agent Reinforcement Learning: A Selective Overview of Theories and Algorithms*. Cham, Bavaria, Germany: Springer International Publishing, 2021, pp. 321–384. [Online]. Available: https://doi.org/10.1007/978-3-030-60990-0_12
- [25] K. Jang *et al.*, “Simulation to scaled city: Zero-shot policy transfer for traffic control via autonomous vehicles,” in *Proceedings of the 10th ACM/IEEE International Conference on Cyber-Physical Systems*, ser. ICCPS ’19. New York, NY, USA: Association for Computing Machinery, 2019, p. 291–300. [Online]. Available: <https://doi.org/10.1145/3302509.3313784>
- [26] C. Finn, X. Y. Tan, Y. Duan, T. Darrell, S. Levine, and P. Abbeel, “Learning visual feature spaces for robotic manipulation with deep spatial autoencoders,” *CoRR*, vol. abs/1509.06113, 2015. [Online]. Available: <http://arxiv.org/abs/1509.06113>
- [27] J. Oh, X. Guo, H. Lee, R. L. Lewis, and S. Singh, “Action-conditional video prediction using deep networks in atari games,” *CoRR*, vol. abs/1507.08750, 2015. [Online]. Available: <http://arxiv.org/abs/1507.08750>
- [28] B. C. Stadie, S. Levine, and P. Abbeel, “Incentivizing exploration in reinforcement learning with deep predictive models,” *CoRR*, vol. abs/1507.00814, 2015. [Online]. Available: <http://arxiv.org/abs/1507.00814>
- [29] S. Gu, T. P. Lillicrap, I. Sutskever, and S. Levine, “Continuous deep q-learning with model-based acceleration,” *CoRR*, vol. abs/1603.00748, 2016. [Online]. Available: <http://arxiv.org/abs/1603.00748>
- [30] Y. Bengio, A. Courville, and P. Vincent, “Representation learning: A review and new perspectives,” *IEEE transactions on pattern analysis and machine intelligence*, vol. 35, pp. 1798–1828, Aug. 2013.
- [31] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, “Deep reinforcement learning: A brief survey,” *IEEE Signal Processing Magazine*, vol. 34, no. 6, pp. 26–38, 2017.
- [32] T. Zhao, Y. Wang, W. Sun, Y. Chen, G. Niu, and M. Sugiyama,

- “Representation learning for continuous action spaces is beneficial for efficient policy learning,” *Neural Networks*, vol. 159, pp. 137–152, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0893608022005019>
- [33] OpenAI, “A taxonomy of RL algorithms,” https://spinningup.openai.com/en/latest/spinningup/rl_intro2.html#citations-below, (accessed on Aug. 9, 2023).
- [34] Y. Wang, H. He, and X. Tan, “Truly proximal policy optimization,” in *Proceedings of the 35th Uncertainty in Artificial Intelligence Conference*, ser. Proceedings of Machine Learning Research, vol. 115. PMLR, Jul. 2020, pp. 113–122. [Online]. Available: <https://proceedings.mlr.press/v115/wang20b.html>
- [35] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *CoRR*, vol. abs/1707.06347, 2017. [Online]. Available: <http://arxiv.org/abs/1707.06347>
- [36] J. Hui, “Proximal policy optimization (PPO) explained,” (accessed Sep. 1, 2023). [Online]. Available: <https://jonathan-hui.medium.com/rl-proximal-policy-optimization-ppo-explained-77f014ec3f12>
- [37] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” in *4th International Conference on Learning Representations, ICLR 2016*, Y. Bengio and Y. LeCun, Eds., May 2016. [Online]. Available: <http://arxiv.org/abs/1506.02438>
- [38] Z. Fan, R. Su, W. Zhang, and Y. Yu, “Hybrid actor-critic reinforcement learning in parameterized action space,” *CoRR*, vol. abs/1903.01344, 2019. [Online]. Available: <http://arxiv.org/abs/1903.01344>
- [39] G. Chen *et al.*, “Robot navigation with map-based deep reinforcement learning,” in *2020 IEEE International Conference on Networking, Sensing and Control (ICNSC)*, 2020, pp. 1–6.
- [40] L. Tai and M. Liu, “Towards cognitive exploration through deep reinforcement learning for mobile robots,” *CoRR*, vol. abs/1610.01733, 2016. [Online]. Available: <http://arxiv.org/abs/1610.01733>
- [41] S. A. Fadzli, S. I. Abdulkadir, M. Makhtar, and A. A. Jamal, “Robotic indoor path planning using dijkstra’s algorithm with multi-layer dictionaries,” in *2015 2nd International Conference on Information Science and Security (ICISS)*, 2015, pp. 1–4.
- [42] L. Kavraki, P. Svestka, J.-C. Latombe, and M. Overmars, “Probabilistic roadmaps

- for path planning in high-dimensional configuration spaces,” *IEEE Transactions on Robotics and Automation*, vol. 12, no. 4, pp. 566–580, 1996.
- [43] S. M. LaValle, “Rapidly-exploring random trees: a new tool for path planning,” *The annual research report*, 1998. [Online]. Available: <https://api.semanticscholar.org/CorpusID:14744621>
- [44] Z. Cao, H. Guo, J. Zhang, D. Niyato, and U. Fastenrath, “Finding the shortest path in stochastic vehicle routing: A cardinality minimization approach,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 17, no. 6, pp. 1688–1702, 2016.
- [45] Z. Cao, H. Guo, J. Zhang, D. Niyato and U. Fastenrath, “Improving the efficiency of stochastic vehicle routing: A partial lagrange multiplier method,” *IEEE Transactions on Vehicular Technology*, vol. 65, no. 6, pp. 3993–4005, 2016.
- [46] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [47] D. Zhu and M. Yan, “Survey on technology of mobile robot path planning,” *Control and Decision*, vol. 25, Jul. 2010.
- [48] C. Rösmann, F. Hoffmann, and T. Bertram, “Integrated online trajectory planning and optimization in distinctive topologies,” *Robotics and Autonomous Systems*, vol. 88, pp. 142–153, 2017. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0921889016300495>
- [49] G. Kahn, A. Villafflor, B. Ding, P. Abbeel, and S. Levine, “Self-supervised deep reinforcement learning with generalized computation graphs for robot navigation,” *CoRR*, vol. abs/1709.10489, 2017. [Online]. Available: <http://arxiv.org/abs/1709.10489>
- [50] R.-J. Wai and A. S. Prasetya, “Adaptive neural network control and optimal path planning of UAV surveillance system with energy consumption prediction,” *IEEE Access*, vol. 7, pp. 126 137–126 153, 2019.
- [51] L. A. Zadeh, “Fuzzy logic,” *IEEE Computer*, vol. 21, no. 4, pp. 83–93, 1988. [Online]. Available: <https://doi.org/10.1109/2.53>
- [52] D. Sartori, D. Zou, L. Pei, and W. Yu, “CNN-based path planning on a map,” in *2021 IEEE International Conference on Robotics and Biomimetics (ROBIO)*, 2021, pp. 1331–1338.

- [53] K. Daniel, A. Nash, S. Koenig, and A. Felner, “Theta*: Any-angle path planning on grids,” *Journal of Artificial Intelligence Research*, vol. 39, pp. 533–579, Oct. 2010. [Online]. Available: <https://doi.org/10.1613%2Fjair.2994>
- [54] R. Nian, J. Liu, and B. Huang, “A review on reinforcement learning: Introduction and applications in industrial process control,” *Computers & Chemical Engineering*, vol. 139, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0098135420300557>
- [55] L. Tai and M. Liu, “A robot exploration strategy based on Q-learning network,” in *2016 IEEE International Conference on Real-time Computing and Robotics (RCAR)*, 2016, pp. 57–62.
- [56] L. Tai, G. Paolo, and M. Liu, “Virtual-to-real deep reinforcement learning: Continuous control of mobile robots for mapless navigation,” in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 31–36.
- [57] A. Brandenburger, D. Rodriguez, and S. Behnke, “Mapless humanoid navigation using learned latent dynamics,” in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021, pp. 1555–1561.
- [58] A. Faust *et al.*, “PRM-RL: Long-range robotic navigation tasks by combining reinforcement learning and sampling-based planning,” in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 2018, pp. 5113–5120.
- [59] A. Francis, A. Faust, H.-T. L. Chiang, J. Hsu, J. C. Kew, M. Fiser, and T.-W. E. Lee, “Long-range indoor navigation with PRM-RL,” *IEEE Transactions on Robotics*, vol. 36, no. 4, pp. 1115–1134, 2020.
- [60] T. Chu, J. Wang, and J. Cao, “Kernel-based reinforcement learning for traffic signal control with adaptive feature selection,” in *53rd IEEE Conference on Decision and Control*, 2014, pp. 1277–1282.
- [61] X. Tang, Y. Chai, and Q. Liu, “A 2D UAV path planning method based on reinforcement learning in the presence of dense obstacles and kinematic constraints,” in *2022 IEEE 11th Data Driven Control and Learning Systems Conference (DDCLS)*, 2022, pp. 306–311.
- [62] Z. Yijing, Z. Zheng, Z. Xiaoyi, and L. Yang, “Q learning algorithm based UAV path learning and obstacle avoidance approach,” in *2017 36th Chinese Control Conference (CCC)*, 2017, pp. 3397–3402.
- [63] Y. F. Chen, M. Everett, M. Liu, and J. P. How, “Socially aware motion planning

- with deep reinforcement learning,” in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 1343–1350.
- [64] D. Ernst, P. Geurts, and L. Wehenkel, “Tree-based batch mode reinforcement learning,” *Journal of Machine Learning Research*, vol. 6, pp. 503–556, Apr. 2005.
- [65] S. Adam, L. Busoniu, and R. Babuska, “Experience replay for real-time reinforcement learning control,” *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, vol. 42, no. 2, pp. 201–212, 2012.
- [66] C. Piciarelli and G. L. Foresti, “Drone patrolling with reinforcement learning,” *Proceedings of the 13th International Conference on Distributed Smart Cameras*, 2019. [Online]. Available: <https://api.semanticscholar.org/CorpusID:203415167>
- [67] C. H. Liu, Z. Dai, Y. Zhao, J. Crowcroft, D. Wu, and K. K. Leung, “Distributed and energy-efficient mobile crowdsensing with charging stations by deep reinforcement learning,” *IEEE Transactions on Mobile Computing*, vol. 20, no. 1, pp. 130–146, 2021.
- [68] A. Krizhevsky, I. Sutskever, and G. Hinton, “Imagenet classification with deep convolutional neural networks,” *Neural Information Processing Systems*, vol. 25, Jan. 2012.
- [69] G. E. Dahl, D. Yu, L. Deng, and A. Acero, “Context-dependent pre-trained deep neural networks for large-vocabulary speech recognition,” *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 20, no. 1, pp. 30–42, 2012.
- [70] L. Tai, S. Li, and M. Liu, “A deep-network solution towards model-less obstacle avoidance,” in *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2016, pp. 2759–2764.
- [71] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. A. Riedmiller, “Playing atari with deep reinforcement learning,” *CoRR*, vol. abs/1312.5602, 2013. [Online]. Available: <http://arxiv.org/abs/1312.5602>
- [72] D. Silver *et al.*, “Mastering the game of Go with deep neural networks and tree search,” *Nature*, vol. 529, no. 7587, pp. 484–489, Jan. 2016. [Online]. Available: <https://doi.org/10.1038/nature16961>
- [73] T. Chu, S. Qu, and J. Wang, “Large-scale multi-agent reinforcement learning using image-based state representation,” in *2016 IEEE 55th Conference on Decision and Control (CDC)*, 2016, pp. 7592–7597.
- [74] M. Theile, H. Bayerlein, R. Nai, D. Gesbert, and M. Caccamo, “UAV

- coverage path planning under varying power constraints using deep reinforcement learning,” in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, Oct. 2020. [Online]. Available: <https://doi.org/10.1109%2FIros45743.2020.9340934>
- [75] H. Bayerlein, M. Theile, M. Caccamo, and D. Gesbert, “UAV path planning for wireless data harvesting: A deep reinforcement learning approach,” in *GLOBECOM 2020 - 2020 IEEE Global Communications Conference*. IEEE, Dec. 2020. [Online]. Available: <https://doi.org/10.1109%2Fglobecom42002.2020.9322234>
- [76] H. van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double Q-learning,” *CoRR*, vol. abs/1509.06461, 2015. [Online]. Available: <http://arxiv.org/abs/1509.06461>
- [77] R. Yamashita, M. Nishio, R. K. G. Do, and K. Togashi, “Convolutional neural networks: an overview and application in radiology,” *Insights into Imaging*, vol. 9, no. 4, pp. 611–629, Aug. 2018. [Online]. Available: <https://doi.org/10.1007/s13244-018-0639-9>
- [78] L. Alzubaidi *et al.*, “Review of deep learning: concepts, CNN architectures, challenges, applications, future directions,” *Journal of Big Data*, vol. 8, no. 1, p. 53, Mar. 2021. [Online]. Available: <https://doi.org/10.1186/s40537-021-00444-8>
- [79] J. Freiknecht and W. Effelsberg, “A survey on the procedural generation of virtual worlds,” *Multimodal Technologies and Interaction*, vol. 1, no. 4, 2017. [Online]. Available: <https://www.mdpi.com/2414-4088/1/4/27>
- [80] E. Galin, A. Peytavie, N. Maréchal, and E. Guérin, “Procedural generation of roads,” *Computer Graphics Forum*, vol. 29, no. 2, pp. 429–438, 2010. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1467-8659.2009.01612.x>
- [81] O. Deussen, P. Hanrahan, B. Lintermann, R. Ech, M. Pharr, and P. Prusinkiewicz, “Realistic modeling and rendering of plant ecosystems,” *Proceedings of the 25th Annual Conference on Computer Graphics and Interactive Techniques*, Mar. 2002.
- [82] Y. I. H. Parish and P. Müller, “Procedural modeling of cities,” in *Proceedings of the 28th Annual Conference on Computer Graphics and Interactive Techniques*, ser. SIGGRAPH ’01. New York, NY, USA: Association for Computing Machinery, 2001, p. 301–308. [Online]. Available: <https://doi.org/10.1145/383259.383292>
- [83] M. Gumin, “Wave Function Collapse Algorithm,” Sep. 2016. [Online]. Available: <https://github.com/mxgmn/WaveFunctionCollapse>

-
- [84] J. Parker, “Unity Wave Function Collapse,” <https://selfsame.itch.io/unitywfc>, (accessed Aug. 2, 2023).
- [85] K. S. Emmanuel, C. Mathuram, A. R. Priyadarshi, R. A. George, and J. Anitha, “A beginners guide to procedural terrain modelling techniques,” in *2019 2nd International Conference on Signal Processing and Communication (ICSPPC)*, 2019, pp. 212–217.
- [86] J. Chen, G. Turk, and B. Macintyre, “Watercolor inspired non-photorealistic rendering for augmented reality,” in *Proceedings of the ACM Symposium on Virtual Reality Software and Technology (VRST) 2008*, Oct. 2008, pp. 231–234.
- [87] K. Perlin, “An image synthesizer,” *SIGGRAPH Comput. Graph.*, vol. 19, no. 3, p. 287–296, Jul. 1985. [Online]. Available: <https://doi.org/10.1145/325165.325247>
- [88] S. Gustavson, “Simplex noise demystified,” https://cgvr.informatik.uni-bremen.de/teaching/cg_literatur/simplexnoise.pdf, (accessed Aug. 2, 2023).
- [89] J. Wyatt, A. Leach, S. M. Schmon, and C. G. Willcocks, “AnoDDPM: Anomaly detection with denoising diffusion probabilistic models using simplex noise,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, Jun. 2022, pp. 650–656.
- [90] S. Abraham, “Noise functions,” <https://www.cs.utexas.edu/~theshark/courses/cs354/lectures/cs354-21.pdf>, 2023, (accessed Jul. 7, 2023).
- [91] N. E. Groeneboom and H. Dahle, “Introducing gamer: A fast and accurate method for ray-tracing galaxies using procedural noise,” *The Astrophysical Journal*, vol. 783, no. 2, p. 138, Feb. 2014. [Online]. Available: <https://dx.doi.org/10.1088/0004-637X/783/2/138>
- [92] S. Narvekar, B. Peng, M. Leonetti, J. Sinapov, M. E. Taylor, and P. Stone, “Curriculum learning for reinforcement learning domains: A framework and survey,” *J. Mach. Learn. Res.*, vol. 21, no. 1, Jan. 2020.
- [93] O. Vinyals *et al.*, “Grandmaster level in starcraft ii using multi-agent reinforcement learning,” *Nature*, vol. 575, no. 7782, pp. 350–354, Nov. 2019. [Online]. Available: <https://doi.org/10.1038/s41586-019-1724-z>
- [94] M. Janosov, C. Virágh, G. Vásárhelyi, and T. Vicsek, “Group chasing tactics: how to catch a faster prey,” *New Journal of Physics*, vol. 19, no. 5, p. 053003, May 2017. [Online]. Available: <https://doi.org/10.1088%2F1367-2630%2Faa69e7>
- [95] J. Li, M. Li, Y. Li, L. Dou, and Z. Wang, “Coordinated multi-robot target hunting

- based on extended cooperative game,” in *2015 IEEE International Conference on Information and Automation*, 2015, pp. 216–221.
- [96] L. Angelani, “Collective predation and escape strategies,” *Phys. Rev. Lett.*, vol. 109, p. 118104, Sep. 2012. [Online]. Available: <https://link.aps.org/doi/10.1103/PhysRevLett.109.118104>
 - [97] C. de Souza, R. Newbury, A. Cosgun, P. Castillo, B. Vidolov, and D. Kuli, “Decentralized multi-agent pursuit using deep reinforcement learning,” *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 4552–4559, Jul. 2021. [Online]. Available: <https://doi.org/10.1109%2Ffra.2021.3068952>
 - [98] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, “Multi-agent actor-critic for mixed cooperative-competitive environments,” *CoRR*, vol. abs/1706.02275, 2017. [Online]. Available: <http://arxiv.org/abs/1706.02275>
 - [99] B. Baker, I. Kanitscheider, T. Markov, Y. Wu, G. Powell, B. McGrew, and I. Mordatch, “Emergent tool use from multi-agent autocurricula,” in *International Conference on Learning Representations*, 2020. [Online]. Available: <https://openreview.net/forum?id=SkxpxJBKwS>
 - [100] Y. Bar-Shalom, X. Rong Li, and T. Kirubarajan, *Estimation for Kinematic Models*. Wiley, 2002, ch. 6, pp. 267–299. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/0471221279.ch6>
 - [101] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *CoRR*, vol. abs/1707.06347, 2017. [Online]. Available: <http://arxiv.org/abs/1707.06347>
 - [102] F. Hoffmann, A. Charlish, M. Ritchie, and H. Griffiths, “Sensor path planning using reinforcement learning,” in *2020 IEEE 23rd International Conference on Information Fusion (FUSION)*, 2020, pp. 1–8.
 - [103] W. Koch, R. Mancuso, R. West, and A. Bestavros, “Reinforcement learning for UAV attitude control,” *ACM Transactions on Cyber-Physical Systems*, vol. 3, Apr. 2018.
 - [104] A. B. Arrieta *et al.*, “Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI,” *Information Fusion*, vol. 58, pp. 82–115, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1566253519308103>
 - [105] A. Antipova, *Urban Environment: The Differences between the City in Europe and the United States*. Cham, Bavaria, Germany: Springer International Publishing, 2018, pp. 35–117. [Online]. Available: https://doi.org/10.1007/978-3-319-74198-7_2

- [106] E. Liang *et al.*, “RLlib: Abstractions for distributed reinforcement learning,” in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, J. Dy and A. Krause, Eds., vol. 80. PMLR, Jul. 2018, pp. 3053–3062. [Online]. Available: <https://proceedings.mlr.press/v80/liang18b.html>
- [107] A. Alqaraawi, M. Schuessler, P. Weiß, E. Costanza, and N. Berthouze, “Evaluating saliency map explanations for convolutional neural networks: A user study,” in *Proceedings of the 25th International Conference on Intelligent User Interfaces*, ser. IUI '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 275–285. [Online]. Available: <https://doi.org/10.1145/3377325.3377519>